学校代码: 10286
分 类 号: TP273
密 级: 公开
U D C: 681.5
学 号: 215741



東南大學

电子信息硕士学位论文

多核计算平台温度优化实时任务调度策略研究

研究生姓名: 彭楚君

导 师 姓 名: 莫 磊 副教授

葛泉波 教 授

申请学位类别 电子信息硕士 学位授予单位 东南大学

领域名称 电子信息 论文答辩日期 2024 年 5 月 27 日

研究方向 多核处理器调度 学位授予日期 2024 年 6 月 19 日

答辩委员会主席 陈夕松教授 评 阅 人 YS401240089

YS401240090

2024 年 6 月 3 日

中国知网 https://www.cnki.net

東南大學

电子信息硕士学位论文

多核计算平台温度优化实时任务调度策略研究

专 业 名 称: 电 子 信 息

研究生姓名: 彭 楚 君

导 师 姓 名: 莫 磊 副教授

葛泉波 教 授

中国知网 https://www.cnki.net

RESEARCH ON REAL-TIME TASK SCHEDULING STRATEGIES FOR TEMPERATURE OPTIMIZATION OF MULTI-CORE COMPUTING PLATFORM

A Thesis Submitted to

Southeast University

For the Professional Degree of Master of Engineering

BY

Peng Chu-Jun

Supervised by

Associate Prof. Mo Lei

and

Prof. Ge Quanbo

Software Engineering

Southeast University

June 2024

中国知网 https://www.cnki.net

摘　要

随着网络信息技术的飞速发展，单核处理器系统已经无法满足人们对处理器计算性能的需求。高性能、高集成度、微体积的多核处理器架构已成为主流并得到了广泛的应用。同时，随着半导体制作工艺的不断进步，芯片中的晶体管尺寸不断缩小，单位面积上可以集成更多的处理单元。多核处理器上的功耗密度不断上升，并且使芯片工作时的温度不断上涨。过高的温度不仅会影响任务的可靠执行，还会加速电子元件老化、增加电路永久故障发生的概率、增加系统冷却成本，影响处理器的使用寿命。另外，处理器温度的增加也会增加处理器的静态功耗，而功耗的增加又会增加处理器温度的增涨速度。处理器的温度和功耗之间存在循环相关性，高温和高功耗密度已经成为限制处理器性能发展的重要因素。因此，本文基于多核处理器平台，研究实时任务的调度和温度优化问题。主要工作如下：

1） 针对同构多核平台，考虑满足任务实时性、相关性、可靠性、系统能量和温度约束的系统性能最大化的调度问题，提出了一个两阶段式的静态调度算法。在第一阶段设计了以最大化系统性能，满足任务实时性、相关性、可靠性和系统能量约束的调度方案，引入一个平衡分数来平衡任务在能量消耗大小和执行时间长短间的选择，从而选择任务最优的处理器和执行频率，得到初始的调度；在第二阶段，基于稳态温度模型，计算初始调度方案的峰值温度，对于违反温度阈值的调度，设计了结合 DVFS 和空闲分布策略的温度优化算法。相比于最优解，本章提出的算法平均可减少 99.6% 的求解时间并且保证求得的解与最优解之间的差距在可接受范围内。

2） 研究异构多核平台上实时任务的温度感知调度问题，考虑将调度和温度优化联合设计，提出了基于模型预测控制的动态温度感知调度算法。由于传统的模型预测控制的调度方案存在模型复杂、步骤繁琐只适用于求解小规模的简单问题的不足，本章对温度预测模型、模型预测控制的控制器进行了改进，简化了求解步骤和时间。同时，针对异构平台，在任务调度时加入热均衡策略，在同等参数选择下，选择温度更低的处理器执行任务。相比于同样使用模型预测控制进行调度的一般调度算法，本章提出的算法能大大缩减求解时间，平均可减少 98.23%，并且适用于具有相关性的实时任务，考虑的问题模型更全面。

3） 研究异构多核平台上实时任务的可靠温度感知调度设计，提出了最大化系统平均无故障时间的温度感知调度算法。针对瞬时故障，建立任务可靠性模型；针对永久故障，综合考虑了造成永久故障发生的两种主要机制：电迁移和热循环，并使用当前准确度最高的系统平均无故障时间模型进行联合建模。通过分析温度对处

中国知网　https://www.cnki.net

理器寿命的影响，研究并证明了移除热循环效应的充分条件，并且通过公式推导和计算可得到使任务温度变化范围最小时的最优频率。该最优频率为选择任务执行频率提供参考作用。此外，针对处理器的峰值温度优化问题，通过公式推导和计算，研究了空闲大小可知的任务空闲插入方法。仿真实验表明，相比于使用遗传算法求解系统平均无故障时间最大化的调度方案，本章提出的算法平均可增加 8.38% 的系统平均无故障时间。

关键词： 任务映射，温度，DVFS，可靠性

中国知网 https://www.cnki.net

Abstract

With the rapid development of network information technology, single-core processor systems have become insufficient to meet people's demand for processor computing performance. Multi-core processor architecture with high performance, high integration and micro-size has become the mainstream and has been widely used. At the same time, with the continuous progress of semiconductor manufacturing technology, the size of transistors in the chip is shrinking, and more processing units can be integrated per unit area. The power consumption density of multi-core processors is rising, and the temperature of the chip is rising. Excessive temperature can not only compromise the reliable execution of tasks but also accelerate the aging of electronic components, raising the likelihood of permanent circuit failures and diminishing the service life of processors. Furthermore, elevated processor temperatures contribute to increased static power consumption, thereby exacerbating temperature levels. This cyclic correlation between processor temperature and power consumption has emerged as a significant constraint on processor performance development. Therefore, based on multi-core processor platform, this paper studies the scheduling and temperature optimization strategy of real-time tasks. The main work is as follows:

1) Aiming at the homogeneous multi-core platform, a two-stage static scheduling algorithm is proposed, considering the scheduling problem of maximizing system performance that meets the constraints of real-time, correlation, reliability, system energy and temperature. In the first stage, a scheduling scheme is designed to maximize the system performance and meet the constraints of real-time, correlation, reliability and system energy, and a balance score is introduced to balance the choice between energy consumption and execution time, so as to select the optimal processor and execution frequency of the task and get the initial scheduling. In the second stage, based on the steady-state temperature model, the peak temperature of the initial scheduling scheme is calculated. For the scheduling that violates the temperature threshold, a temperature optimization algorithm combining DVFS and idle distribution strategy is designed. Compared with the optimal solution, the algorithm proposed in this chapter can reduce the solution time by 99.6% on average and ensure that the difference between the obtained solution and the optimal solution is within an acceptable range.

2) The temperature-aware scheduling problem of real-time tasks on heterogeneous multi-core platforms is studied. Considering the joint design of scheduling and temperature

中国知网 https://www.cnki.net

optimization, a dynamic temperature-aware scheduling algorithm based on model predictive control is proposed. Because the traditional scheduling scheme of model predictive control has the shortcomings of complex model and complicated steps, which are only suitable for solving small-scale simple problems, this chapter improves the temperature predictive model and the controller of model predictive control, simplifying the solving steps and time. At the same time, for heterogeneous platforms, thermal equilibrium strategy is added to task scheduling, and the processor with lower temperature is selected to execute tasks under the same parameter selection. Compared with the general scheduling algorithm which also uses model predictive control, the algorithm proposed in this chapter can greatly reduce the solution time by 98.23% on average, and is suitable for real-time tasks with correlation, and the problem model considered is more comprehensive.

3) The reliable temperature-aware scheduling design of real-time tasks on heterogeneous multi-core platforms is studied, and a temperature-aware scheduling scheme is proposed to maximize the MTTF of the system. Aiming at the instantaneous fault, the mission reliability model is established; Aiming at the permanent fault, two main mechanisms causing the permanent fault, electro-migration and thermal cycle, are comprehensively considered, and the joint modeling is carried out by using the system average fault-free model with the highest accuracy at present. By analyzing the influence of temperature on the life of the processor, the sufficient conditions for removing the thermal cycle effect are studied and proved, and the optimal frequency that minimizes the temperature variation range of the task can be obtained through formula derivation and calculation. The optimal frequency plays a reference role in selecting the task execution frequency. In addition, aiming at the optimization of the peak temperature of the processor, the method of task idle insertion with known idle size is studied through formula derivation and calculation. Simulation experiments show that the algorithm proposed in this chapter can increase the MTTF by 8.38% on average compared with the scheduling scheme using genetic algorithm to maximize the MTTF.

Keywords: Task mapping, Temperature, DVFS, Reliability

中国知网 https://www.cnki.net

目 录

第一章 绪论.....1
1.1 研究背景与意义.....1
1.2 国内外研究现状.....2
1.2.1 多核处理器上的调度问题研究.....2
1.2.2 多核处理器上的温度管理问题研究.....4
1.2.3 多核处理器上的可靠性问题研究.....5
1.3 论文主要内容.....7
1.4 本章小结.....9
第二章 处理器温度管理相关工作.....11
2.1 引言.....11
2.2 高温对处理器的影响.....11
2.2.1 高温对系统可靠性的影响.....11
2.2.2 高温对系统能耗的影响.....12
2.2.3 高温对散热系统的影响.....12
2.3 常用处理器温度管理技术.....13
2.3.1 动态电压/频率管理技术.....13
2.3.2 任务迁移技术.....14
2.3.3 空闲分布技术.....14
2.4 本章小结.....15
第三章 同构多核平台上近似实时任务的调度和温度设计.....17
3.1 引言.....17
3.2 系统模型和问题定义.....17
3.2.1 任务模型.....17
3.2.2 处理器模型.....18
3.2.3 可靠性模型.....18
3.2.4 功耗模型.....19
3.2.5 温度模型.....19
3.2.6 问题描述.....20
3.3 算法设计.....23
3.3.1 任务调度优化算法.....24
3.3.2 温度优化算法.....26
3.4 仿真结果.....27
3.4.1 实验设置.....27
3.4.2 温度优化效果对比.....27

中国知网 https://www.cnki.net

3.4.3 与最优解的对比.....29
3.4.4 与其他算法的对比.....30
3.5 本章小结.....33
第四章 异构多核平台上实时任务的温度感知调度设计.....35
4.1 引言.....35
4.2 系统模型和问题定义.....35
4.2.1 处理器模型.....35
4.2.2 MPC 模型.....36
4.2.3 传统 MPC 温度预测模型.....36
4.2.4 改进温度预测模型.....38
4.2.5 问题描述.....39
4.3 算法设计.....40
4.4 仿真结果.....42
4.4.1 实验设置.....42
4.4.2 MPC 模型预测对温度优化的作用.....42
4.4.3 不同预测步数的 MPC 结果对比.....44
4.4.4 与启发式算法的性能对比.....45
4.4.5 和其他算法的对比.....47
4.5 本章小结.....49
第五章 异构多核平台上实时任务的可靠温度感知调度设计.....51
5.1 引言.....51
5.2 系统模型和问题定义.....51
5.2.1 芯片平均无故障时间模型.....51
5.2.2 问题描述.....52
5.3 算法设计.....54
5.3.1 任务温度变化范围对处理器温度变化范围的影响.....54
5.3.2 温度变化范围最小化的任务最优执行频率.....56
5.3.3 基于 MTTF 的可靠温度感知调度算法设计.....57
5.4 仿真结果.....60
5.4.1 实验设置.....60
5.4.2 与 MPCT 算法的性能对比.....60
5.4.3 和其他算法的对比.....62
5.5 本章小结.....64
第六章 总结与展望.....67
6.1 全文总结.....67
6.2 研究展望.....68
参考文献.....69

中国知网 https://www.cnki.net

插图目录

1-1 论文研究内容架构 8

2-1 常用处理器温度管理技术示例 15

3-1 实际应用任务的 DAG 示意图 18
3-2 任务调度算法流程图 26
3-3 不同任务映射方案下的处理器温度 29
3-4 不同任务数下 OPT 与 TATS 的性能对比 30
3-5 对温度不同处理下的峰值温度对比 31
3-6 与其他算法的峰值温度和 QOS 对比 32

4-1 MPC 模型架构 36
4-2 任务深度计算示意图 37
4-3 温度优化效果示例图 43
4-4 不同预测步数下的实验对比 44
4-5 不同实验案例下 QOS 和 makespan 对比 45
4-6 不同实验案例下温度和求解时间对比 46
4-7 与其他算法的峰值温度和 QOS 对比 47

5-1 MRTS 与 MPCT 算法在不同实验案例下峰值温度和求解时间对比 60
5-2 MRTS 与 MPCT 算法在不同实验案例下系统平均无故障时间对比 61
5-3 MRTS 与其他算法在不同实验案例下峰值温度和求解时间对比 63
5-4 MRTS 与其他算法在不同实验案例下系统平均无故障时间对比 63

中国知网 https://www.cnki.net

表格目录

3.1 本章中用到的符号 21
3.2 处理器参数和常量 27

4.1 大核处理器参数和常量 42
4.2 与其他算法的性能对比 48

中国知网 https://www.cnki.net

算法目录

3.1 任务调度优化算法 25
3.2 温度优化算法 28
4.1 基于模型预测控制的温度感知调度算法（MPCT） 41
5.1 基于 MTTF 的可靠温度感知调度算法（MRTS） 59

中国知网 https://www.cnki.net

术语与符号列表

DAG	Directed Acyclic Graph
DPM	Dynamic Power Management
DVFS	Dynamic Voltage and Frequency Scaling
EDF	Earliest Deadline First
EM	Electro-Migration
LSTM	Long Short Term Memory
MILP	Mixed-integer linear programming
MPC	Model Predictive Control
MTTF	Mean Time To Failure
QOS	Quality of Service
SM	Stress Migration
TC	Thermal Cycling
TDDB	Time-Dependent Dielectric Breakdown

中国知网 https://www.cnki.net

第一章 绪论

1.1 研究背景与意义

处理器是计算机的核心，自 1971 年 Intel 发布了全世界第一款微处理器芯片 4004 以来，处理器芯片从结构到工艺经历了飞速发展。高性能、高可靠、低功耗、微体积一直是处理器发展的主要方向[1]。近年来，随着网络信息技术、大数据技术、人工智能技术的发展，人们对处理器的计算能力提出更高的要求。受频率、功耗等因素的制约，单处理器架构的发展出现了瓶颈，由此，高性能、高集成度的多处理器体系架构被提出并得到了广泛的应用[2]。同时，随着半导体制作工艺的改进，芯片中晶体管尺寸缩小，在同一尺寸内可以放置更多的处理单元。这种基于多核处理器结构的芯片通过并行计算，可显著提升计算性能。

然而，高集成度使得多核结构比起以往的单核结构具有更高的功率密度[3]。芯片内部功耗密度的增大，会增加单位面积产生的热量，系统运行时的温度大幅度提高且难以耗散。当芯片过热时会导致芯片老化不均匀，加速芯片的损耗和出现故障的概率，严重影响到系统运行的实时性和可靠性。因此，多核处理器架构中快速增涨的功耗密度和过高的温度造成的热点成为限制多核处理器性能发展的重要因素。为了控制芯片的温度和功耗密度不超过处理器运行的安全范围，高集成度芯片上的处理单元无法全部开启或同时运行在最高频率状态，部分处理单元不得不处于休眠状态以阻止芯片温度的持续增涨，这就是所谓的“暗硅”效应[4]。此外，过高的温度不仅会威胁到芯片的可靠执行，还会增加冷却成本，并且温度越高所需的冷却成本越高；且芯片温度的增加也会增加处理器的静态功耗[5]。因此，如何有效控制芯片温度在近年来受到了广泛地研究和关注。

现有方法主要从硬件提升和软件优化这两个方面对多核处理器系统进行温度控制[6]。硬件方面的提升主要通过加装散热效果更好的散热设备来降低系统的温度，例如散热片、散热风扇、水冷等。但是散热效果更好的散热设备不可避免的会提高系统的整体制造成本、体积以及系统的能耗，从而使得总体成本效益降低[7]。而软件方面的优化主要是指通过程序或者优化方法，在处理器工作过程中，考虑热效应相关的因素，例如工作电压/频率、动态/静态功率、任务分配/调度/迁移等，从而对功耗进行优化管理，降低芯片运行时的温度[8]。通过软件优化方式灵活性高、经济效益好且不需要任何外部设备的辅助，所以基于软件方式的温度控制方法一直是多核处理器研究领域的一个热门研究课题，本课题也计划从软件优化方面入手进行多核处理器的温度管理和控制。

多核处理器已成为现代处理系统的主流解决方案，高效任务管理方案或任务调度算法对其充分发挥高性能和低功耗的优势起着至关重要的作用。多核处理器架构提高了系统的计算性能，但同时也带来了一些挑战，例如多核任务调度、多核并发控制、高效核

中国知网 https://www.cnki.net

间通信等[9]。其中，基于操作系统的任务调度算法对系统整体性能影响重大。由于任务与任务之间、核与核之间在任务数据的处理和传输上存在相关性，多核环境相对于单核环境来说，参数和变量维度更高，因此多核系统调度算法已经成为研究的热门。在不同的应用场景下，调度算法的表现差异很大，所以针对特定的应用环境设计特定的任务调度算法很有必要，如考虑能量优化，满足任务实时性、可靠性约束，降低处理器温度等。

高性能、高并发、高集成度的多核处理器体系架构有着相当广大的应用范围和发展空间，但随之带来的高温度问题也限制了多核处理器的发展[10]。在 2011 年，我国国家自然科学基金“十二五”发展规划明确提出了发展和建设低功耗、高可靠超级计算系统的重要性和必要性；2021 年“十四五”发展规划将“多功能与高效能集成电路”列入优先发展领域。高效能、低功耗、高可靠性成为多核处理器芯片发展的重要方向。因此对多核处理器进行温度控制研究是必不可少的，具有较高实际意义和市场经济价值。本文基于多核处理器计算平台，对满足温度和能量约束的任务调度方案进行讨论和研究，旨在尽可能发挥处理器性能的同时，完成实时任务的温度感知可靠调度。

1.2 国内外研究现状

1.2.1 多核处理器上的调度问题研究

多核处理器上的调度问题受到广泛的关注和研究。相比于单核系统[11–13]，多个处理器[14] 增加了系统的复杂性。多核处理器系统分为同构[15] 和异构[16,17] 两种，同构是指集成的处理器在结构和性能上完全相同，而异构可以将不同功能、属性或大小的处理器集成在一起，可以根据具体需求实现更加丰富的功能。多核处理器上的调度问题是指，对于同构或者异构的多核处理器平台，给定一组具有相关性的任务，如何调度以实现系统性能的最大化目标的问题，且该问题是 NP 难问题。

文献 [18] 针对具有相关性的任务调度问题，提出了一个以最小化系统动态功耗为目标，同时实现任务映射和任务最优频率选择的混合整数线性规划方案，该方案对处理器功耗计算公式进行了线性化。文献 [19] 考虑了相邻处理器对处理器温度的影响，提出了一种将处理器温度计算公式线性化的方案，将以最小化系统峰值温度为目标考虑具有相关性的实时任务的调度问题转换为混合整数线性规划问题。这种通过形式化建模，将多核处理器上的任务调度问题转为如整数规划[20]、动态规划[21]、分支定界[22] 等可以求解的优化问题，可以获得理论上的全局最优解。但是求解所耗费的时间和算力较大，并且随着调度规模的增大，耗费的时间和计算成本成指数级增长。因此，这种形式化建模的方案只适用于求解小规模、简单的调度问题。部分研究者使用随机优化算法来求解调度问题。文献 [23] 提出了一种改进的遗传算法来求解以最小化最大完工时间为目标的调度问题，主要通过使用三种不同的方法产生初始解来提高初始种群的质量和多样性。并且采用人工配对的交叉方法，有效地保留效果好的解和改善效果差的解，从而减少求解的时间并且改善解的质量。针对多核平台上具有相关性任务的调度问题，文献 [24] 提

中国知网 https://www.cnki.net

出了一种基于遗传算法和模拟退火算法的混合元启发式算法，该算法考虑到遗传算法和模拟退火算法分别具有全局和局部搜索倾向，结合使用可以弥补各自的不足。这种使用随机优化算法进行多核处理器上任务调度问题求解的方案权衡了解的最优性和搜索效率，随机遍历解空间中的可行解，并保留当前找到的近似最优解，但是性能不太稳定，有时可能得不到较优的结果。当前，部分研究者借助机器学习等方法，设计智能任务调度方案。针对多核处理器上的任务调度问题，文献 [25] 借助强化学习技术调节任务的执行频率，从而降低整个调度方案消耗的总能耗，达到节能的效果。针对实时系统中的周期任务，文献 [26] 提出了一种基于深度强化学习模型的节能调度方案，该方案使用堆叠自编码器来学习系统中任何状态的每次频率调整的回报值，从而获得更好的挖掘数据间的关联信息，减少更多的系统能耗。这种机器学习等方法求解任务调度问题的智能调度方案可以获得较好的效果和性能，但是一般需要大量的数据和时间进行训练学习，并且适用性不强，当问题模型改变时，需要重新进行训练和测试。此外，对调度问题求解的研究最为广泛的是启发式算法。针对具有相关性的实时任务，文献 [27] 出了一种新的调度算法，该算法根据每个任务的后继任务为每个任务分配不同的关键度，关键度越高的任务越需要优先执行，并且结合关键度重新设置任务的最坏情况下的响应时间界限，从而保证任务的可调度性。为了最小化系统的计算和通信能量并平衡工作负载，文献 [28] 首先提出了一个有效的线性化方案，将任务调度问题转化为一个混合整数线性规划问题来寻找最优解。此外为了减少计算时间，提出了一个三步式启发式算法，包括任务分配、任务执行频率缩放和边缘调度以及通信竞争管理，从而获得更低的系统能耗和更少的通信竞争。这种对多核处理器上的任务调度问题设计启发式求解方案的方法能够在多项式时间内获得可行解但一般不是最优解，具有高效性、自适应性和高度可扩展性，应用最为广泛。

实时系统可分为硬实时系统和软实时系统。硬实时系统有一个刚性的、不可改变的时间限制，它不允许出现任何任务超出时限的错误。超时错误会带来系统损害甚至导致系统失败、或者导致系统不能实现它的预期目标。软实时系统的时限是柔性灵活的，它可以容忍偶然的超时错误。任务超时失败造成的后果并不严重。近似任务可用于建模具有灵活机制的硬实时任务，如信息收集和多媒体计算等[29]。近似任务[30,31] 的周期可分为强制和可选两部分，所有周期都必须在截止时间前完成。强制部分必须执行完全部的周期数，用来保证任务的实时性，可选部分可执行一部分周期数，可选部分可执行的周期数越多，系统能获得的服务质量就越高。近似实时任务可应用于具有弹性机制的实时任务。文献 [32] 基于同构多核平台，提出了一个面向近似实时任务的热感知调度方案，采用动态电源管理（Dynamic Power Management, DPM）结合动态电压/频率调整（Dynamic Voltage and Frequency Scaling, DVFS）的方法降低处理器的温度，必要时使用任务迁移策略。但是这篇文章在调整任务执行频率时没有考虑到任务执行的可靠性问题，任务执行的频率过低，有可能会发生故障导致任务中断，从而影响任务的实时性。针对具有相关性近似任务的调度问题，文献 [33] 提出了一种近似实时任务调度方

中国知网　https://www.cnki.net

法，首先根据系统功率限制、任务的截止时间和任务依赖性的约束，确定每个任务的执行频率。在运行时，通过使用动态电压/频率调整技术和插入空闲来降低处理器的温度。文献 [34] 提出了用于在支持 DVFS 技术的异构多核系统上对实时周期性任务进行能量和温度有效的调度策略 ETA-HP。该策略分为四个阶段，即截止期划分、任务到核分配、温度感知调度和能量感知调度。通过 DVFS 控制芯片温度。当温度过高时，通过减少可选周期数来降低处理器峰值温度。

1.2.2 多核处理器上的温度管理问题研究

在芯片设计和工作过程中，处理器温度管理一直是芯片研究的关键问题之一[35,36]。高效的芯片温度管理机制不仅可以提升系统的计算性能，同时可以延长芯片上集成的各种元器件的寿命[37,38]。国内外研究者都对多核处理器系统的温度管理和温度优化相关问题进行了大量研究。

在温度管理过程中，需要实时获取和预测处理器的工作温度，因此需要对处理器的温度模型进行研究。目前，有关温度模型的研究主要分为基于物理性质的机理建模方法和基于智能算法的数据驱动建模方法两类。国外研究者基于机理建模方法，提出了基于微分方程描述的多核处理器温度模型[39]，该模型主要考虑了在没有散热片情况下处理器与处理器之间、处理器与环境温度之间的热影响。在此基础上，文献 [40] 考虑的温度模型包含了处理器与散热片之间的热传导，但是没有考虑环境温度对处理器温度的影响。针对温度模型不精确的情况，文献 [41] 使用动态电源管理技术，提出了一种基于双层分布式模型预测控制（Model Predictive Control，MPC）的温度感知调度方法：MPC 层用于预测处理器的温度，安全层用于调整处理器的工作频率，并采用分布式方法求解 MPC 优化问题。该方法可在不需要知道精确的硬件模型参数情况下，通过系统状态迭代方式，更新模型参数同时调整处理器的工作频率，从而实现对处理器温度的调整。针对模型未知的情况，文献 [42] 利用长短期记忆（Long Short Term Memory，LSTM）网络，对实际运行时处理器的温度数据进行训练和测试，在执行训练任务的过程中，利用芯片温度传感器记录处理器的温度，并对历史数据进行处理，从而训练得到合适的处理器温度模型。

常用的处理器温度控制的方法有动态工作电压/频率调整（DVFS）、动态电源调整（DPM）、空闲分布、任务迁移、任务重分配等，这些方法经常结合起来使用，对功耗进行优化管理，降低处理器运行时的温度。文献 [11] 设计了工作负载感知电压和频率缩放算法（WA-DVFS），在负载过高时在满足可靠性约束下降低任务的执行频率，并且在处理器温度过高时采用任务重分配策略来控制芯片温度。文献 [43] 提出了一种轻量级热管理机制，该机制能够监测和调节片上多核系统的处理器的工作温度。所提出的方法不仅能够降低处理器内核的工作温度，而且同时还能提高执行应用程序的性能，并减少功耗。在异构多核系统方面，文献 [44] 提出了一种热管理和能源管理机制，该机制通过资源映射和线程分区，对 CPU 和 GPU 的性能进行优化，实现了热梯度的降低和能源效率

中国知网 https://www.cnki.net

的提高。文献 [45] 提出了另一种针对异构多核平台的调度策略，通过将 DPM 和 DVFS 以及全局最早截止时间优先（Earliest Deadline First，EDF）调度算法相结合，极大降低了系统的能耗，并分析了任务调度对功耗的影响，但并未考虑任务迁移。文献 [46] 提出了一个两阶段的启发式算法，考虑系统可靠性，基于任务分配对最大完工时间和温度的影响的分析，确定任务的分配、复制、操作频率和执行顺序，以最小化最大完工时间，同时满足实时、可靠性和温度约束。此外，文献 [47] 提出一种高效准确的异构多核处理器动态温度管理的方法，该方法综合考虑芯片的最高温度以及芯片内部温度差所带来的应力失效问题，通过模型预测空栈计算出芯片的最优运行功耗，并利用 DVFS 技术，结合任务迁移方式，降低芯片温度，使得芯片处于高效安全的运行状态。上述研究通常利用 DVFS 来调整任务执行时间、能量、温度，但未考虑 DVFS 对系统可靠性的影响。当处理单元工作电压/频率比较低的时候，出现暂态错误的概率会上升[48]。

部分研究者考虑到功耗与温度的相关性，从最小化能量上入手降低系统温度[49,50]。文献 [51] 设计了一种两阶段的节能温度感知调度方案，第一阶段将较大功耗因子的任务分配给具有较小功耗因子的处理器降低任务执行的动态功耗；第二阶段对温度过高的地方插入空闲，来限制处理器的温度。文献 [52] 考虑异构多核处理器模型，为每个处理器设置了一个热安全功率密度值，来约束处理器的最高温度，并且结合 DVFS 和任务迁移使系统在满足温度阈值的约束下最大化系统服务质量。文献 [53] 提出了一个名为 Q-Hype 的策略，用于在支持 DVFS 的处理器上调度随机任务。所提出的策略试图最小化最大完成时间并降低内核的能量消耗。同样，文献 [54] 也考虑对处理器设置功率约束来限制处理器的温度，该文献中为每个处理器设置核心级功率约束的热安全功率，旨在温度和可靠性约束下最大化软实时任务的服务质量。同时，改进了任务复制策略，在尽可能节省资源的情况下确保任务的可靠性。这些研究从功率角度优化处理器温度，没有考虑热传递的空间性，忽略相邻处理器的热影响，容易造成没有预料到的热点。

1.2.3　多核处理器上的可靠性问题研究

处理器中出现的故障类型可分为瞬时故障和永久故障。瞬时故障是一种短时间故障，它可以在不损伤硬件电路的情况下自行修复。瞬时故障的发生主要会导致任务执行中断或失败，影响任务执行的可靠性和实时性。与可恢复的瞬时故障不同，永久故障是指电路或电子元件由于制作工艺误差、老化、电迁移等原因，发生的永久性的损伤。永久故障会影响处理器的使用寿命和可靠性。因此，国内外研究者对如何减少故障发生概率，保障系统的可靠性，延长处理器的使用寿命，进行了深入研究。

文献 [55] 使用冗余容错的方法提高系统的可靠性，针对瞬时故障，当任务执行出错时，重新执行该任务；针对永久故障，对硬件模块进行备份。这种硬件备份，每个处理器上需要增加额外的硬件资源，且操作复杂，不具有适用性[56]。另外，部分研究者使用任务复制的方案，通过消耗更多的资源和性能保障任务的可靠执行[57–59]。文献 [60] 使用任务复制保障任务执行的可靠性，当单个处理器出错时使用任务迁移技术，延长系统

中国知网　https://www.cnki.net

的可使用时间。文献 [61] 提出了根据任务级别建立任务副本的任务复制方案，当处理器的空闲时间有限时，根据任务的关键度级别来确定是否需要对任务建立副本，关键度高的任务优先进行任务复制，以在有限的时间和资源的情况下尽可能保证关键度高的任务的可靠执行。这种使用任务复制的方法可有效保障任务的可靠执行，增加系统的可靠性，但同时也增加了系统的能耗，造成一定程度上的冗余和资源浪费。动态电压和频率调节技术是多核系统中最流行和最常用的降低功耗的技术之一。然而，这种技术可能会导致任务可靠性下降，因为处理器上任务的瞬时故障发生的概率与任务的执行频率和任务执行时间有关，任务执行时的频率越低、任务执行时间越长，任务发生瞬时故障的概率也就越高。因此，考虑到任务执行频率对任务可靠性的影响，为了保障任务执行的可靠性，部分研究者结合 DVFS 技术来保障任务的可靠执行[62–64]。文献 [64] 考虑到频率调整对任务可靠性的影响，提出了一种基于强化学习的增强 DVFS 调度方案，该方案可以降低多核嵌入式系统运行时零星任务的功耗，而不降低任务的可靠性。文献 [65] 为每个任务分配最高的频率执行，并且重新执行每个执行失败的任务。该方案尽可能的保障的任务执行的可靠性，但是需要耗费大量的功耗，且没有考虑高频率工作下处理器的温度问题以及对硬件的损伤问题。

电迁移、应力迁移、与时间相关的电介质击穿、热循环等多种与温度相关的机制都有可能导致处理器永久故障的发生。文献 [66] 考虑电迁移对系统寿命的影响，建立系统平均无故障模型，并设计了基于遗传算法的可靠温度感知调度方案，可以延长系统的平均无故障时间。文献 [67] 针对考虑的电迁移和与时间相关的电介质击穿对处理器硬件的负面影响，建立处理器寿命可靠性模型，提出了一个以最小化系统功耗，同时满足处理器温度和寿命可靠性约束的混合整数线性规划问题的调度方案。文献 [68] 同时考虑的瞬时故障和永久故障，使用二叉树分析来优化处理器的可用无故障时间，提出了一种虚拟映射技术，通过两个步骤的编码进行可靠性感知的优化设计。文献 [69] 提出了一种多目标优化技术，通过决定每个任务及其备份到多处理器的分配以及任务和备份的工作电压/频率，以降低瞬时故障的发生率，同时减缓处理器的老化速度以避免永久故障。考虑到瞬时故障和永久故障间的的冲突，并且现有的方法大多是将两类故障作为独立的优化目标或约束进行处理，并且使用两种不同的度量来量化这两种类型的可靠性。文献 [70] 推导出一个新的解析表达式来计算处理器由瞬态故障影响下的系统平均无故障时间。使用这个新公式和一个现有的评估系统平均无故障时间的方法，提出了一个可以同时描述瞬时故障和永久故障相关的系统可靠性的同一衡量标准，提出并解决了考虑永久故障、瞬时故障和吞吐量限制的最大化系统平均无故障时间的问题。上述工作中，大多数研究只考虑了单一故障模型，如单独考虑瞬时故障或单独考虑永久故障，而两者故障之间发生的概率是冲突的，提高任务执行的频率或者使用任务复制可以降低瞬时故障的发生概率，但需要消耗更多的系统能耗从而导致更高的系统温度，给处理器的硬件带来负面影响。因此考虑单一的故障模型，有可能导致另一种故障发生概率提升，从而影响系统的总体寿命和可靠性。

中国知网 https://www.cnki.net

综上所述，国内外学者都对多核处理器系统的任务调度和温度优化问题有着广泛的研究，但是大多数研究仅仅考虑了单一温度优化技术，比如 DVFS、DPM、近似计算、任务迁移等，并未对它们进行综合优化，或者通过控制功耗/功耗密度的方式来间接控制芯片温度。另外部分研究者通过牺牲掉性能的方式来降低芯片的温度，或者考虑的任务模型较为简单，在任务调度时不能保证任务的实时性、相关性和可靠性。

1.3 论文主要内容

当前，多核处理器系统面临三大挑战。第一个是任务调度问题，多核处理器上的任务调度问题已被证明是 NP 难问题。现有的任务调度方案大多存在考虑问题不全面的问题，如不考虑具有相关性的任务、忽略任务可靠性问题或者不考虑系统的能耗或温度问题等。第二个是多核处理器系统的功耗和温度管理问题，由单纯提高核心数量来提升芯片系统性能的发展模式已不可行，温度和功耗密度成为限制处理器性能发展的重要因素，多核处理器系统面临的主要问题在于调和系统计算性能与芯片功耗/温度、可靠性之间的矛盾，不仅要高性能更要高效能。第三个是任务和处理器的可靠性问题，现有的研究很少同时考虑瞬时故障和永久故障。提升任务的执行频率可以降低任务瞬时故障的发生概率，提升任务的可靠性，而频率的提升容易带来更高的温度，从而增加处理器永久故障的发生概率，影响处理器的可靠性。因此，本文基于多核处理器平台，研究具有相关性的实时任务的调度和温度优化问题，并对任务和处理器的可靠性问题进行研究。本文的研究内容的基本框架如图 1-1所示，各章具体内容如下：

1. 第一章介绍了多核处理器系统中的调度和温度控制问题的研究背景和意义，然后调研了国内外研究者在多核处理器上的调度问题、功耗和温度管理问题以及可靠性问题的研究现状，分析了现有工作中存在的不足。最后给出本文的组织架构。

2. 第二章对处理器温度管理的相关工作进行了介绍，从系统可靠性、功耗和冷却系统成本三个方面分析了温度对处理器的负面影响。接着分析了功耗管理与温度管理间的差异，表明处理器功耗管理不能代替温度管理。此外介绍了常用于管理处理器温度的三种技术，并通过文字和图对这些技术的应用场景和缺陷进行了分析。

3. 第三章研究了同构多核平台上近似实时任务的调度和温度优化温度问题，建立了系统模型，描述了研究问题。考虑较为全面的处理器温度计算模型，同时考虑的处理器间的横向热影响和处理器与散热片、散热片与环境间纵向热影响。针对处理器温度与功耗间存在循环相关性的性质，提出了一个两阶段式的静态调度算法。在第一阶段设计了以最大化系统性能，满足任务实时性、相关性、可靠性和系统能量约束的调度方案，引入一个平衡分数来平衡任务在能量消耗大小和执行时间长短间的选择，从而选择任务最优的处理器和执行频率，得到初始的调度；在第二阶段，基于稳态温度模型，计算初始调度方案的峰值温度，对于违反温度阈值的调度，设计了结合

中国知网 https://www.cnki.net

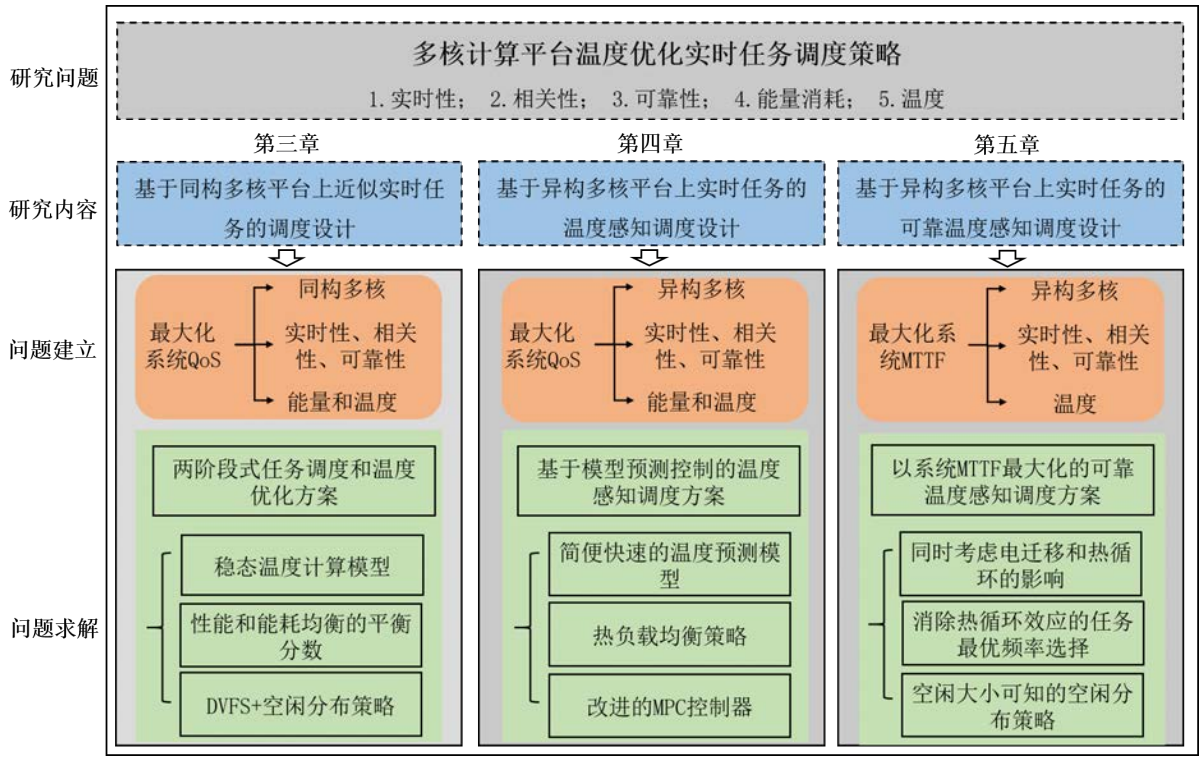


图 1-1 论文研究内容架构

DVFS 和空闲分布策略的温度优化算法。通过仿真实验，提出的算法相比于用求解器求解最优解，极大减少了求解时间，平均可减少 99.6% 的求解时间并且保证求得的解与最优解之间的差距在可接受范围内；并与其他相关的算法进行了实验对比。

4. 第四章研究了异构多核平台上实时任务的温度感知调度问题，提出了基于模型预测控制的动态温度感知调度算法。首先介绍了传统模型预测控制中的温度预测模型，阐述了其所存在的模型复杂、步骤繁琐只适用于求解小规模的简单问题的不足。因此，本章对处理器温度计算模型进行了改进，便于快速估算处理器上的温度。另外相比于传统的将采样时间作为步长的模型预测控制，本章对模型预测控制的控制器进行了改进，将任务深度作为步长，简化了求解步骤和时间。相比于同样使用模型预测控制进行调度的一般调度算法，本章提出的算法能大大缩减求解时间，平均可减少 98.23%，并且适用于具有相关性的实时任务，考虑的问题模型更全面。此外，针对异构平台，在任务调度时加入热均衡策略。最后通过实验对比证明了本章算法在性能和温度优化方面的有效性。

5. 第五章研究了异构多核平台上实时任务的可靠温度感知调度问题，同时考虑永久故障和瞬时故障，提出了以最大化系统平均无故障时间的温度感知可靠调度方案。针对瞬时故障，建立任务可靠性模型；针对永久故障，综合考虑了造成永久故障发生的两种主要机制：电迁移和热循环，并使用当前准确度最高的系统平均无故障时间模型进行联合建模。通过分析温度对处理器寿命的影响，本章研究并证明了移除热循

中国知网 https://www.cnki.net

环效应的充分条件，并且通过公式推导和计算可得到使任务温度变化范围最小时的最优频率。该最优频率在选择任务执行频率时起参考作用。此外，针对处理器的峰值温度的优化问题，通过公式推导和计算，研究了空闲大小可知的任务空闲插入方法，从而使处理器温度过高的时候快速调节到阈值温度以下。最后通过实验对比证明了本章算法在最大化系统平均无故障时间和温度优化方面的有效性。仿真实验表明，相比于使用遗传算法求解系统平均无故障时间最大化的调度方案，本章提出的算法平均可增加 8.38% 的系统平均无故障时间。

6. 第六章对全文工作进行总结并对今后的工作做出了展望。

1.4 本章小结

本章介绍了本文的研究背景和意义，并针对多核处理器上面临的三大挑战，即多核处理器上的任务调度问题、功耗和温度管理问题以及可靠性问题，分析了国内外研究现状和现有工作的不足，提出了本文的研究问题，并对全文组织结构及各章主要内容进行了阐述。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第二章 处理器温度管理相关工作

2.1 引言

在这一章中我们将介绍处理器温度管理相关工作。首先，2.2小节从高温对处理器带来的负面影响出发，介绍了高温对处理器可靠性的影响及作用机制、处理器温度与功耗间的关系以及高温对散热系统的不利影响。其次，2.3小节分析了温度管理与功耗管理的不同，介绍了常用的温度管理技术和他们的应用场景。不恰当的使用这些温度管理技术可能会带来处理器性能上的损失。

2.2 高温对处理器的影响

2.2.1 高温对系统可靠性的影响

随着半导体技术的提升，芯片的特征尺寸持续缩短，功耗密度和集成度的增加使得芯片工作时的平均温度持续上升，带来了任务执行中断、电子元件加速老化、电路故障发生概率增加、处理器寿命减短等问题，严重影响了芯片的性能和可靠性。高温对芯片最严重的的后果就是导致芯片失效，与温度相关的导致电子元件失效的机制主要有电迁移、应力迁移、与时间相关的电介质击穿、热循环和热梯度机制。

电迁移（Electro-Migration, EM）是指在金属导线中的电子在大电流作用下，产生电子迁移的现象[71]。在电路互连时，由于电子的移动，电子与金属离子发生碰撞，金属离子就会沿着电子的方向进行流动。这将会导致两种负面影响：一是移动后的离子会在金属上留下空位，如果某个地方金属离子大量空缺，则金属线可能会在此处断开，发生断路；第二，被移动的离子最终会停留在某个地方，如果在某个地方聚集了大量来自别的地方的金属离子，此处会变宽。当宽到可以触碰到其他线路时，就会出现短路现象，导致电路失效。只要电路处于工作状态，就会发生电迁移现象，且工作的时间越久，电迁移导致的金属线形变的现象就会越明显。而电路温度的升高会加剧电子和离子的运动，从而影响电路的可靠性和寿命。

应力迁移（Stress Migration, SM）是指将电源关闭状态下的芯片放置在一定温度的环境下，一段时间后可能会出现金属线的空洞甚至断开的现象[72]。金属原子在机械应力梯度下会发生移动而导致金属线变形，从而导致电路失效。温度是影响芯片应力迁移的一个关键因素。一般来说，温度越高，应力迁移发生的可能性就越大。这是因为高温会增加材料中缺陷和位错的运动能量，从而使应力迁移更容易发生。与时间相关的电介质击穿（Time-Dependent Dielectric Breakdown, TDDB）是指在栅极加上电压，使器件处于积累状态，经过一段时间后，介质内的电场强度，逐渐增大，氧化层内积聚了缺陷，

中国知网 https://www.cnki.net

氧化膜就会被击穿，造成电路短路和元件损坏[73]。在氧化膜被击穿前的时间就是元件的使用寿命。

过高的处理器温度不是影响芯片寿命和可靠性的唯一原因，热循环和热梯度机制也会增加处理器故障的发生概率。热循环（Thermal Cycling, TC）是指由于电介质材料和金属间膨胀系数间存在较大差异，处理器的温度短时间内发生快速的波动，可能会导致介质裂开或者焊料疲劳等永久损伤[74]。热循环可能是由器件功率的正常上升或者下降引起，也可能是因为处理器上任务负载的变化或者设备的功耗管理策略引起的，比如处理器短时间内在几种功耗模式进行频繁的切换。处理器的温度变化范围越大或越频繁，热循环机制对处理器硬件的负面影响越大。

热梯度是指整个芯片温度在空间上的不平衡，热梯度和局部高温点严重影响封装可靠性。由于多核或众核芯片上负载不均衡，这样温度就会不平衡，引起大的温度梯度。过大的温度梯度会对互连电阻造成影响，从而导致时钟偏差，引起电路中的时序冲突，造成电路故障。

2.2.2 高温对系统能耗的影响

处理器中的功耗主要可分为动态功耗和静态功耗。在芯片中，处理器的功耗过大容易产生过高的温度，而温度的升高又会增加处理器的功耗。温度对功耗的影响主要体现在静态功耗上。处理器的静态功耗主要来源于漏电流，漏电流可表示为：

$$I_{leak,l} = I_s \left[AT^2 e^{(\beta v_l + \theta)/T} + Be^{(\gamma v_l + \delta)} \right], \tag{2.1}$$

其中 v_l 表示处理器的工作电压，I_s 表示在特定温度和电源电压下的基准漏电流，T 表示当前处理器的温度，A、B、β、γ、θ and δ 是由制作工艺决定的常量。从公式 (2.1)中可以看出处理器温度对漏电流的影响量为 $T^2e^{1/T}$。在处理器系统中，温度和功耗存在循环相关性。

2.2.3 高温对散热系统的影响

高温不仅影响芯片的寿命和可靠性，还会增加系统的功耗。随着芯片活跃时平均温度的升高，为了降低处理器的温度，保证处理器的可靠运行，需要配备散热效果更好、使用成本更高的散热系统。芯片常用的散热设备是散热风扇，散热风扇的散热能力可由散热设备与环境之间的热导率 $G_{h,e}$ 衡量，如下：

$$G_{h,e} = h_1 v_1 \left(1 - e^{\frac{h_2 v_1^{h_3} + h_4}{h_1 v_1}} \right), \tag{2.2}$$

其中 h_1、h_2、h_3 和 h_4 是关于芯片制作工艺的常数，v_1 表示散热风扇的转速。从公式 (2.2)可以看出，$G_{h,e}$ 随着散热风扇转速的增大而增大，$G_{h,e}$ 越大表示散热风扇的散热效果越好。而散热风扇的功耗与风扇的转速呈正比关系，即 $P \propto v_1^3$。因此，处理器温度的升高不仅会增加处理器的功耗，还会增加散热系统的功耗和成本。

中国知网 https://www.cnki.net

2.3　常用处理器温度管理技术

过高的温度会给处理器带来可靠性、功耗和性能上的诸多负面影响。为了降低处理器的温度，保证任务的可靠执行，延长处理器的寿命，工业领域和学术领域对处理器上的温度控制问题进行了广泛的研究。首先需要区分温度管理与功耗管理的不同。虽然处理器的温度主要是由处理器功耗引起的，使用的功耗管理的相关技术同样也可以用于管理处理器温度，如动态电压/频率管理技术（DVFS）、时钟门控技术、任务迁移等。但是功耗和温度存在本质上的不同，功耗管理并不能代替温度管理，具体情况表现为以下几点：1）功耗是可以瞬时变化的，但是温度的变化是一个相对缓慢的过程。功耗分布与温度分布并不等价，温度相当于是功耗的累积；2）处理器的温度与功耗密度相关。两个具有相同功耗的处理器，面积较小的处理器上具有更高的温度。相对来说，处理器上的功耗密度越大，温度上升的越快；3）功耗管理也有可能出现没有预料到的高温情况。如对当前温度最高的处理器使用任务迁移策略，将功耗转移到温度更低的处理器，可能出现芯片整体峰值温度反而更高的情况。功耗的转移，并不一定就能减少温度，可能只是热点的转移。此外，由于空间上热量是可以传递的，当某个处理器上功耗为 0 时，也可能会出现温度上升的情况。基于以上情况，功耗管理并不能代替温度管理，通过最小化功耗或者能量来最小化芯片峰值温度的策略，并不一定有效。因此，必须对处理器上的温度控制技术和温度感知调度方案进行深入研究。

2.3.1　动态电压/频率管理技术

原先动态电压/频率管理（DVFS）技术是为了调整处理器上的功耗或者能量，随着处理器上温度过热问题日益突出，DVFS 技术也被广泛应用于处理器温度管理上。DVFS 基于动态功耗模型中芯片功耗与其工作频率和工作电压的平方成正比的工作原理提出的，即 $P_{i,l}^{dyn} = C_{eff} v_l^2 f_l$，其中 C_{eff} 为有效电容，v_l 和 f_l 分别是处理器的工作电压和频率。静态功耗也与处理器的工作电压相关，具体表现为 $P_{i,l}^{sta} = N_g I_{leak,l} v_l$，其中 N_g 表示处理器内部门电路数量的总和，$I_{leak,l}$ 为漏电流。因此，当降低处理器的工作电压时，处理器的动态功耗和静态功耗都可以减少。此外，降低处理器的工作电压后会限制处理器的工作频率，工作频率与工作电压的关系可表示如下：

$$f_l \propto \frac{(v_l - v_t^2)}{v_l}, \tag{2.3}$$

其中 v_t 表示阈值电压。因此，当处理器的工作电压降低时，处理器的频率也会随之降低，处理器获得更低的功耗的同时，处理器执行任务的时间也会随之延长，有可能会影响处理器的性能或任务的实时性。如图 2-1所示，图中的红色虚线表示任务集的截止时间。相比于图中 2-1a，对任务 τ_3 使用 DVFS 技术，任务 τ_3 的执行时间变长，导致任务 τ_5 不能在截止时间前完成。所以在对处理器进行动态电压/频率调整时需要注意处理器上负载的运行情况，具体需要调整多少电压/频率需要进行合理的计算。DVFS 技术的使用

中国知网　https://www.cnki.net

需要软件和硬件的同时支持，如果芯片上没有设置电压频率控制器，就无法使用 DVFS 技术。DVFS 技术一般结合其他技术如任务迁移等来共同进行处理器的温度控制。

2.3.2 任务迁移技术

任务迁移技术是指在多核处理器系统中，将任务从一个处理器迁移到另一个处理器上执行的操作，是调整任务映射和调度的一种方法。对于多核处理器系统，任务迁移技术可在以下几个场景使用：第一种情况是处理器之间任务负载相差较大，造成较大的温度差，将温度过高的处理器上的部分任务迁移到温度较低的处理器上，减少处理器之间的温度不平衡的情况，降低芯片总体的峰值温度。这种为解决多核处理器间由负载不均衡带来的温度不均衡的类似策略还有热均衡策略，两者间的区别是热均衡策略是在调度任务时就考虑处理器的温度情况，将任务调度到温度较低的处理器上执行，而不需要二次迁移；第二种情况是当前处理器上的任务不能有效执行时，如处理器出现故障不能继续执行后续的任务，于是将任务迁移到其他可执行的处理器上；再比如处理器上使用了 DVFS 技术，延长了任务的执行时间，导致处理器上的后续任务不能在截止时间前完成，此时为了保障该任务的实时性，可以将其迁移到其他处理器上；第三种情况是面向异构多核处理器系统，芯片中的处理器存在性能、功耗等方面的差异，可根据任务执行的目标，如获得更低的功耗或更快的完成任务，将任务迁移到对应的处理器上执行。又比如调度具有优先级的任务，当任务优先级发生改变时，有时也有进行任务迁移的需求。图 2-1b和 2-1c展示了使用任务迁移技术中的第二种情况，由于任务 τ_3 的执行时间变长，其后面的任务 τ_5 不能在截止时间前完成，将任务 τ_5 迁移到处理器 p_2 上执行，从而使所有任务都能满足截止时间约束。

2.3.3 空闲分布技术

空闲分布技术是指在当前处理器要执行当前任务时，推迟该任务的开始执行时间，使处理器冷却一段时间，温度下降后再重新工作。抽象来看，空闲分布技术是在实际任务开始执行前插入一个空闲任务，常用于处理器温度过高，可能会引起热故障或者违反温度阈值约束时来降低处理器的温度。空闲分布技术需要特别注意插入的空闲时间的长短，对于对任务实时性要求较高的系统，插入空闲的时长过长，可能会出现任务不能在截止时间前完成的情况。特别是执行具有相关性的任务，推迟当前任务的开始执行时间相当于他和他的后继任务的开始时间都推迟了，有可能会出现满足当前任务执行的实时性但是后继任务违反实时性要求的情况。另外，对于考虑处理器温度阈值约束的系统，如果插入空闲的时长过短，不足以处理器上的温度冷却到安全值，使得空闲时间过后执行任务的处理器温度仍能违反温度阈值约束。因此，在使用空闲分布策略时，需要对出入的空闲时长的大小进行合理设计。图 2-1d展示了使用空闲分布技术的情况，在图 2-1c中，任务 τ_5 的结束时间距离截止时间还有较大差距，可以任务 τ_5 在前插入一段空闲时间，使任务 τ_5 在不违反截止时间约束下优化处理器的温度。

中国知网 https://www.cnki.net

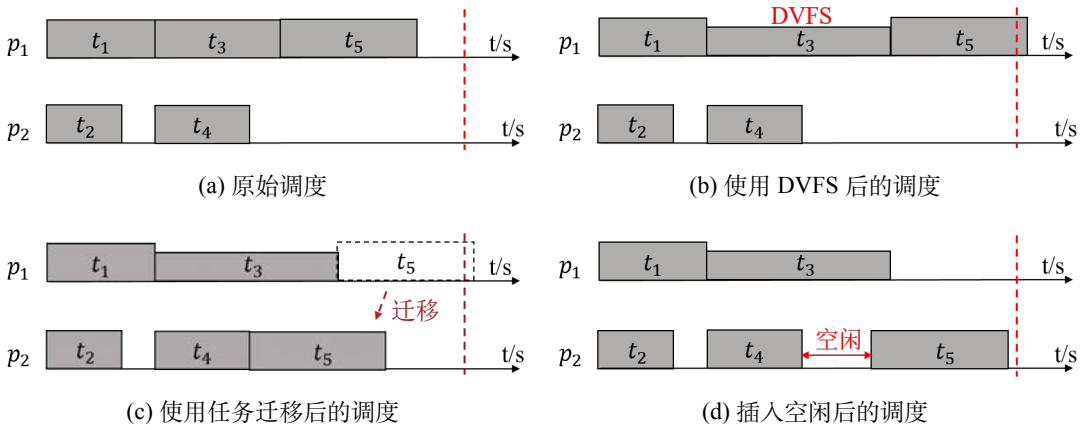


图 2-1 常用处理器温度管理技术示例

2.4　本章小结

本章介绍了温度对处理器硬件的影响以及与温度相关的几种失效机制，过高的温度容易增加处理器永久故障的发生概率，减少处理器的寿命。并且处理器温度的升高也会增加系统的动态功耗和冷却成本。因此，必须进行多核处理器上的温度管理工作。此外分析了功耗管理与温度管理的不同之处，阐述了功耗管理不能代替温度管理的原因。最后对几种常用的温度管理技术进行介绍，并对他们的应用场景和缺陷进行了分析。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第三章 同构多核平台上近似实时任务的调度和温度设计

3.1 引言

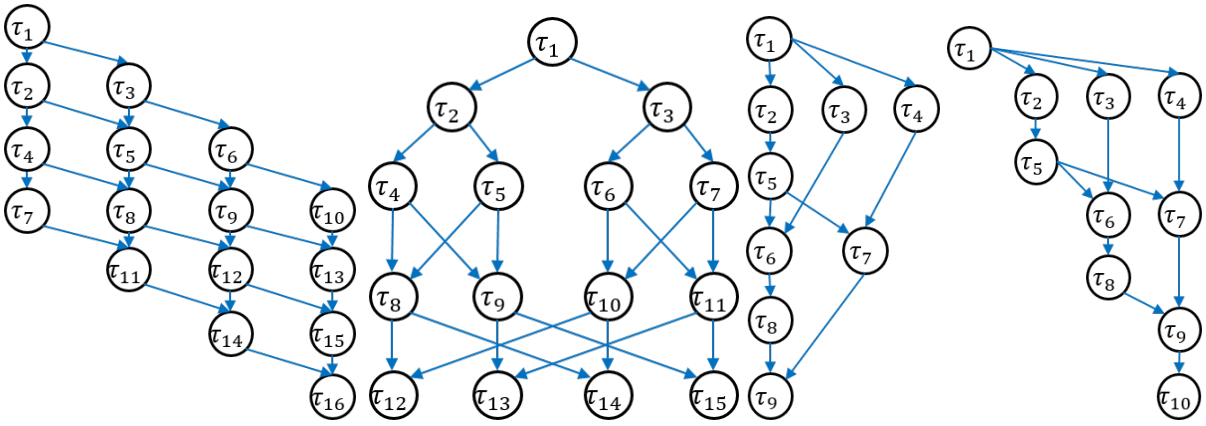
高集成度的多核处理器架构已经成为高性能计算系统的主流架构。然而随着暗硅效应的出现，通过粗略的增加处理器核心数量来增加处理器性能的方案已不可行，过高的温度和功耗密度已经成为限制处理器性能发展的重要因素。为了描述系统性能，考虑具有可选周期的近似任务，使用任务执行的可选周期数来衡量系统的性能。要想获得更好的性能，需要使用更高的频率执行更多的周期数，而这同时也会带来更多的能量消耗和温度。对于处理器上的温度计算，考虑较为全面准确的温度计算模型，同时考虑了处理器间横向热影响以及处理器与散热片、散热片与环境间的纵向热影响。此外考虑到任务执行过程中可能会发生瞬时故障，造成任务执行中断或失败，建立了任务可靠性模型，保障任务的可靠执行。因此，基于多核处理器平台，本章研究的问题是以最大化系统性能为目标，满足任务实时性、相关性和可靠性约束，同时满足系统能量和温度约束的调度方案设计。为了实现这一目标，本章将调度和温度优化分开进行设计，本章的工作包括：1）为了简化处理器温度计算时间和步骤，使用稳态温度模型进行处理器温度的计算，并将多核处理器的温度转为关于频率的矩阵形式的表达；2）在选择任务执行的处理器和频率时，为了均衡时间和能量消耗，制定了一个平衡分数，从而为任务选择更优的处理器和频率；3）结合 DVFS 和空闲插入策略，设计保证任务实时性和相关性的温度优化方案；4）通过仿真实验对比，展现本章提出的调度方案的性能。

3.2 系统模型和问题定义

3.2.1 任务模型

文考虑具有 N 个相关性的实时任务 $\{\tau_1, \ldots, \tau_N\}$ 的任务集 $\mathcal{T}$，任务的相关性由有向无环图（Directed Acyclic Graph, DAG）[75] 表示。图 3-1展示了四个实际应用的任务 DAG 图，分别表示为拉普拉斯变换、快速傅立叶变换、Lu 分解法和高斯消元法。DAG 图包括节点和连接线两部分，每个节点表示一个任务，每条线表示一项依赖关系。被箭头指向的任务表示具有先继任务，只有当任务的所有先继任务执行完毕，任务才可以开始执行。所有任务都考虑的是近似任务模型，每个任务的执行周期都有强制 M_i 和可选 O_i 两部分[29]。强制部分 M_i 是必须执行的部分且是固定不可变动的，用来保证任务的可靠执行；可选部分 $o_i \in [0, O_i]$ 可选择执行部分周期，用来衡量系统的服务质量，可选部分执行的周期数越多，系统的服务质量越高（Quality of Service, QOS）。所有的周期数都需要在任务的截止时间前执行完成。

中国知网 https://www.cnki.net



(a) 拉普拉斯变换　(b) 快速傅立叶变换　(c) Lu 分解　(d) 高斯消元

图 3-1 实际应用任务的 DAG 示意图

3.2.2 处理器模型

考虑具有 M 个处理器 $\{p_1,\ldots,p_M\}$ 的同构多核平台，每个处理器上同一时间只能执行一个任务且任务间不可抢占。处理器支持 DVFS 技术。设定每个处理器上可实现多个等级的电压/频率 V/F。每个处理器有 L 个电压/频率等级，即 $\mathcal{VF} \triangleq \{(v_1,f_1),\ldots,(v_L,f_L)\}$。为了确定任务的执行频率，引入二元变量 $Y_{i,l}$。当任务 τ_i 的频率和电压选择为 (v_l,f_l) 时，令 $Y_{i,l}=1$，其他情况 $Y_{i,l}=0$。

3.2.3 可靠性模型

处理器温度和频率调整都会影响任务执行的可靠性。为了衡量温度变化和电压/频率调整对可靠性的影响，需要建立任务可靠性模型[76]。处理器温度过高容易造成集成电路发生永久故障（permanent failure）。而通过 DVFS 调整工作电压/频率，当任务执行频率越低，执行时间会越长，发生瞬时故障（transient failure）的概率也会越高。瞬时故障是指受电磁波干扰或者宇宙辐射影响，电路中晶体管或逻辑状态发生的状态异常的现象。瞬时故障是一种短时间故障，它可以在不损伤硬件电路的情况下自行修复。考虑系统中瞬时故障的发生，通常采用泊松故障概率模型来描述任务执行的可靠性，当处理器以 f_l 频率去执行周期为 c_i 的任务 τ_i 时，任务执行结果可靠性可用以下公式来衡量：

$$r_{i,l}=e^{-\eta\times\frac{c_i}{f_l}\times 10^{\frac{s(f_{\max}-f_l)}{f_{\max}-f_{\min}}}}, \tag{3.1}$$

其中 η 表示最大故障参数，s 表示频率缩放灵敏度的常数，$f_{\max}=\max_{\forall l}\{f_l\}$ 和 $f_{\min}=\min_{\forall l}\{f_l\}$ 分别表述处理器上的最大和最小频率。为了简便计算，将原公式进化简。因为 $\frac{(f_{\max}-f_l)}{f_{\max}-f_{\min}}\le 1$，有 $10^{\frac{s(f_{\max}-f_l)}{f_{\max}-f_{\min}}}\le 10^s$。因此，任务 τ_i 的可靠性计算公式可以简化为：

$$r_i=\sum_{l=1}^{L}\left(Y_{i,l}e^{-\eta\times\frac{c_i}{f_l}\times 10^s}\right), \tag{3.2}$$

中国知网 https://www.cnki.net

其中 $\frac{c_i}{f_l}$ 可表示任务的执行时间，任务的可靠性公式简化为关于任务执行时间和任务频率选择的表达式。

3.2.4　功耗模型

多核处理器上的温度控制问题离不开功耗建模。系统的功耗模型由动态功耗 P_{dyn} 和静态功耗 P_{sta} 组成[50]。动态功耗可表示为 $P_{i,l}^{dyn} = C_{eff}v_l^2 f_l$，其中 C_{eff} 为有效电容，v_l 为处理器的供电电压。动态功耗是任务在处理器上执行时消耗的能量，与任务的执行频率密切相关。静态功耗由漏电流产生，处理器在开机状态就会产生静态功耗，静态功耗公式为 $P_{i,l}^{sta} = N_g I_{leak,l} v_l$，其中 N_g 表示处理器内部门电路数量的总和，$I_{leak,l} = I_s[AT^2 e^{(\beta v_l+\theta)/T} + Be^{(\gamma v_l+\delta)}]$，并且 I_s 是在特定温度和电源电压下的基准漏电流，T 表示当前处理器的温度，A、B、β、γ、θ 和 δ 是由制作工艺决定的常量。

由于原静态公式参数较多且是关于电压 v_l 和温度 T 指数计算形式的表达，形式复杂，不便于直接用于计算，因此采用简化后的线性静态功耗公式[77]。静态功耗可被简化为关于处理器温度 T 的线性模型，$P_{i,l}^{sta} = K_1(v_l)T + K_2(v_l)$，其中 $K_1(v_l)$ 和 $K_2(v_l)$ 在给定的 v_l 下是常数，具体值由处理器的具体参数决定。因此，任务 τ_i 在处理器 p_n 上以频率 f_l 执行时的总功耗为:

$$P_{i,l} = P_{i,l}^{dyn} + P_{i,l}^{sta} = C_{eff}v_l^2 f_l + K_1(v_l)T + K_2(v_l), \tag{3.3}$$

且系统执行完 DAG 任务后的总能耗为所有任务的功耗和执行时间的乘积之和，系统的总能耗为:

$$E_{tol} = \sum_{i=1}^{N}\sum_{l=1}^{L} Y_{i,l}[C_{eff}v_l^2 c_i + (K_1(v_l)T + K_2(v_l))c_i/f_l]. \tag{3.4}$$

3.2.5　温度模型

在多核处理器系统中，要准确计算处理器 p_n 的温度不仅要考虑其功耗和自身执行任务的影响，还必须考虑相邻处理器横向热传导的影响以及处理器与散热片间、散热片与环境间的纵向热传导的影响。在这里，本章采用比较成熟的热 RC 模型[51]。$T_n(t)$ 表示在 t 时刻，处理器 p_n 的温度，阈值温度为 T_{thr}。

$$C_k\frac{dT_n(t)}{dt} + \sum_{m=1}^{M} G_{n,m}(T_n(t) - T_m(t)) + G_{n,h}(T_n(t) - T_h(t)) = P_{i,l}, \tag{3.5}$$

$$C_h\frac{dT_h(t)}{dt} + \sum_{g=1}^{Z} G_{h,g}\left(T_h(t) - T_g(t)\right) + G_{n,h}(T_n(t) - T_h(t)) = G_{h,e}(T_e - T_h(t)), \tag{3.6}$$

其中 $T_h(t)$ 表示在 t 时刻散热片 h 的温度，T_e 表示环境温度，C_n 表示处理器的热电容。$G_{n,m}$ 是处理器 p_n 和 p_m 之间的热导率:

$$G_{n,m} = \frac{W_{nm}th_{si}K_{si}}{L_{nm}}, \tag{3.7}$$

中国知网　https://www.cnki.net

其中 W_{nm} 表示处理器 p_n 和 p_m 之间的长度，L_{km} 表示处理器间的中心距离。处理器 p_n 与散热片 h 间的热导率表示为 $G_{n,h}$:

$$G_{n,h} = \frac{area_{pn}}{R_{chip} area_{chip}}, \tag{3.8}$$

其中 $area_{pk}$ 和 $area_{chip}$ 分别表示处理器 p_n 和整个芯片的面积。$R_{chip} = \frac{th_{si}}{K_{si} area_{chip}}$，且 th_{si} 和 K_{si} 分别表示芯片的制作材料硅的厚度和热导率。$G_{h,g}$ 表示散热片 h 与散热片 g 之间的热导率:

$$G_{h,g} = \frac{W_{hg} th_{cop} K_{cop}}{L_{hg}}, \tag{3.9}$$

其中 W_{hg} 和 L_{hg} 分别表示散热片 h 与散热片 g 之间的长度和中心距离，th_{cop} 和 K_{cpo} 分别表示散热片的制作材料铜的厚度和热导率。散热片 h 与环境间的热导率表示为 $G_{h,e}$:

$$G_{h,e} = \frac{area_h}{R_{HS} area_{heatsi}}, \tag{3.10}$$

其中 $area_h$ 和 $area_{heatsi}$ 分别表示散热片 h 与整个散热元件的面积。$R_{HS} = \frac{T_{active} - T_e}{P_{chip}} - R_{chip}$，且 T_{active} 表示所有处理器处于活跃状态时芯片的平均温度，P_{chip} 表示芯片的总功耗。

3.2.6 问题描述

基于以上模型，本章的研究内容是在给定一组具有 N 个相关性的实时近似任务和 M 个同构处理器上，在满足任务的实时性、相关性和可靠性的约束下，保证系统的能量阈值约束和温度阈值约束，使系统的服务质量 QOS 最大化。因此，本章需要确定的变量有：1）任务到处理器的映射，2）任务执行的频率选择，3）任务可选周期的周期数，4）任务执行的开始时间，5）任务执行的先后顺序。另外，令集合 $\mathcal{M} = \{1, ..., M\}$，$\mathcal{T} = \{1, ..., N\}$，$\mathcal{L} = \{1, ..., L\}$。表 3.1汇总了本章引入的符号。

1. 任务映射：本文考虑非抢占式任务，且每个任务同一时间只能在一个处理器上执行，因此引入二元变量 $X_{i,n}$，当 $X_{i,n} = 1$ 时，表示任务 τ_i 在处理器 p_n 上执行，约束表示如下：

$$\sum_{n=1}^{M} X_{i,n} = 1, \ \forall i \in \mathcal{T}. \tag{3.11}$$

2. 任务频率选择：由于本文考虑的是任务级的 DVFS，也就是说，每个任务只能选择一个 V/F 等级来执行，且在执行过程中不会发生跳频现象，因此在任务的频率选择方面需要加入如下约束：

$$\sum_{l=1}^{L} Y_{i,l} = 1, \ \forall i \in \mathcal{T}. \tag{3.12}$$

3. 任务可选周期的周期数：本文考虑的是近似任务模型，可选周期数的多少可衡量系统服务质量的好坏。为了更好的描述任务的可选部分执行情况，引入连续变量 λ_i，且

中国知网 https://www.cnki.net

表 3.1 本章中用到的符号

名称	含义
N	任务数量
M	处理器数量
L	电压/频率等级数量
τ_i	第 i 个任务
p_n	第 n 个处理器
(v_l, f_l)	第 l 个电压/频率等级
$P_{i,l}$	第 i 个任务在第 l 个电压/频率等级下的功率
E_{tol}	执行 N 个任务所需的总能量
r_i	第 i 个任务的可靠性
λ_i	第 i 个任务的可选周期率
ts_i	第 i 个任务的开始时间
te_i	第 i 个任务的结束时间
d_i	第 i 个任务的截止时间
H	整个 DAG 任务的截止时间
T_n	第 n 个处理器上的温度
$X_{i,n}$	$= \begin{cases} 1 & \text{如果任务 } \tau_i \text{ 映射到处理器 } P_n \text{ 上} \\ 0 & \text{其他} \end{cases}$
$Y_{i,l}$	$= \begin{cases} 1 & \text{如果任务 } \tau_i \text{ 以频率 } f_l \text{ 执行} \\ 0 & \text{其他} \end{cases}$
$R_{i,j}$	$= \begin{cases} 1 & \text{如果任务 } \tau_j \text{ 依赖于任务 } \tau_i \\ 0 & \text{其他} \end{cases}$
$O_{i,j}$	$= \begin{cases} 1 & \text{如果任务 } \tau_i \text{ 在任务 } \tau_j \text{ 之前执行} \\ 0 & \text{其他} \end{cases}$

$\lambda_i \in [0,1]$。任务 τ_i 的可选部分的执行周期数可表示为 $o_i = \lambda_i O_i$。可选周期的周期数约束表示如下：

$$\lambda_i \in [0,1],\ \forall i \in \mathcal{T}. \tag{3.13}$$

4. 实时性约束：所有任务的所有周期数都需要在截止时间前执行完毕，无论是强制周期还是可选周期。因此，每个任务 τ_i 的执行周期可表示为 $c_i = M_i + \lambda_i O_i$。相应的，任务 τ_i 的执行时间可表示为 $\sum_{l=1}^{L}(Y_{i,l}c_i/f_l)$。令任务 τ_i 的开始执行时间为 ts_i，执行结束时间为 te_i，截止时间为 d_i。整个 DAG 任务的截止时间记为 H。实时性约束表示如下：

$$te_i \le d_i,\ \forall i \in \mathcal{T}. \tag{3.14}$$

中国知网　https://www.cnki.net

5. 任务顺序约束：为了清晰展示任务间的相关性，引入二元变量 $R_{i,j}$，若 $R_{i,j}$ 值为 1，表示任务 τ_j 依赖于任务 τ_i，只有当任务 τ_i 执行完毕才能开始执行 τ_j；若为 0，表示任务 τ_j 不依赖于任务 τ_i。此外，引入二元变量 $Q_{i,j}$ 来表示任务的执行的先后顺序。当变量 $R_{i,j}=1$ 时，$Q_{i,j}=1$；此外对于无依赖关系的任务，当任务 τ_i 在任务 τ_j 之前执行时，我们有 $Q_{i,j}=1$，否则 $Q_{i,j}=0$。任务执行顺序的约束表示如下：

$$\begin{gathered}\forall i \neq j \in \mathcal{T},\ \forall n \neq m \in \mathcal{M},\ R_{i,j}=1,\\ te_i \leq ts_j + (2 - X_{i,n} - X_{j,m})\, H,\\ te_i \leq ts_j + (2 - X_{i,n} - X_{j,n})\, H,\end{gathered} \tag{3.15}$$

$$\begin{gathered}\forall i \neq j \in \mathcal{T},\ \forall n \in \mathcal{M},\ R_{i,j}=0,\\ te_i \leq ts_j + (2 - X_{i,n} - X_{j,n})\, H + (1 - Q_{i,j})\, H,\\ te_j \leq ts_i + (2 - X_{i,n} - X_{j,n})\, H + Q_{i,j} H,\end{gathered} \tag{3.16}$$

上述公式中，当任务 τ_j 依赖于任务 τ_i 时，无论两任务是不是被分配在同一处理器上执行，都需要满足任务 τ_j 的开始时间大于任务 τ_i 的结束时间，因此不等式 (3.15)用于确保具有依赖关系的任务执行的先后顺序; 而对于没有依赖关系的任务，当两任务被分配在同一处理器上执行时，需要用不等式 (3.16)来确定在同一处理上执行时的先后顺序，以满足同一处理器上同一时间只能执行一个任务的要求。

6. 任务可靠性约束：主要考虑系统瞬时故障的发生对任务执行的影响，建立任务可靠性模型。任务的可靠性与任务的执行时间和频率相关。任务的可靠性越高表示任务执行的可靠程度越高，每个任务的可靠性需大于一个最小值，约束如下：

$$r_i \geq R_{MIN},\ \forall i \in \mathcal{T}. \tag{3.17}$$

7. 能量约束：任务的执行频率和可选周期数不同，消耗的能量也不同。要求所有任务执行后的总能耗低于阈值，即：

$$E_{tol} \leq E_{thr},\ \forall i \in \mathcal{T}. \tag{3.18}$$

8. 温度约束：公式 (3.5)和 (3.6)展示了瞬时温度计算公式，即实时考虑处理器之间的横向热传递，也考虑处理器与散热片和环境间的纵向热传递，并且涉及到的参数多，计算复杂。因此，考虑使用稳态温度计算模式，即将调度周期 H 划分为 F 个小间隔，在每个小间隔内，处理器温度保持不变，因此公式 (3.5)和 (3.6)中的微分项可为 0，即 $C_n \frac{dT_n(t)}{dt} \to 0$ 且 $C_h \frac{dT_h(t)}{dt} \to 0$。同时将功耗公式 (3.3)代入，原公式变换为：

$$\begin{aligned}T_n^t \left(\sum\nolimits_{m=1}^{M} G_{n,m} + G_{n,h} - K_1^{v,k,t} \right) - \sum\nolimits_{m=1}^{M} G_{n,m} T_m^t = \\ G_{n,h} T_h + C_n^{e,v,t} f_{i,n}^t + K_2^{v,k,t},\end{aligned} \tag{3.19}$$

中国知网 https://www.cnki.net

$$T_h^t\left(\sum_{g=1}^{Z} G_{h,g} + G_{h,e} - G_{n,h}\right) - \sum_{g=1}^{Z} G_{h,g}T_g^t = G_{h,e}T_e - G_{n,h}T_n^t, \tag{3.20}$$

其中 $f_{i,n}^t$ 表示在 t 的时间间隔时任务 τ_i 在处理器 p_n 上的执行频率。基于公式 (3.19)，考虑 M 个处理器的情况，可以得到以下等式:

$$\begin{bmatrix} A^{v,1,t} & -G_{1,2} & \cdots & -G_{1,M} \\ -G_{2,1} & A^{v,2,t} & \cdots & -G_{2,M} \\ \cdots & \cdots & \cdots & \cdots \\ -G_{M,1} & -G_{M,2} & \cdots & A^{v,M,t} \end{bmatrix} \begin{bmatrix} T_1^t \\ T_2^t \\ \cdots \\ T_M^t \end{bmatrix} = \begin{bmatrix} C_1^{e,v,t} f_{i,1}^t \\ C_2^{e,v,t} f_{j,2}^t \\ \cdots \\ C_M^{e,v,t} f_{N,M}^t \end{bmatrix} + \begin{bmatrix} G_{1,h}T_h + K_2^{v,1,t} \\ G_{2,g}T_g + K_2^{v,2,t} \\ \cdots \\ G_{M,Z}T_Z + K_2^{v,M,t} \end{bmatrix}. \tag{3.21}$$

令 $A^{v,k,t} = \sum_{m=1}^{M} G_{k,m} + G_{k,h} - K_1^{v,k,t}$，有公式如下:

$$\begin{bmatrix} T_1^t \\ T_2^t \\ \cdots \\ T_M^t \end{bmatrix} = (G^{v,k,t})^{-1} \begin{bmatrix} C_1^{e,v,t} f_{i,1}^t \\ C_2^{e,v,t} f_{j,2}^t \\ \cdots \\ C_M^{e,v,t} f_{N,M}^t \end{bmatrix} + (G^{v,k,t})^{-1} G^{h,v,k,t}, \tag{3.22}$$

$$T_n^t = \psi^{v,k,t} \cdot f_{i,k}^t + \phi^{h,v,k,t}, \tag{3.23}$$

其中 $\psi^{v,k,t} = (G^{v,k,t})^{-1}[C_1^{e,v,t}, \ldots, C_M^{e,v,t}]^T$ 且 $\phi^{e,v,k,t} = (G^{v,k,t})^{-1}G^{h,v,k,t}$。综上，系统的温度约束表示如下:

$$T_n(t) \leq T_{thr},\ \forall n \in \mathcal{M},\ \forall t \in [0, H]. \tag{3.24}$$

9. 优化问题：本章的研究内容是在给定一组具有相关性的实时近似任务和 M 个同构处理器上，在满足任务的实时性、相关性和可靠性的约束下，保证系统的能量阈值约束和温度阈值约束，使系统的服务质量 QOS 最大化。系统的服务质量由可选周期数衡量。通常，任务可选部分的执行周期数越多，系统的 QOS 越高。系统的 QOS 与可选周期数的关系主要有线型和指数型两种，这里本章考虑线型描述形式。优化问题表示为:

$$\max_{\boldsymbol{X_{i,n}}, \boldsymbol{Y_{i,l}}, \boldsymbol{\lambda_i}, \boldsymbol{ts_i}, \boldsymbol{Q_{i,j}}} \sum_{i=1}^{N} (\lambda_i O_i). \tag{3.25}$$

$$\text{s.t. } (3.11) \sim (3.18),\ (3.24)$$

3.3 算法设计

多核处理器上的任务调度问题已被证明是 NP 难的。此外，如何对处理器的温度进行约束和优化控制也是一大难点。对于处理器的温度计算，可见公式 (3.5)，计算处理器 t 时刻的温度需要当前处理器上的功耗数据，而由处理器功耗的计算公式 (3.3)可知，要计算当前时刻的处理器的功耗，需要处理器的温度数据支撑。处理器的温度和功耗两者

中国知网　https://www.cnki.net

相互影响，存在循环相关性，在求解问题时二者难以解耦。因此为了降低计算和求解的复杂度，本章将问题分解为多核处理器上任务调度和多核处理器上温度优化两个子问题，并设计启发式算法来求解这些子问题。

3.3.1 任务调度优化算法

为了同时最小化能量和最大完成时间，首先引入一个平衡分数来描述任务在选择执行的处理器和执行频率时，在能耗和时间上的均衡，其定义如下:

$$BAS(\tau_i, p', f', p, f) = \frac{E_p(\tau_i, p', f') - E_p(\tau_i, p, f)}{E_p(\tau_i, p, f)} + \frac{T_c(\tau_i, p', f') - T_c(\tau_i, p, f)}{T_c(\tau_i, p, f)} + \frac{T_f(\tau_i, p', f') - T_f(\tau_i, p, f)}{T_f(\tau_i, p, f)}, \quad (3.26)$$

其中 $E_p(\tau_i, p', f') = \frac{M_i + o_i}{f'} P_i$ 表示任务 τ_i 在处理器 p' 上以频率 f' 执行所消耗的能量，而 $T_c(\tau_i, p', f') = \frac{M_i + o_i}{f'}$ 和 $T_f(\tau_i, p', f')$ 分别表示任务 τ_i 在处理器 p' 上以频率 f' 执行的执行时间和结束时间。BAS 由三部分组成:

1. $\frac{E_p(\tau_i,p',f') - E_p(\tau_i,p,f)}{E_p(\tau_i,p,f)}$ 是归一化的能量因子，用于比较在选择不同处理器和频率执行任务 τ_i 时所消耗的能量大小。

2. $\frac{T_c(\tau_i,p',f') - T_c(\tau_i,p,f)}{T_c(\tau_i,p,f)}$ 是归一化的执行时间因子，用于比较选择在不同处理器和频率来执行任务 τ_i 时的任务执行时间。这里只考虑了任务强制部分的执行时间，而任务的可选部分在算法 3.1后续内容中进行优化。

3. $\frac{T_f(\tau_i,p',f') - T_f(\tau_i,p,f)}{T_f(\tau_i,p,f)}$ 用来比较在选择不同处理器和频率来执行任务 τ_i 时的结束时间。

如果 $BAS(\tau_i, p', f', p, f) < BAS(\tau_i, p, f, p', f')$，表示相比于 (p, f)，选择 (p', f') 来执行任务 τ_i，可以在能量消耗和时间方面获得更高的收益。因此，将 τ_i 分配给处理器 p' 并分配频率 f' 来执行该任务。通过对每个任务应用上述方法，可以根据任务相关性的顺序获得任务分配和频率分配的决策。

图 3-2展示了本章提出的温度优化算法的流程图。在算法 3.1中，首先给当前任务分配初始的处理器和执行频率（第 3 行）。通过遍历所有的处理器和频率等级，计算相应的 BAS 值，并将 BAS 值最小的处理器和频率分配给当前任务（第 13 行）。在此基础上，更新任务的结束时间 te_i、执行时间 $ex_i = \frac{M_i}{f'}$、能耗 $E_i^1 = ex_i P_i$ 和处理器的最早空闲时间 $ptime$（第 14 行）。然后根据任务的截止时间和相关性约束，计算每个任务的最晚结束时间 ls_{end} 和最晚开始时间 ls_{start}（第 17 ∼20 行）。此外，ls^A_{start} 表示任务 τ_i 的所有后续任务的最晚开始时间集合。最后，根据任务可靠性和最晚结束时间约束，在不违反任务相关性的情况下增加每个任务的可选周期，直到总能量消耗达到能量阈值时停止(第 21∼ 31行)。

中国知网 https://www.cnki.net

算法 3.1 任务调度优化算法

输入: 任务集 $\mathcal{T}$, 处理器集 $\mathcal{M}$, 频率集 $\mathcal{VF}$, 任务顺序 $order$，相关性 $\mathcal{R}$，截止时间 $\mathcal{D}$

输出: 初始调度方案 $S1$

$ptime = zeros(1, M)$
for $\forall i \in \mathcal{T}$ **do**
    $p' = p_1, f' = f_1$
    for $\forall j \in \mathcal{M}$ **do**
        for $\forall k \in \mathcal{L}$ **do**
            根据公式(3.26)计算 $BAS(\tau_i, p', f', p, f)$ 和 $BAS(\tau_i, p, f, p', f')$
            根据公式(3.2)计算 r_i
            if $BAS(\tau_i, p', f', p, f) < BAS(\tau_i, p, f, p', f')$ & $r_i \geq R_{MIN}$ **then**
                $p' = p, f' = f$
            end if
        end for
    end for
    任务 τ_i 以 (p', f') 执行
    更新结束时间 te_i、执行时间 $ex_i = \frac{M_i}{f'}$、能耗 $E_i^1 = ex_i P_i$ 和处理器最早空闲时间 $ptime(p') = \max\{te_i, ptime(p')\}$
end for
计算剩余的能量为 $E_r = E_{thr} - \sum_{i=1}^{N}(E_i^1)$
for $\forall i \in \mathcal{T}$（逆序） **do**
    $ls_{end,i} = min(d_i, ls_{start}^A)$ // 最晚结束时间
    $ls_{start,i} = ls_{end,i} - ex_i$ // 最晚开始时间
end for
    for $\forall i \in \mathcal{T}$ **do**
        if $E_r > 0$ **then**
            计算任务的最大可选周期数 $o_i = \min\{O_i, (ls_{end,i} - te_i)f'\}$
            根据公式 (3.2)计算 r_i
            if $r_i < R_{MIN}$ **then**
                $o_i = \frac{\ln R_{MIN}}{-\eta \times 10^D} f' - M_i$
            end if
            $E_i^2 = \frac{o_i P_i}{f'}$
            $E_r = E_r - E_i^2$
        end if
    end for

中国知网　https://www.cnki.net

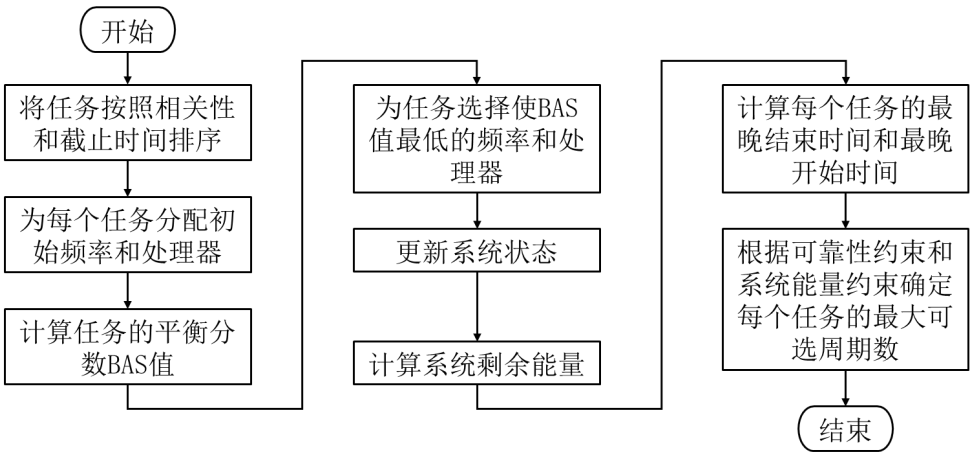


图 3-2 任务调度算法流程图

3.3.2 温度优化算法

通过算法 3.1获得了关于研究问题的初始调度方案 $S1$。根据稳态温度计算模式，可以通过公式 (3.22)计算出调度 $S1$ 的峰值温度 $T_{\max}$ 值。如果系统的峰值温度 $T_{\max}$ 大于阈值温度 T_{thr} 则执行温度优化算法。处理器温度的计算过程可分为瞬时温度计算和稳态温度计算。虽然瞬时温度计算的精度较高，但系统模型复杂且难以计算。相比之下，稳态温度计算使用处理器温度的稳态值而不是瞬时值，即短时间间隔内的处理器的温度是恒定的，简化了处理器温度计算模式和时间。

在算法 3.2中，将时间划分为 N_x 个小区间，并计算每个区间中的温度（第 3 行）。对于超过温度阈值 T_{thr} 的任务，通过插入空闲和调整频率来降低处理器的温度（第 2∼36 行）。首先，执行松弛插入。由于算法 3.1的目标是在任务相关性约束下最小化能量和最大完工时间，因此某些任务之间不存在空闲时间。为了在任务执行之前插入空闲时间，在每个间隔中，计算任务的最晚开始时间 ls_{start}（第 4 行）。然后，根据最晚开始时间 ls_{start} 和最大空闲时间 $sl_{\max}$（第 9 行）确定每个任务可插入的空闲时间大小。如果 $sl_{\max}$ 没有限制，插入任务的空闲时间将导致后续任务违反实时性约束，因此，本章为 $sl_{\max}$ 设置一个范围。任务空闲时间插入 sl 完成后，后续任务的开始时间、结束时间和最晚开始时间可以进行相应的调整（第 10∼12 行）。

松弛迭代完成后，再次计算整个调度的峰值温度（第 15 行）。如果满足温度限制，则温度调节完成（第 16∼18 行）。否则，将调整超过温度阈值的任务频率（第 19∼34 行）。当任务执行频率降低时，任务执行时间会增加，影响任务可靠性和实时性能。因此，处理违反可靠性和实时约束的任务是必要的。首先，调整可选周期数的数量，直到满足可靠性约束（第 26 行）。在此基础上，为满足实时约束，进一步调整可选周期数，使任务 τ_i 的结束时间小于最晚结束时间 l_{end}（第 29∼31 行）。调整频率级别和可选周期后，如果峰值温度低于阈值温度，则调整完成（第 35 行）。否则，将继续上述方法，直到峰值温度达到温度阈值。请注意，在调整过程中，频率等级的降低和可选周期数的减少，执行

中国知网 https://www.cnki.net

任务所需的能量也将减少。因此，进行温度优化调整不会违反能量约束。通过算法 3.2，可以为同构多核平台找到一个满足实时性、相关性、可靠性、能量和温度等多重要求的近似实时任务的最终调度方案。

3.4 仿真结果

3.4.1 实验设置

本文中的所有仿真均由 Matlab 实现。具体的处理器硬件配置参数和常量见表 3.2[51]，环境温度为 45°C。使用随机生成的任务集进行仿真，其中任务数量设置为 $N = [6, 10, 20, 30]$，每个任务的强制和可选部分的周期数设置在 $[4 \times 10^7, 6 \times 10^8]$ 范围内。任务集的最大截止期限设置为 $H = \sum_{i=1}^{N} M_i / f_{\max}$，并且每个任务的截止时间设置为 $d_i = dep_i H / \max_{\forall i}\{dep_i\}$，其中 dep_i 表示任务 τ_i 的深度。

表 3.2 处理器参数和常量

电压 v	频率 f	常量$K_1(v)$	常量$K_2(v)$
0.80	0.679	0.1666	7.3249
0.85	0.803	0.1754	8.6126
0.90	1.010	0.1846	10.238
0.95	1.260	0.1942	12.315
1.00	1.530	0.2043	14.998

3.4.2 温度优化效果对比

插入空闲时间和调整任务频率可以有效降低系统温度，为了更好的展现两者结合使用时对降低处理器峰值温度的表现，图 3-3展示了在不同状态下的处理器峰值温度。图 3-3a表示，在没有进行温度优化的情况下，系统的峰值温度出现在处理器 p_2 上，并且在执行任务 τ_4 时处理器温度可以达到 73.9°C。为任务 τ_4 插入空闲时间会将峰值温度降低到 71.73°C。然而，由于任务之间的相关性，任务 τ_4 在处理器 p_2 上的执行也会被延迟，如图 3-3b所示。另一方面，降低任务 τ_4 的执行频率也能降低处理器的峰值温度 (70.69°C)，但同时增加了执行时间，如图 3-3c所示。图 3-3d显示了同时使用空闲分布策略和频率调整的情况。通过这样做，处理器 p_2 的峰值温度被降低到 68.35°C。虽然任务执行时间有所增加，但仍能满足任务的实时性要求。因此，通过结合使用空闲分布策略和频率调整策略可以获得更好的温度优化效果。另外，虽然对任务的调整只在处理器 p_2 上进行，处理器 p_1 上的峰值温度跟随处理器 p_2 的降低而降低。这是因为本章的温度模型中考虑到了相邻处理器的横向热影响，相邻处理器间也存在热传递现象。

中国知网　https://www.cnki.net

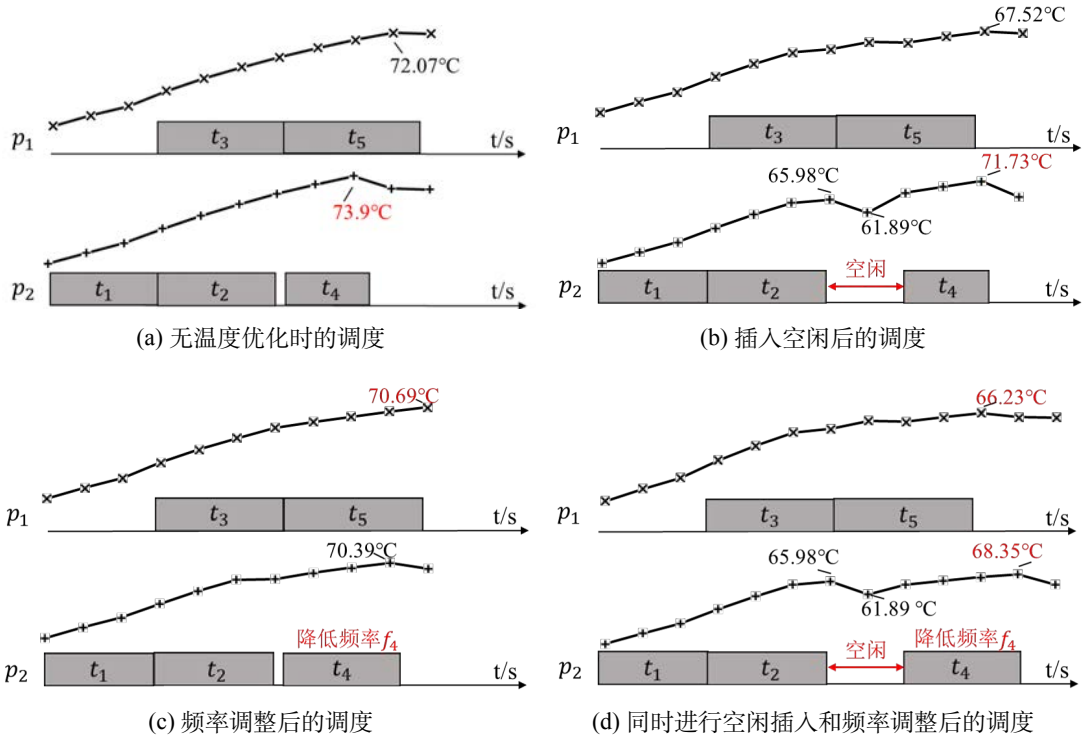
算法 3.2 温度优化算法

输入: 初始调度 $S1$，处理器选择 p'，频率选择 f'，开始时间 ts，结束时间 te，sl_{max}

输出: 最终调度方案 $S2$

根据公式 (3.22)计算调度方案 $S1$ 的峰值温度 T_{max}
while $T_{\max} > T_{thr}$ **do**
    $N_x = 10 * H$
    计算每个任务的最晚开始时间 ls_{start} 和最晚结束时间 ls_{end}
    for $\forall i \in \mathcal{N}_{\S}$ **do**
        for $\forall j \in \mathcal{M}$ **do**
            $\tau_i \leftarrow$ 任务温度超过阈值
            $sl_{\max,i} = ts_i - ls_{start,i}$
            $sl = \min\{ts_i - ls_{start,i}, sl_{\max}\}$
            if $sl + ts \leq = ls_{start}$ **then**
                插入空闲 sl，更新 ts，te 和 ls_{start}
            end if
        end for
    end for
    根据公式 (3.22)计算峰值温度 T_{max}
    if $T_{\max} < T_{thr}$ **then**
        结束本次循环
    end if
    for $\forall i \in \mathcal{T}$ **do**
        for $\forall j \in \mathcal{M}$ **do**
            $\tau_i \leftarrow$ 任务温度超过阈值
            根据公式 (3.2)计算 $r_i(f'-1)$
            if ($f' \neq$ minimum) & ($r_i \geq R_{MIN}$) **then**
                $f' = f'$ - 1;
            else
                $o_i = \frac{\ln R_{MIN}}{-\eta \times 10^D} \times f' - M_i$;
            end if
            更新任务 τ_i 的结束时间 te_i
            if $te_i > ls_{end}$ **then**
                $o_i = (ls_{end} - ts_i) \times f' - M_i$
            end if
            更新 ts，te 和 ls_{end}
        end for
    end for
    重复 16~ 18行
end while

中国知网 https://www.cnki.net



(a) 无温度优化时的调度　(b) 插入空闲后的调度

(c) 频率调整后的调度　(d) 同时进行空闲插入和频率调整后的调度

图 3-3 不同任务映射方案下的处理器温度

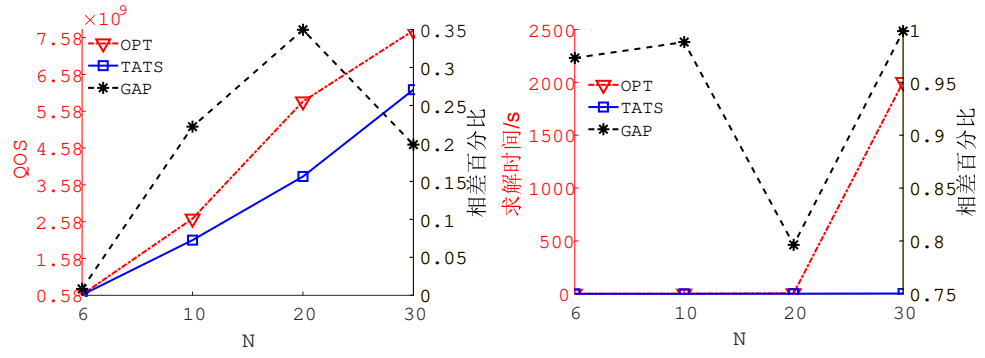
3.4.3　与最优解的对比

为了评估本章提出的 TATS 调度算法的性能，将本章提出的启发式算法得到的调度与无温度优化情况下的最优解进行对比。在第一阶段解决的是一个混合整数线性规划（Mixed-integer linear programming, MILP）问题，它可以通过 Gorubi 求解以获得在能量、可靠性和实时约束下最大化系统 QOS 的最优调度。图 3-4展示了本章提出的 TATS 算法与最优解在不同任务数 N 的实验案例下，系统 QOS、求解时间、峰值温度和 makespan 这四个方面的表现。其中红色的虚线表示最优解 OPT，蓝色的实线表示 TATS 算法得到的结果，而黑色的虚线表示 TATS 算法与最优结果 OPT 的差值。

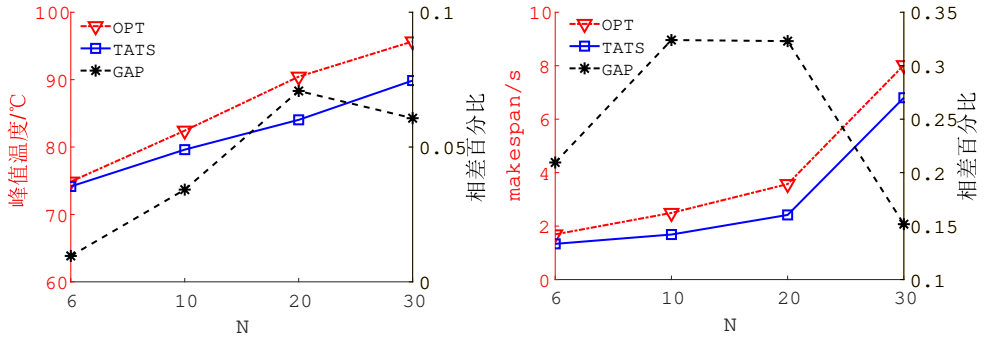
在系统 QOS 方面，相比于最优的情况，本章提出的 TATS 算法平均会损失了 20.24% 的系统性能，可见图 3-4a。在所有实验案例中，本章提出的 TATS 算法在最好的情况下可获得 100% 的 QOS 最优解，在最差的情况下只能获得 61% 的 QOS 最优解。在求解时间方面，用本章提出的 TATS 算法进行调度的时间远小于用求解器求解的时间。随着任务数量 N 的增加，用 Gorubi 求解最优解的方法和本章提出的 TATS 算法的求解时间都会增加，可见图 3-4b。然而，与最优方法相比，启发式算法的求解时间可以忽略不计。相比于最优解，本章提出的 TATS 算法最高可以减少 99.9% 的求解时间，平均可减少 99.6% 的求解时间。

图 3-4c展示了 TATS 算法与最优解在处理器峰值温度上的表现，TATS 算法求解的调度方案可以获得更低的峰值温度。相比于没有进行温度优化的最优调度方案，TATS

中国知网　https://www.cnki.net



(a) OPT 与 TATS 的系统 QOS 对比图　　(b) OPT 与 TATS 的求解时间对比图



(c) OPT 与 TATS 的峰值温度对比图　　(d) OPT 与 TATS 的 makespan 对比图

图 3-4 不同任务数下 OPT 与 TATS 的性能对比

算法求解的调度方案可以降低 3.97% 的峰值温度。此外，可以看到在任务数量较少时，二者的峰值温度差距不大，如任务数量 N 为 6 和 10 时，平均差值为 2.19%；当任务数量较大时，差值变大，如任务数量 N 为 20 和 30 时，平均差值为 6.58%。makespan 表示执行完整个任务集的完成时间。在 makespan 方面，TATS 算法求解的调度方案也都小于最优解解得的调度方案，可见图 3-4d。相比于最优解，TATS 算法平均只能得到 79.76% 的最优系统 QOS，而系统 QOS 是由任务的可选周期数衡量的，对于考虑相关性的、具有硬时间约束的实时任务，周期数量的减少有可能会减少任务集总体的完成时间的。

3.4.4 与其他算法的对比

本章提出的 TATS 算法将处理器温度作为约束考虑，处理器的峰值温度需满足温度阈值约束。为了比较对处理器温度参数作不同考虑时对处理器峰值温度的作用效果，本节实验选用了两种不同的算法作为对比，其中 OP 算法[38] 以最小化系统能耗为目标，没有将处理器温度考虑在问题求解中，而 MMT 算法[78] 将处理器温度放在目标函数中，目标是最小化系统的峰值温度。以下是这两种算法的简要介绍：

1. 在线峰值功率和峰值温度管理算法（OP）[38]。该算法考虑了同构多核平台上具有相关性的周期任务的调度问题，以最小化系统功耗为目标。通过热传感器测量处理器温

中国知网 https://www.cnki.net

度，不考虑任务的可靠性。使用 DVFS 和空闲分布策略来减少系统的能耗。

2. 处理器峰值温度最小化算法（MMT）[78]。该算法将任务到多核处理器上的调度问题转换为资源分配问题，通过动态调整分配的资源来最小化系统的峰值温度，考虑任务的实时性约束，但是不考虑任务的相关性和可靠性约束。

图 3-5比较了本章提出的 TATS 与 OP 算法和 MMT 算法在不同实验案例下的峰值温度对比。从图中可以明显看出，三种算法中峰值温度最低的是以最小化系统峰值温度为目标的 MMT 算法。相比于 OP 算法，本章提出的 TATS 算法在峰值温度上平均可降低 2.04%。这是因为 OP 算法通过最小化能耗来间接优化处理器温度，没有直接对处理器温度进行处理，并且只考虑功耗没有考虑到相邻处理器的热影响，有可能导致没有预想到的热点。同时这也证明了通过减少处理器功耗来控制处理器温度的方案是不可行的。相比于 MMT 算法，本章提出的 TATS 算法在峰值温度上平均增加了 13.85%。这是因为 MMT 算法是以最小化系统峰值温度为目标的，表示系统以性能最低的状态执行任务；而 TATS 算法是以最大化系统 QOS 为目标的，需要最大化系统的性能，温度只作为一项约束进行考虑。并且相比于 TATS 算法，MMT 算法不考虑任务的相关性和可靠性约束，任务模型更加简单，只需要保证任务满足实时性约束即可，更易获得更低的系统峰值温度。因此，本章提出的 TATS 算法在处理器峰值温度优化方面的表现是在尽可能保证系统性能最大化的情况下进行温度优化的。

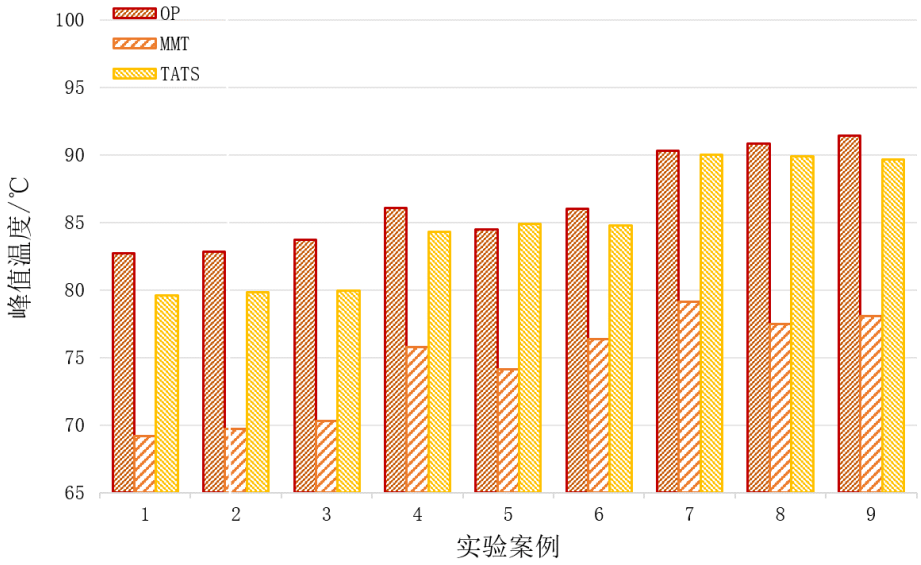


图 3-5 对温度不同处理下的峰值温度对比

此外，为了展现本章提出的 TATS 算法在系统 QOS 的表现，选用了两种不同的算法作为对比，其中 OP 算法[38] 以最小化系统能耗为目标，没有将处理器温度考虑在问题求解中，而 MMT 算法[78] 以最小化系统峰值温度为目标。以下是这两种算法的简要介绍：

中国知网　https://www.cnki.net

1. 近似任务的温度感知调度算法（RePAiR）[32]。该算法考虑了同构多核平台上近似实时任务的温度感知调度问题，以最大化系统 QOS 为目标，而不考虑任务之间的相关性、任务可靠性和能量约束。使用 DVFS 和任务迁移策略来降低处理器温度。

2. 截止时间感知的近似任务温度感知调度算法（DELI）[79]。该算法考虑了同构多核平台上近似实时任务的温度感知调度问题，以最大化系统 QOS 为目标，考虑具有相关性的任务，而不考虑任务的可靠性和能量约束。通过空闲分布策略和处理器休眠策略来降低处理器的峰值温度。

图 3-6比较了本章提出的 TATS 算法与 RePAiR 算法和 DELI 算法在不同实验案例下的性能对比，其中的左坐标轴标注代表处理器峰值温度的柱状图，右坐标轴标注代表系统 QOS 的折线图。从图中可以明显看出，本章提出的 TATS 算法在峰值温度和系统 QOS 上都有较好的表现。

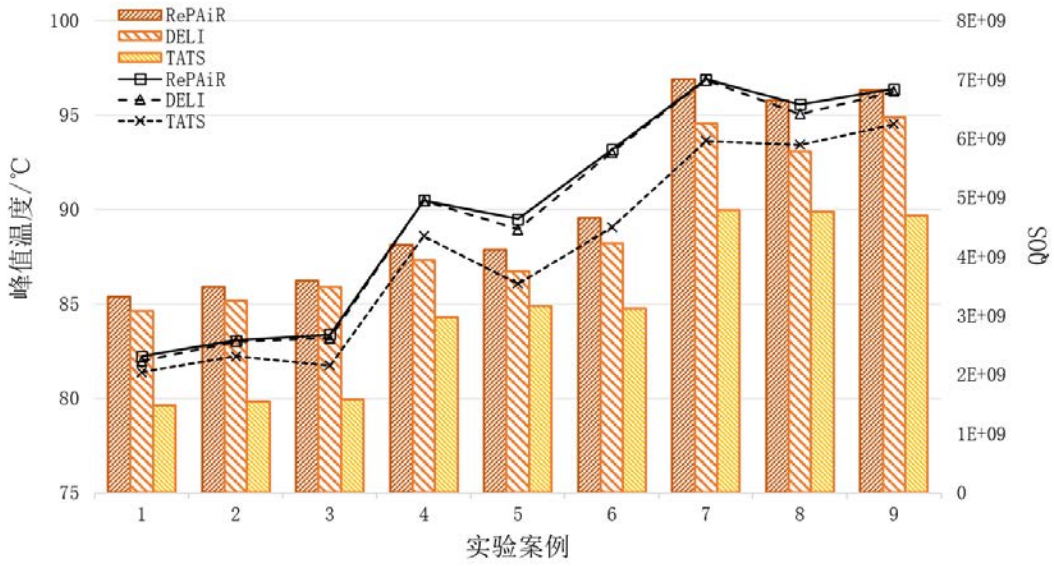


图 3-6 与其他算法的峰值温度和 QOS 对比

在峰值温度优化方面，相比于 RePAiR 算法，本章提出的 TATS 算法在峰值温度上平均可降低 6.05%；相比于 DELI 算法，TATS 算法在峰值温度上平均可降低 4.72%。这是因为对于温度较高的任务，RePAiR 算法使用任务迁移方法将任务重新映射到温度较低的处理器。然而，这种方案只是转移热量而不没有减少热量，即这种方法仍然可能会对处理器的峰值温度构成威胁。并且 RePAiR 算法考虑了近似任务模型但没有考虑温度阈值约束，因此容易造成较高的峰值温度。另外，DELI 算法考虑具有相关性的任实时任务在多核平台上的调度问题，对于温度较高的任务，主要使用空闲分布策略和使处理器核心处于休眠状态的策略来降低处理器的峰值温度。无论是插入空闲还是休眠处理器，本质上都是推迟任务的开始执行时间，容易影响任务的实时性和相关性。这种单一

中国知网 https://www.cnki.net

的温度优化策略在调度具有相关性的实时任务时作用受限，并不能尽可能的降低处理器的峰值温度。因此同样容易造成较高的峰值温度。

在系统 QOS 方面，相比于 RePAiR 算法，本章提出的 TATS 算法平均减少了 14.93% 的系统 QOS；相比于 DELI 算法，TATS 算法平均减少了 13.52% 的系统 QOS。分析 QOS 减少的原因大致可分为两方面。一是因为 RePAiR 算法和 DELI 算法没有对处理器峰值温度进行限制，在获得更多系统 QOS 的同时，处理器的温度也处于较高水平，这与本章在温度约束下实现系统性能最大化的可靠调度的目标不一致；二是因为 TATS 算法对系统使用的总能耗进行了约束，而 RePAiR 算法和 DELI 算法没有，所以 RePAiR 算法和 DELI 算法可以无限制的执行更多的可选周期，获得更多的系统 QOS。在实际应用中，系统的资源是有限的，任务需要在有限资源下更好的展现性能，因此，对系统调度时的总能耗进行约束是合理的。

综合来看，本章提出的 TATS 算法在处理器峰值温度优化和系统 QOS 上具有良好表现。相比于同类算法，本章提出的方案考虑的问题更全面更复杂，更贴近实际应用。

3.5　本章小结

本章针对多核同构平台上近似任务的调度和温度优化问题，提出了一种两阶段温度感知调度方案 TATS，解决了在时间、能量、可靠性和温度约束下的近似实时任务系统性能最大化的调度问题。在第一阶段，制定了以最小化能量和最小化最大完工时间为目标的平衡分数，得到任务到处理器的分配和任务频率的初步选择。在第二阶段，本章在稳态处理器温度分析的基础上设计了一种结合 DVFS 和空闲分布策略的温度优化算法来降低处理器的温度。对处理器峰值温度超过阈值约束的调度进行温度优化。仿真实验表明，本章提出的 TATS 算法可以提高系统服务质量，降低运行时间、能耗和温度。与最优解相比，该方法平均减少了 99.6% 的求解时间。此外，与其他算法相比，本章提出的 TATS 算法考虑的问题更全面更贴合实际应用，并在处理器峰值温度和系统服务质量上具有良好表现。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第四章　异构多核平台上实时任务的温度感知调度设计

4.1　引言

不同于第三章考虑的同构多核处理器平台，本章考虑基于大小核结构的异构多核平台。相比于同构多核处理器平台，异构多核处理器平台集成了两种或以上的不同性能参数的处理器。通过将任务调度到不同种类的处理器上执行，可以实现更高效的性能、更低的功耗以及更灵活的计算等功能。并且在第三章中，考虑的是将调度和温度优化分开进行的静态调度方案。因此，本章的研究目标是基于异构多核平台，提出以系统性能最大化为目标，满足各项约束同时进行温度优化的温度感知调度方案。本章的主要工作有：1）为了快速准确的对处理器温度进行计算，通过公式推导，可得到基于频率选择的处理器温度快速计算的方法；2）在选择任务执行的处理器时，结合热负载均衡策略，并通过实验证明了该方法的有效性；3）将模型预测控制与调度结合，对控制器进行合理改进，提出了基于模型预测控制的温度感知调度方案；4）通过仿真实验对比，展现本章提出的调度算法和温度优化算法的性能。

4.2　系统模型和问题定义

由于本章的研究内容是基于第三章的同构多核平台上实时任务的调度和温度设计问题，因此在系统模型上除了处理器模型和温度模型发生了改变，其他的任务模型、功耗模型、可靠性模型和能量模型直接引用第三章的内容。

4.2.1　处理器模型

考虑具有 M 个处理器 $\{p_1,\ldots,p_M\}$ 的异构多核平台。异构架构考虑 ARM（Advanced RISC Machines Ltd）公司提出的 big.LITTLE 节能异构多核处理器架构。big.LITTLE 架构包含两个性能不同的处理器集群，big 集群和 LITTLE 集群。同一集群内的处理器是同构的，big 集群处理器的计算性能一般高于 LITTLE 集群，可将占用资源较多、优先级较高的任务优先分配在 big 集群的处理器上。因此，本章考虑的多核处理器架构包含两个不同的集群，令总处理器集为 $\mathcal{P} \triangleq \{\mathcal{P}_{\mathcal{B}},\mathcal{P}_{\mathcal{L}}\}$，其中 $\mathcal{P}_{\mathcal{B}} \triangleq \{p_1,\ldots,p_{M_B}\}$ 表示 big 集群的处理器集，$\mathcal{P}_{\mathcal{L}} \triangleq \{p_{M_{B+1}},\ldots,p_{M_B+M_L}\}$ 表示 LITTLE 集群的处理器集。M_B 和 M_L 分别表示 big 集群和 LITTLE 集群的处理器数，且 $M = M_B + M_L$。

设定每个处理器上可实现多个等级的电压/频率 V/F。对于 big 集群中的处理器，每个处理器有 l_B 个电压/频率等级，即 $\mathcal{VF}_{\mathcal{B}} \triangleq \{(v_1,f_1),\ldots,(v_{l_B},f_{l_B})\}$；同样的对于 LITTLE 集群，每个处理器有 l_L 个电压/频率等级，即 $\mathcal{VF}_{\mathcal{L}} \triangleq \{(v_{l_{B+1}},f_{l_{B+1}}),\ldots,(v_{l_B+l_L},f_{l_B+l_L})\}$。

中国知网　https://www.cnki.net

为了确定任务的执行频率，引入二元变量 $Y_{i,l}$。当任务 τ_i 的频率和电压选择为 (v_l, f_l) 时，令 $Y_{i,l}=1$，其他情况 $Y_{i,l}=0$。相应的，任务 τ_i 的执行时间可表示为 $\sum_{l=1}^{l_{sum}}(Y_{i,l}c_i/f_l)$，其中 $l_{sum}=l_B+l_L$，表示系统中处理器频率的总等级数。

4.2.2 MPC 模型

模型预测控制（Model Predictive Control, MPC）是一种最优控制技术，被广泛的应用于工业领域，如化石和石油制造领域[80]。主要包括目标函数和约束模块、预测模块和滚动优化模块[81]，相关模型架构可见图 4-1和图 4-2。MPC 的核心思想是滚动优化，采用滚动式的有限时域优化的策略。在每一采样时刻，MPC 可基于系统当前状态和预测模型预测系统未来 p 步的状态，并根据该时刻的目标函数和约束，得到未来一段时间，一般称为预测区间的可行解 $[u_k, u_{k+1}, \ldots, u_{k+p}]$。滚动优化是指对于求解得到的输入序列，MPC 控制器只将 t_k 时刻的输入 u_k 作用于系统而忽略掉序列的其他值 $[u_{k+1}, \ldots, u_{k+p}]$；在下一个时刻 t_{k+1}，MPC 控制器会重复上述优化求解过程，重新计算可行解并只将第一个值作用于系统。也就是说，优化过程不是一次离线完成的，而是反复在线进行的。通过滚动优化策略，始终在实际的基础上求解优化目标，兼顾考虑了对未来有限时域内的理想优化和实际执行中的不确定性的影响。

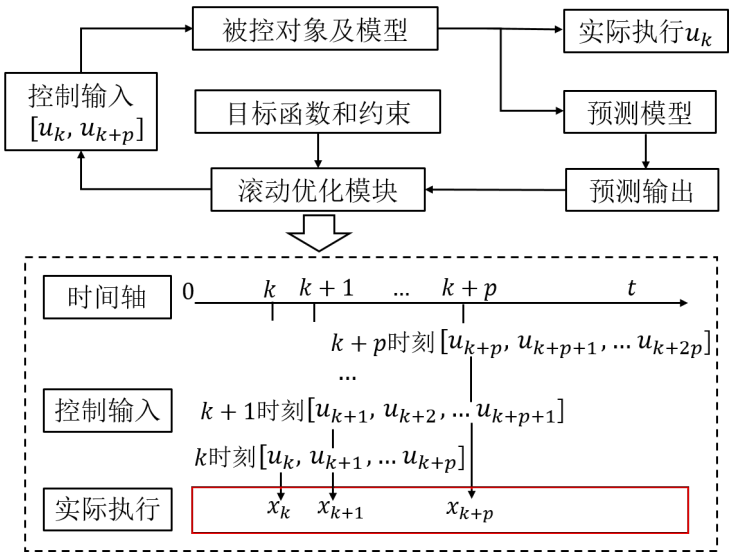


图 4-1 MPC 模型架构

4.2.3 传统 MPC 温度预测模型

传统的模型预测控制的温度预测方式是将处理器上与温度相关的参数转为参数矩阵，在每个采样时刻对温度进行预测。相比于第三章中先制定出整个 DAG 任务的调度方案再进行温度优化的模式，本章考虑更加合理的应用场景。考虑在调度的同时进行温度优化，得到最终满足温度约束的任务映射方案。而第三章中使用的温度模型，不仅考

中国知网 https://www.cnki.net

```
N;\\任务数;
V ;\\顶点
de[N] = {0}; \\记录深度
de[V] = 1;
dfs(V, de[V])
{
  for u ∈ V' \\ V的后继任务集合
    {de[u] = de[V] + 1;
     dfs(u, de[u])}
}
list = sort(de); \\获得DAG任务关
于深度大小的排序
```

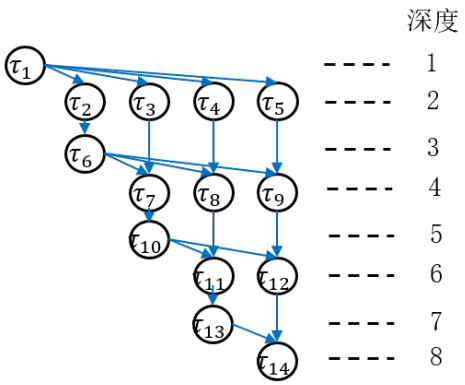


图 4-2 任务深度计算示意图

虑了相邻处理器横向热传导的影响，还考虑了处理器与散热片之间的纵向热传导，实际应用中可以获得更精确的温度值，但同时也需耗费更多的计算时间，不适合应用于对实时性要求更高的应用场景中。因此，这里考虑较为简便的温度计算模型，忽略掉相邻处理器之间以及散热片对处理器的热影响，仅保留处理器与环境之间的热交互。如公式 (4.1)所示，$T_n(t)$ 表示在 t 时刻，处理器 p_n 的温度，阈值温度设置为 T_{thr}。

$$C_n \frac{dT_n(t)}{dt} = P_{i,l} - G_{n,e}\left(T_n(t) - T_e\right), \tag{4.1}$$

其中 C_n 表示处理器的热电容，$T_n(t)$ 是时间 t 时处理器 p_n 的温度，T_e 表示环境温度。$G_{n,e} = \frac{area_n}{R_e area_{chip}}$ 表示处理器 p_n 与环境之间的热传导，且 $area_n$ 表示处理器 p_n 的面积，$area_{chip}$ 表示芯片的面积。$R_e = \frac{T_{active} - T_e}{P_{chip}} - R_{chip}$，其中 $R_{chip} = \frac{th_{si}}{K_{si} area_{chip}}$，$th_{si}$ 表示处理器的厚度，K_{si} 是硅的热导率，T_{active} 表示所有处理器繁忙时的平均活动温度，P_{chip} 表示芯片的总功耗。

由于公式 (4.1)可以看作是关于温度 $T_n(t)$ 的一阶微分方程，使用前向欧拉公式对其进行离散化，步长设置为 h：

$$T_n(t+h) = \left(1 - \frac{hG_{n,e}}{C_n}\right) T_n(t) + \frac{h}{C_n} P_{i,l} + \frac{hG_{n,e}T_e}{C_n}, \tag{4.2}$$

则，考虑核心数量 $M = 2$ 时，模型预测控制根据采样时刻 k 的温度预测模型可表示如下：

$$\begin{bmatrix} T_1(k+1) \\ T_2(k+1) \end{bmatrix} = \begin{bmatrix} A & 0 \\ 0 & A \end{bmatrix} \begin{bmatrix} T_1(k) \\ T_2(k) \end{bmatrix} + \begin{bmatrix} B & 0 \\ 0 & B \end{bmatrix} \begin{bmatrix} P_1(k) \\ P_2(k) \end{bmatrix} + E, \tag{4.3}$$

其中 $A = \left(1 - \frac{hG_{n,e}}{C_n}\right)$，$B = \frac{h}{C_n}$，$E = \frac{hG_{n,e}T_e}{C_n}$ 可视为常量矩阵。令 T_{ma} 表示处理器的温度向量，p_{ma} 表示处理器的功耗向量，$A_{ma} = \begin{bmatrix} A & 0 \\ 0 & A \end{bmatrix}$，$B_{ma} = \begin{bmatrix} B & 0 \\ 0 & B \end{bmatrix}$。则公式 (4.3)可表示为：

$$\begin{cases} T_{ma}(k+1) = A_{ma} T_{ma}(k) + B_{ma} P_{ma}(k) + E \\ Y_{ma}(k+1) = I_{ma} T_{ma}(k+1), \end{cases} \tag{4.4}$$

中国知网　https://www.cnki.net

其中 $Y_{ma}(k+1)$ 表示 k 时刻预测的 $k+1$ 时刻系统的温度输出向量，I_{ma} 用于控制系统输出哪些处理器的温度值。在 k 时刻，预测步长为 p 时，未来 p 步内处理器上的功耗作为输入向量，表示如下：

$$\mathcal{P}_{ma} = [P_{ma}(k), P_{ma}(k+1), ..., P_{ma}(k+p-1)], \tag{4.5}$$

相应的，未来 p 步内处理器上的温度作为输出向量可表示为：

$$\mathcal{Y}_{ma} = [Y_{ma}(k+1), Y_{ma}(k+2), ..., Y_{ma}(k+p)] \tag{4.6}$$

根据 k 时刻的温度向量 $T_{ma}(k)$ 以及推导式 (4.4)，可以得到 $k+2$ 时刻预测的处理器的温度向量，即：

$$\begin{aligned} T_{ma}(k+2) = A_{ma}T_{ma}(k+1) + B_{ma}P_{ma}(k+1) + E = \\ A_{ma}^2 T_{ma}(k) + A_{ma}B_{ma}P_{ma}(k) + B_{ma}P_{ma}(k+1) + A_{ma}E + E, \end{aligned} \tag{4.7}$$

$k+3$ 时刻预测的处理器的温度向量为：

$$\begin{aligned} T_{ma}(k+3) = A_{ma}T_{ma}(k+2) + B_{ma}P_{ma}(k+2) + E = A_{ma}^3 T_{ma}(k) + \\ A_{ma}^2 B_{ma}P_{ma}(k) + A_{ma}B_{ma}P_{ma}(k+1) + B_{ma}P_{ma}(k+2) + A_{ma}^2 E + A_{ma}E + E, \end{aligned} \tag{4.8}$$

以此类推，可得到 $k+p$ 时刻预测的处理器的温度向量：

$$\begin{aligned} T_{ma}(k+p) = A_{ma}^p T_{ma}(k) + A_{ma}^{p-1} B_{ma}P_{ma}(k) + \\ A_{ma}^{p-2} B_{ma}P_{ma}(k+1) + ... + A_{ma}^1 B_{ma}P_{ma}(k+p-2) \\ + B_{ma}P_{ma}(k+p-1) + A_{ma}^{p-1}E + A_{ma}^{p-2}E + ... + A_{ma}^1 E + E, \end{aligned} \tag{4.9}$$

根据上述式子可知，使用传统的模型预测控制的温度预测模型，在预测 $k+p$ 时刻处理器的温度时，需要对系数矩阵进行 p 次方的计算，在实际求解中需要耗费大量的计算时间和资源。另外模型预测控制的温度预测模型中的系数矩阵与实际计算存在一定的误差。经过 p 次方的计算，系数矩阵的误差将被放大，影响温度预测模型的准确的。因此，在下一小节中，将展示本章改进后的温度预测模型。

4.2.4 改进温度预测模型

在温度管理和优化过程中，需要实时获取和监测处理器的工作温度。处理器温度的获取方式主要有红外测量、大型有限元软件模拟仿真和基于 RC 公式模型计算三种。红外测量需要硬件支持，大型有限元软件模拟仿真需要耗费大量的计算资源，因此一般研究主要从温度的 RC 公式模型入手。但是使用原本的 RC 公式，即考虑处理器之间的横向热传递，也考虑处理器与散热片和环境间的纵向热传递，并且涉及到的参数多，计算复杂。而基于传统模型预测控制的温度预测模型是通过参数矩阵和功耗分布向量对未来

中国知网 https://www.cnki.net

温度的推导，在预测步数较长的情况下，需要耗费大量的计算时间和资源。因此，考虑对原公式进行简化，简化温度计算，并利用有限的参数达到局部预测温度的目的。

对原公式 (4.1)可以看作是关于温度的一阶微分方程，使用以得到离散化热模型如公式，将功耗公式中的 $P_{i,l}$ 代入可得：

$$C_n\frac{dT_n(t)}{dt}=C_{eff}v_l^2f_l+K_1(v_l)T_n(t)+K_2(v_l)-G_{n,e}T_n(t)-G_{n,e}T_e, \tag{4.10}$$

将所有 $T_n(t)$ 归并到等号左边：

$$\frac{dT_n(t)}{dt}+T_n(t)\left(\frac{G_{n,e}-K_1(v_l)}{C_n}\right)=\frac{C_{eff}v_l^2f_l+K_2(v_l)+G_{n,e}T_e}{C_n}, \tag{4.11}$$

假定在 t 时刻，处理器上的频率等级已知，即 $K_1(v_l)$、$K_2(v_l)$ 和 $C_{eff}v_l^2f_l$ 都是常值，则公式 (4.11)是为一阶线性微分方程的表达式。通过使用一阶线性微分方程的定积分求解公式求解上式，可得：

$$T_n(t)=\frac{C_{eff}v_l^2f_l+K_2(v_l)+G_{n,e}T_e}{G_{n,e}-K_1(v_l)}\left(1-e^{-\frac{G_{n,e}-K_1(v_l)}{C_n}\Delta t}\right)+T_ae^{-\frac{G_{n,e}-K_1(v_l)}{C_n}\Delta t}, \tag{4.12}$$

其中 T_a 表示 Δt 前时刻处理器的温度。令 $\theta_1=\frac{G_{n,e}-K_1(v_l)}{C_n}$，$\theta_2=\frac{C_{eff}v_l^2f_l+K_2(v_l)+G_{n,e}T_e}{C_n}$，则公式可表示为：

$$T_n(t)=\sum\nolimits_{l=1}^{l_{sum}}Y_{i,l}\left[\frac{\theta_2}{\theta_1}\left(1-e^{-\theta_1\Delta t}\right)+T_ae^{-\theta_1\Delta t}\right], \tag{4.13}$$

通过上述推导和变换，在处理器 p_n 上执行任务 τ_i 的温度转变为关于频率选择优化变量 $Y_{i,l}$ 和变化时间 Δt 的表达式（$X_{i,n}=1$）。配合使用任务深度作为步长的模型预测控制控制器，可用公式 (4.13)对处理器温度进行快速的计算和预测。相比于传统的模型预测控制的温度预测模型，在预测第 $k+p$ 时刻处理器的温度时，无需对系数矩阵的 p 次方进行计算，极大的简化处理器温度求解的时间和步骤。

4.2.5　问题描述

本章研究的是在多核异构处理器上的近似 DAG 任务的温度感知调度问题，目标是在任务实时性、相关性、可靠性、能量和处理器温度约束下，最大化系统的 QOS。本章利用模型预测控制技术的滚动优化思想，在基于第三章提出的 TATS 算法基础上，提出了一个基于模型预测控制的温度优化控制方案，实现各项约束下系统 QOS 的最大化调度，并结合模型预测控制中的温度预测模块实现处理器温度的在线约束。

要使用模型预测控制架构求解问题，除了要对预测模型进行设置，还需要对目标函数和约束、预测区间和采样间隔进行合理设置。原问题的描述非常复杂，可见公式 (3.25)，若使用传统的用求解器求最优解的方法，会消耗大量的计算时间和资源，不利于任务的系统的实际运行，使用模型预测控制求解整个问题是不现实的。因此，在每个采样时刻，首先借助启发式算法 TATS，在不考虑温度约束的情况下求解以最大化系统 QOS 为目标

中国知网　https://www.cnki.net

的调度方案。其次通过温度预测模块对处理器温度进行预测，若超过温度阈值，激活模型预测控制的控制器，反馈调整处理器的频率，在不违反任务实时性、相关性和可靠性约束下使预测区间内处理器的温度满足阈值约束；若满足温度约束则无需经过模型预测控制控制器的调整。因此，设置模型预测控制优化问题的目标函数和约束为：

$$\min_{\boldsymbol{Y_{i,l}},\boldsymbol{\lambda_i},\boldsymbol{ts_i}} \sum_{k=1}^{p}\sum_{n=1}^{M}\left(f_n(t+k|t)-f_{\max}\right)^2, \tag{4.14}$$

$$\text{s.t. (3.12)} \sim \text{(3.15)}, \text{(3.17)}, \text{(3.24)}$$

其中 p 表示预测步数，$f_n(t+k|t)$ 表示在采样 t 时刻第 k 步处理器上执行任务 τ_i 的实际频率，且 $f_n = X_{i,n}\sum_{l=1}^{l_{sum}} Y_{i,l} f_l$。相比于传统模型预测控制中使用基于时间间隔的采样方式，本文考虑使用基于相关性任务深度间隔的的采样方式。基于 DAG 任务模型，任务可根据深度进行排序，每个深度可作为一个步长。

上述优化问题可描述为，在预测区间内，在满足处理器温度阈值约束，同时不违反任务的实时性、相关性和可靠性约束的情况下，最大化系统的性能，此处性能表现为任务执行的频率。相比于原问题，此处减少了任务分配项 $X_{i,n}$（公式 (3.11)）、能量约束项（公式 (3.18)）和同一处理器上任务执行顺序的约束项（公式 (3.16)）。这是因为在进行模型预测控制优化控制前，启发式算法 TATS 已经确定了任务到处理器的分配，即 $X_{i,n}$ 是已知的，同一处理器上任务执行的先后顺序也是确定的；并且在进行模型预测控制优化控制时是降低任务执行的频率，同时对超过截止时间的任务减少部分可选周期，在能量消耗上是减少的，满足了能量约束，在模型预测控制优化问题求解中可以忽略。因此，本章需要确定的优化变量有：1）任务的频率选择，2）任务可选周期的周期数，3）任务执行的开始时间。

4.3 算法设计

传统的使用模型预测控制进行任务调度的方法，存在模型复杂、步骤繁琐只适用于求解小规模的简单问题的不足。本章根据简化的处理器温度计算模型，推导出可以快速估算处理器温度的温度预测模型，相比于传统的使用系数矩阵预测温度的模型，极大的简化的求解时间和步骤。并对模型预测控制的控制器进行了改进，相比于传统的将采样的时间间隔作为步长的优化模式，本章使用任务深度作为步长的优化模式，极大的简化了求解步骤和时间。此外，针对异构平台，在任务调度时加入热均衡策略，在同等参数选择下，选择温度更低的处理器执行任务。为了同时实现多核处理器上的任务调度和温度的联合优化设计，同时满足任务可靠性、相关性和实时性需求，将第三章提出的启发式算法 TATS 和模型预测控制相结合，生成使系统 QOS 最大的温度感知调度方案。算法的具体步骤可见算法4.1。

首先，对任务和处理器进行初始化工作（第 1～ 3 行）。对等待调度的 DAG 任务按照深度大小顺序排序，可见图 4-2，并获取任务的相关数据转为参数矩阵，如相关性

中国知网 https://www.cnki.net

算法 4.1 基于模型预测控制的温度感知调度算法（MPCT）

输入: 任务集 $\mathcal{T}$, 处理器集 $\mathcal{M}$, 频率集 $\mathcal{VF}_{\mathcal{B}}, \mathcal{VF}_{\mathcal{L}}$, 任务顺序 $order$，相关性 $\mathcal{R}$，截止时间 $\mathcal{D}$，周期数 $\mathcal{M}$ 和 $\mathcal{O}$

输出: 温度感知调度方案

1: $p^{'}$ 和 $f^{'}$ 用于存放任务执行的处理器和频率，ts 和 te 表示任务的开始和结束时间
2: 处理器最早空闲时间 $ptime = zeros(1, M)$，处理器温度 $pT = \{T_0\}$，T_0 表示初始温度
3: 获得 DAG 任务的深度，将任务按照深度排序，DE 表示总深度，de 表示当前任务的深度
4: **while** $de \leq DE$ **do**
5:　　根据预测步数，获取当前需要调度的任务集 $\mathcal{A}$
6:　　生成任务集 $\mathcal{A}$ 的一些参数矩阵，如相关性 $\mathcal{R}^{'}$、截止时间 $\mathcal{D}^{'}$、总周期数 ($\mathcal{M}^{'}$,$\mathcal{O}^{'}$) 和总能量阈值 $E^{'}$
7:　　使用第三章启发式算法 TATS 生成关于任务集 $\mathcal{A}$ 的以最大化系统 QOS 为目标的调度方案
8:　　热负载均衡策略，获得初始的调度方案 $S1$
9:　　根据公式 (5.3)预测温度，峰值温度为 T_{max}
10:　　**if** $T_{max} < T_{thr}$ **then**
11:　　使用 DVFS 策略进行温度优化
12:　　**end if**
13:　　根据执行步数 q，执行可执行的任务集 $\mathcal{B}$
14:　　更新处理器的最早空闲时间 $ptime$ 和温度 pT
15:　　更新深度值 $de = de + p - q$
16: **end while**
17: 所有任务调度完毕，获得最终的调度方案

$\mathcal{R} = \{R_{i,j}\}$, $\forall i, j \in N$、截止时间 $\mathcal{D} = \{d_i\}$, $\forall i \in N$、总周期数 ($\mathcal{M} = \{M_i\}$, $\forall i \in N$ 和 $\mathcal{O} = \{O_i\}$, $\forall i \in N$)。同时获取系统中处理器的相关参数，如最早空闲时间 $ptime = \{0\}$ 和初始温度 $pT = \{T_0\}$, T_0 表示初始温度。接着，根据任务的相关性可获得 DAG 任务的深度，将任务按照深度排序，总深度值可表示为 DE, de 表示当前任务的深度（第 3 行）。

根据预测步数 p，获取当前需要调度的任务集 $\mathcal{A}$（第 5 行）。生成任务集 $\mathcal{A}$ 的相关性参数矩阵 $\mathcal{R}^{'} = \{R_{i,j}\}$, $\forall i, j \in \mathcal{A}$、截止时间矩阵 $\mathcal{D}^{'} = \{d_i\}$, $\forall i \in \mathcal{A}$、总周期数矩阵 ($\mathcal{M}^{'} = \{M_i\}, \mathcal{O}^{'} = \{O_i\}$, $\forall i \in \mathcal{A}$) 和总能量阈值 $E^{'}$（第 6 行）。使用第三章提出的启发式算法 TATS 生成关于任务集 $\mathcal{A}$ 的以最大化任务 QOS 为目标的调度方案 $S1$，并且满足截止时间、相关性、能量和可靠性的约束。在原来启发式算法的基础上，增加了热负载均衡模块，在不影响截止时间和后继任务的截止时间的情况下，选择温度最低的处理器执

中国知网　https://www.cnki.net

行任务（第 7∼ 8 行）。最终获得任务的在处理器上的分配 $\mathcal{X}^{'}=\{X_{i,k}\}$, $\forall i \in \mathcal{A}$, $\forall k \in M$ 以及任务的执行频率 $\mathcal{Y}^{'}=\{Y_{i,l}\}$, $\forall i \in \mathcal{A}$, $\forall l \in L$、可选周期数 $o^{'}=\{o_i\}$, $\forall i \in \mathcal{A}$ 和任务的开始时间 $\mathcal{TS}=\{ts_i\}$, $\forall i \in \mathcal{A}$。

获得调度方案 $S1$ 后，调用温度预测模块，根据公式 (5.3)预测处理器的温度，峰值温度表示为 T_{max}（第 9 行）。如果峰值温度超过阈值约束，则需要进行温度优化调整，激活 MPC 优化控制模块。对于温度过高的任务使用 DVFS 策略，在不违反任务实时性和可靠性约束下，降低任务执行频率的等级，直到满足温度阈值约束（第 11 行）。如果初始调度方案的峰值温度满足阈值约束，则无需进行温度优化。优化完成后，获得关于任务子集 $\mathcal{A}$ 的最终调度参数，包括 $\mathcal{X}^{'}$、$\mathcal{Y}^{'}$、$o^{'}$ 和 $\mathcal{TS}$。

获得满足目标和温度约束的调度后，可进行任务执行操作。根据执行步数 $q, q \leq p$，获取当前可执行的任务集 $\mathcal{B}$，并根据相关的处理器参数 $\mathcal{X}'$ 和频率参数 $\mathcal{Y}'$ 执行任务（第 13 行）。并且，更新当前处理器的最早空闲时间 $ptime$ 和温度 pT（第 14 行）。更新当前任务的深度值（第 15 行）。循环进行上述的步骤，直到所有任务执行完毕，获得最终的满足任务约束和温度约束的以最大化系统 QOS 为目标的温度感知调度方案（第 17 行）。

4.4 仿真结果

4.4.1 实验设置

本文中的所有仿真均在 Matlab 上仿真实现。由于本章考虑类似大小核结构的异构多核处理器系统，小核处理器的相关参数可见表 3.2，大核处理器的相关参数可见表 4.1[82]。通过生成不同周期数和任务数的任务集进行实验模拟。每个任务的强制和可选部分的周期设置在 $[4\times10^7, 6\times10^8]$ 的范围内。任务集的最大期限被设置为 $H=\sum_{i=1}^{N} M_i/f_{\max}$ 并且每个任务的截止时间被设置为 $d_i = de_i H/\max_{\forall i}\{de_i\}$，其中 de_i 表示任务 τ_i 的深度。

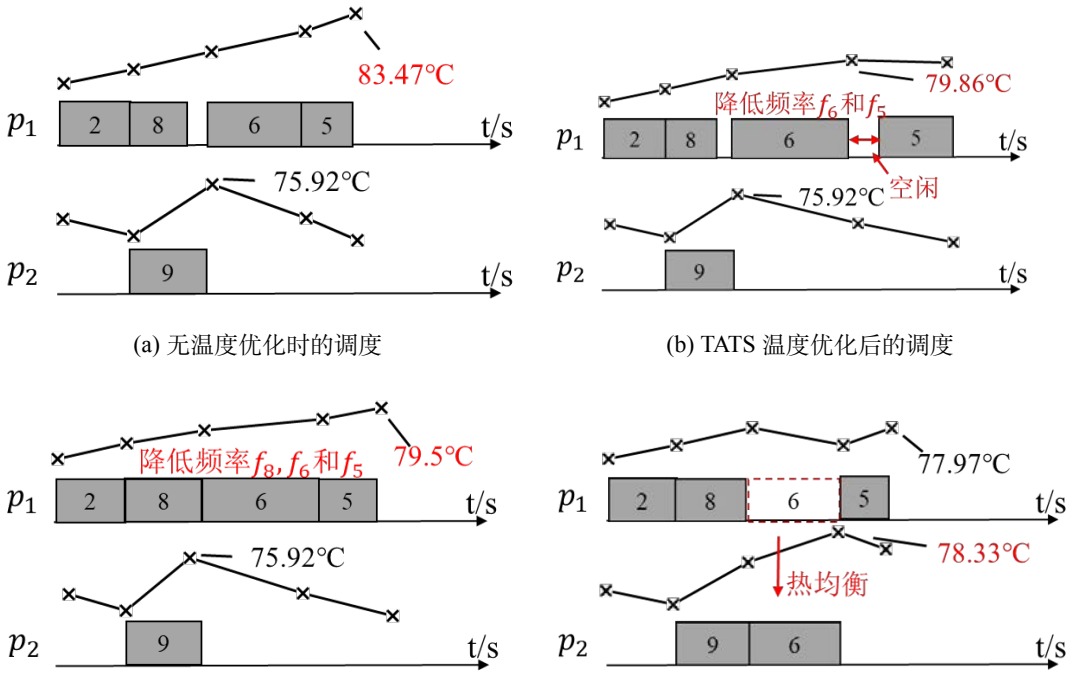
表 4.1 大核处理器参数和常量

电压 v	频率 f	常量$K_1(v)$	常量$K_2(v)$
0.90	1.010	0.1753	8.7261
0.95	1.260	0.1816	9.2661
1.00	1.810	0.1942	11.315
1.05	1.260	0.2063	13.347
1.10	2.100	0.2145	15.748

4.4.2 MPC 模型预测对温度优化的作用

为了比较本章提出的算法在温度控制方面性能，对不同场景下的温度优化方案进行对比，相关实验对比可见图 4-3。利用模型预测控制的温度预测模块，可提前对将来可

中国知网 https://www.cnki.net



(a) 无温度优化时的调度

(b) TATS 温度优化后的调度

(c) 滚动温度优化后的调度

(d) 同时进行滚动优化和热均衡温度优化后的调度

图 4-3 温度优化效果示例图

能出现的温度违反阈值的行为进行优化操作，以达到尽可能的避免温度阈值违反行为的发生。相比于在检测到温度超过阈值的行为后再进行温度优化控制的操作，更能将处理器温度限制在安全温度以下，更具有可行性和可靠性。

图 4-3a表示的是没有温度阈值约束时任务在处理器上的温度表现，可以看到芯片的峰值温度在处理器 p_1 上，且峰值温度为 $83.47°C$；图 4-3b表示的是使用第三章提出的启发式算法 TATS 中的温度优化算法，对检测到的超过温度阈值的任务进行温度控制的表现，使用的是 DVFS 和空闲插入结合的方法，降低了任务 τ_6 和 τ_5 的执行频率，并在任务 τ_5 前插入了一段空闲时间以降低处理器的峰值温度，当前可以看到芯片的峰值温度在处理器 p_1 上，且峰值温度为 $79.86°C$。相比于图 4-3a，峰值温度降低了 $3.61°C$；图 4-3c表示的是使用模型预测控制进行温度控制的示意图，提前使用 DVFS 策略，降低了任务 τ_8、τ_6 和 τ_5 的执行频率，最后处理器的峰值温度为 $79.50°C$，相比于图 4-3a中的情况降低了 $3.97°C$；图 4-3d是使用热负载均衡策略结合模型预测控制温度优化的示意图，对超过温度阈值的任务 τ_6 转移到温度更低的处理器 p_1 上执行，从而降低系统的峰值温度。可以看到当前芯片的峰值温度在处理器 p_2 上，图中处理器的最高温度为 $78.33°C$，相比于图 4-3a中的情况降低了 $5.14°C$。

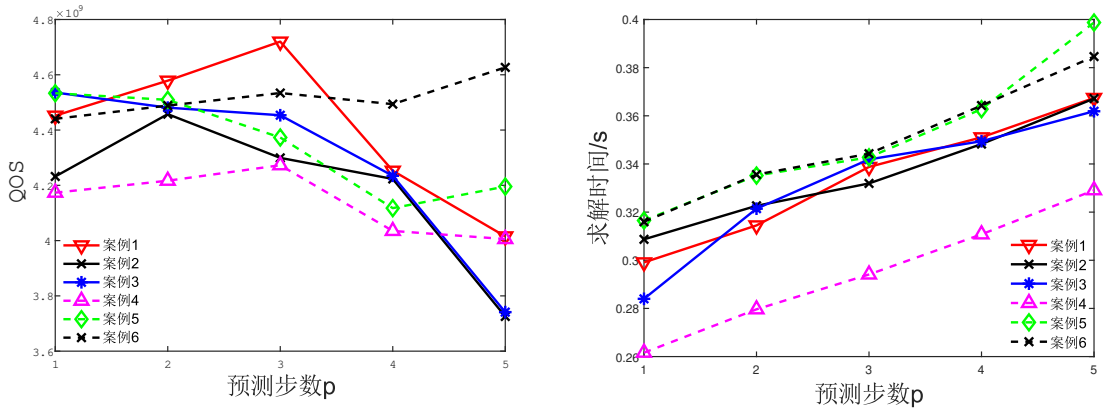
相比于第三章提出的启发式算法 TATS 中的温度优化算法，结合 DVFS 和空闲插入的温度优化方案，使用模型预测控制进行温度控制的方案得到的调度的处理器峰值温度更低，任务的最终完成时间也更短，效果更好。这主要是因为模型预测控制的滚动优化策略结合温度预测模块，对于预测到的温度违反阈值的行为，提前进行温度优化，同时

中国知网　https://www.cnki.net

结合热负载均衡策略，达到更好的温度控制效果。

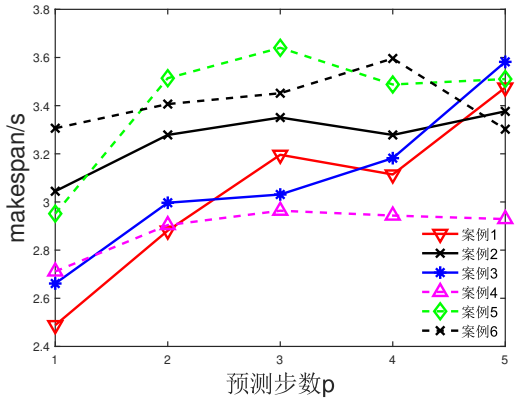
4.4.3 不同预测步数的 MPC 结果对比

不同的预测步长，具有不同的效果，为了展示不同预测步长对模型预测控制调度算法的影响，图 4-4展示了 6 个实验案例在取不同预测步数 p 值下的调度效果，不同颜色和形状的折线代表不同的案例。图 4-4分别展示了系统 QOS、求解时间、makespan 和峰值温度随预测步数变化的变化趋势。

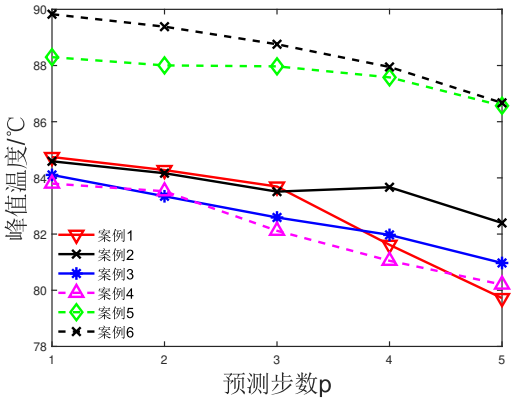


(a) 系统 QOS 对比图

(b) 求解时间对比图



(c) makespan 对比图



(d) 峰值温度对比图

图 4-4 不同预测步数下的实验对比

在系统的 QOS 方面，随着预测步数的增加，各个实验案例的系统 QOS 总体上呈现先增加后减少的趋势，可见图 4-4a。这是因为模型预测控制的预测模型在进行温度预测时，要满足所有预测区间的温度约束。因此，当预测区间内存在违反温度阈值约束的行为时，控制器会提前使用 DVFS 策略对温度进行优化，预测步数越大，当前任务的执行频率就会越低。适当的降低当前任务的执行频率可以为后续任务留有更多的执行空间，也就能执行更多的可选周期；但是如果当前任务的执行频率过低，任务执行时间会变长，为了保证任务在截止时间前执行完毕，就会出现牺牲掉部分可选周期来保证任务

中国知网 https://www.cnki.net

的实时性的情况，系统总体的 QOS 反而会减少。在计算时间方面，模型预测控制的预测步数越长，各个实验案例的整个调度方案所需的求解时间也越久。在每一次优化求解中，运算时间会随预测步数的增大而增加，呈逐步上升趋势，可见图 4-4b。

在 makespan 方面，随着模型预测控制的预测步数的增加，makespa 总体上先是增加的，后面增量趋于平缓甚至减少，可见图 4-4c。分析 makespan 先是增加是因为随着模型预测控制的预测步数的增加，系统 QOS 先是增加了，系统执行的可选周期数更多，因此需要更多的时间来执行任务；后续趋于平缓甚至有减小的情况是因为受任务的截止时间和相关性的限制，makespaan 的增加并不是无限制的。在峰值温度方面，随着预测步数的增加，各个实验案例的峰值温度总体上呈逐渐减少的趋势，可见图 4-4d。预测步数增加意味着预测区间变大了，为了满足整个预测区间内处理器的峰值温度满足温度阈值约束，控制器会提前使用 DVFS 策略对温度进行优化，因此获得更低的峰值温度。综上，在基于模型预测控制的温度感知调度方案中，并不是预测步数越大越好，预测步长的大小也会影响调度性能。在实际使用中，因根据实际应用效果选择合适的预测步长。

4.4.4　与启发式算法的性能对比

将本章提出的基于模型预测控制的温度感知调度方案与第三章提出的启发式调度算法 TATS 进行对比，分别比较了两个算法在不同实验案例下的系统 QOS、makespan、系统峰值温度和求解时间方面的性能，相关实验结果可见图 4-5和图 4-6。其中 MPC 和 MPCT 分别表示使用模型预测控制架构进行任务调度时无和有温度优化的情况，而 HEU 和 HEUT 分别表示使用启发式算法 TATS 进行任务调度时无和有温度优化的情况。在图 4-5中，柱状图用来表示各个实验案例的系统 QOS，折线图用来表示 makespan。

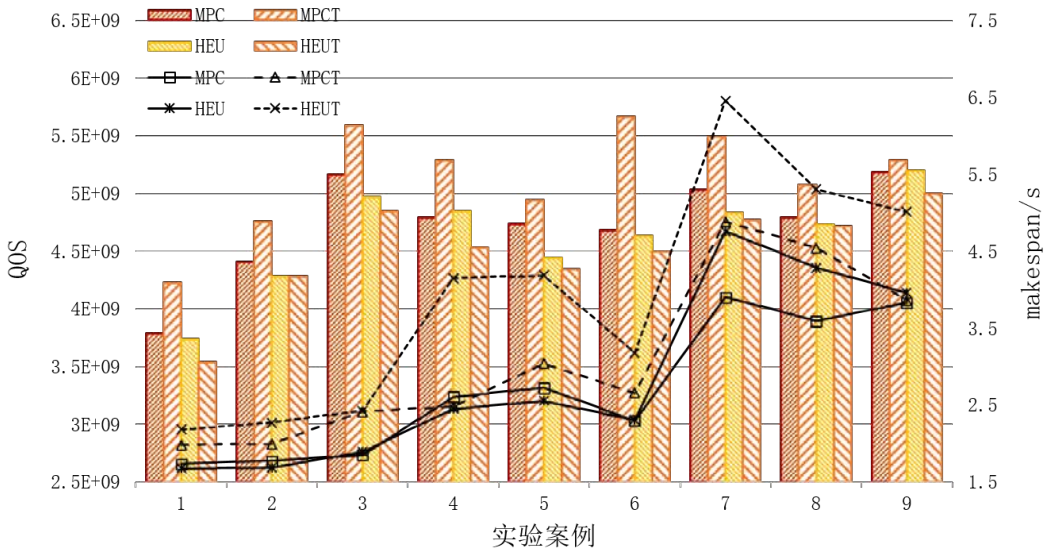


图 4-5 不同实验案例下 QOS 和 makespan 对比

与 TATS 算法对比，本章提出的基于模型预测控制的温度感知调度方案在系统 QOS 和

中国知网　https://www.cnki.net

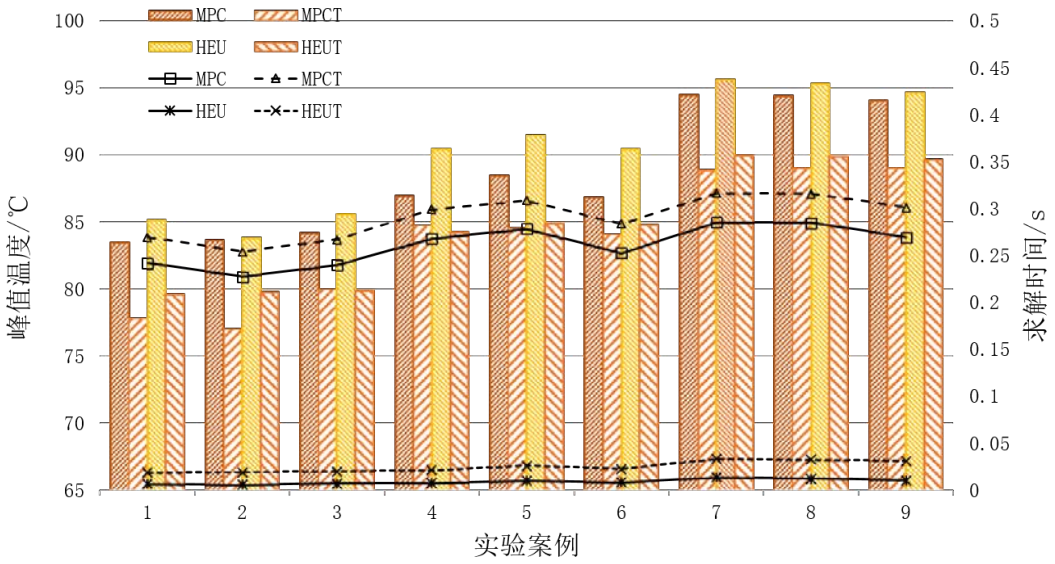


图 4-6 不同实验案例下温度和求解时间对比

makespan 上具有更好的表现，在系统 QOS 方面平均可增加 14.54%，最多可增加 26.14%；在 makespan 方面平均减少 18.58%，最多减少 40.1%。在图 4-6中，柱状图表示各个实验案例的处理器的峰值温度，折线图用来表示生成整个调度所用的求解时间。TATS 的温度优化方案是结合使用空闲插入和 DVFS 策略。本章提出的基于模型预测控制的温度感知调度方案的使用预测模块和滚动优化模块，结合了热负载均衡策略和 DVFS 策略。相比于 TATS 算法，由于使用了预测模型可提前优化处理器的温度，基于模型预测控制的调度方案在处理器的峰值温度方面有略微降低。在所有的实验案例中，基于模型预测控制的调度算法平均可减少 1.01% 的峰值温度，最多可减少 3.48%。在求解时间方面，由于模型预测控制使用滚动优化策略，循环解得当前任务的最优调度策略，因此相比于 TATS 算法，需要消耗更多的求解时间，可见图 4-6中的折线。

本章提出的基于模型预测控制的温度感知调度方案同样可适用于同构多核计算平台，并且相比于 TATS 算法，本章提出的 MPCT 算法在系统 QOS、makespan 和峰值温度方面具有更好的性能表现。但是由于 MPCT 算法使用的是滚动优化策略，对任务的调度实现先预测后执行的策略，并且执行区间是小于预测区间的，在每次优化求解中会丢弃掉部分无用的解并重复求解，在求解过程中会消耗大量的时间来获得冗余的结果，因此 MPCT 算法的整体求解时间要比 TATS 算法多。并且 MPCT 算法的的求解时间与滚动优化是设定的预测步长和执行步长相关，一般步长越小求解时间越长。此外，TATS 算法中考虑的处理器的温度模型更加准确全面，同时考虑了处理器间的横向热影响以及处理器与散热片、散热片与环境温度间的纵向热影响；而 MPCT 算法中为了实现快速简便的估算处理器温度模型，使用不考虑散热片的简单温度模型，并且忽略了处理器间的横向热影响。因此，MPCT 算法中对处理器温度的预测可能会与实际运行情况有一定的误差，在实际应用中可根据实际处理器的温度数据对 MPCT 算法中温度预测模型中

中国知网 https://www.cnki.net

的参数进行调整，同时可增加反馈调整模块，减少 MPCT 算法中温度预测模型的误差。因此，相比于 MPCT 算法，TATS 算法更适用于要求生成任务调度方案的时间更小的应用场景并且考虑的处理器温度的场景更加全面，对处理器温度的计算更加准确，两个算法可适用于不同的应用背景，不可相互替代。

4.4.5　和其他算法的对比

为了更好的体现本章提出的基于模型预测控制的温度感知调度算法的性能，本节实验选用了两种不同的算法作为对比。以下是这两种算法的简要介绍：

1. 基于模型预测控制的处理器热管理算法（Two）[83]。该算法使用模型预测控制架构进行温度控制，目标是在温度约束下使处理器性能最大化。任务模型比较简单，没有考虑近似任务以及实时性、相关性、可靠性等因素。对于温度的预测是将 RC 模型转为参数的状态矩阵进行温度预测，每个采样时间，利用 YALMIP 求解器进行求解。

2. 截止时间感知的近似任务温度感知调度算法（DELI）[79]。该算法考虑了同构多核平台上近似实时任务的温度感知调度问题，以最大化系统 QOS 为目标，考虑具有相关性的任务，而不考虑任务的可靠性和能量约束。通过空闲分布策略和处理器休眠策略来降低处理器的峰值温度。

图 4-7比较了本章提出的基于模型预测控制的温度感知调度算法（MPCT）与 Two 算法和 DELI 算法在不同实验案例下的性能对比，其中的左坐标轴标注代表处理器峰值温度的柱状图，右坐标轴标注代表系统 QOS 的折线图。此外表 4.2汇总了本章提出的基

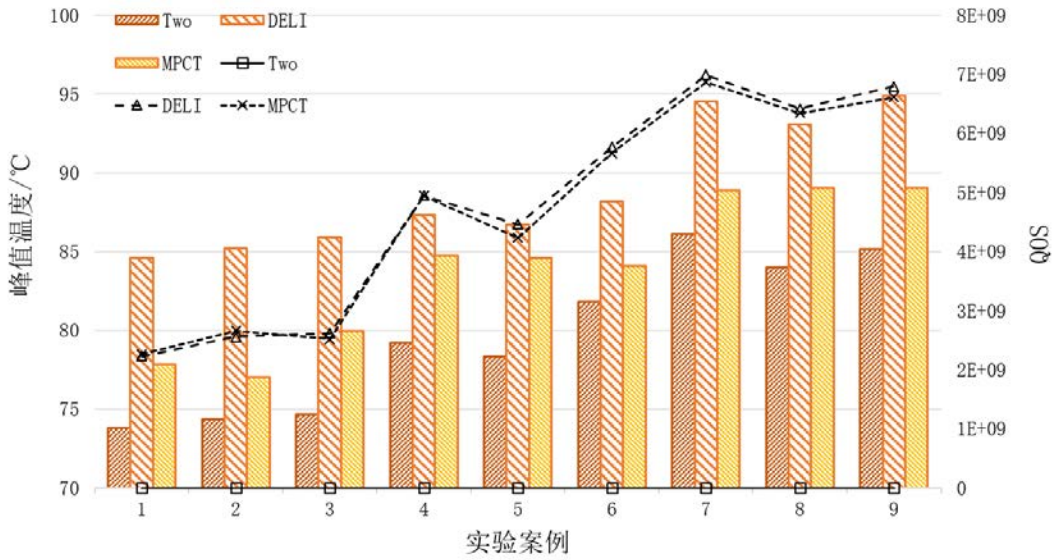


图 4-7 与其他算法的峰值温度和 QOS 对比

于模型预测控制的温度感知调度算法与 Two 算法和 DELI 算法在求解时间、系统 QOS

中国知网　https://www.cnki.net

和峰值温度上的性能表现。其中 N 表示不同的任务数量，每个任务数下对应展示了 3 条实验数据。

表 4.2 与其他算法的性能对比

	Two	$DELI$	$MPCT$	Two	$DELI$	$MPCT$	Two	$DELI$	$MPCT$
		$N=10$			$N=20$			$N=30$	
QOS	0	2313	2233.7	0	4948.5	4946.7	0	6989.2	6869.4
	0	2580.6	2562.3	0	4463.3	4232	0	6417.8	6343.7
	0	2680.1	2617.9	0	5766.9	5669.8	0	6799.4	6615.7
时间	10.536	0.166	0.269	16.189	0.260	0.299	32.092	0.359	0.316
	10.574	0.181	0.254	18.364	0.245	0.308	26.065	0.351	0.316
	11.718	0.145	0.267	15.297	0.307	0.283	26.340	0.374	0.301
温度	73.82	84.62	77.83	79.21	87.33	84.74	86.13	94.56	88.88
	74.36	85.19	77.04	78.33	86.74	84.59	84.02	93.07	89.04
	74.68	85.90	79.98	81.85	88.21	84.11	85.14	94.88	89.03

相比于 Two 算法，本章提出的方案在系统 QOS 和求解时间上具有较好表现。Two 算法也是使用模型预测控制架构进行温度控制，目标是在温度约束下使处理器性能最大化。对于处理器温度的预测模型，Two 算法是将 RC 模型转为参数的状态矩阵进行温度预测。并且在每个采样时间，利用 YALMIP 求解器对优化问题进行求解。因此，在计算时间方面，相比于 Two 算法，本章提出的 MPCT 算法具有卓越的优势，平均可减少 98.23% 的求解时间。主要原因在于 Two 算法使用求解器进行求解，单次求解时间比较长。并且 Two 算法考虑每隔一次采样时间进行一次求解，采样间隔短，求解次数较多，因此 Two 算法方案的总求解时间非常大。在系统的 QOS 方面，由于 Two 算法的任务模型不考虑近似任务，QOS 为 0。也因此，算法 Two 在能耗方面消耗不大，峰值温度也较低。

相比于 DELI 算法，本章提出的 MPCT 算法在系统峰值温度上具有较好表现。DELI 算法考虑了近似任务模型，任务需要在截止时间前完成，考虑具有相关性的任务，没有考虑任务的可靠性和能量约束。在多核处理器上对温度过高的任务，主要通过空闲分布策略和处理器休眠策略来降低处理器的峰值温度。无论是插入空闲还是休眠处理器，本质上都是推迟任务的开始执行时间，容易影响任务的实时性和相关性。这种单一的温度优化策略在调度具有相关性的实时任务时作用受限，并不能尽可能的降低处理器的峰值温度。因此同样容易造成较高的峰值温度。相比于 DELI 算法，本章提出的 MPCT 算法在系统峰值温度上平均可降低 5.68%，且可以满足温度阈值约束，但在系统 QOS 上平均减少了 1.20%。分析 QOS 减少的原因大致可分为两方面。一是因为 DELI 算法没有

中国知网 https://www.cnki.net

对处理器峰值温度进行限制，在获得更多系统 QOS 的同时，处理器的温度也处于较高水平，这与本章在温度约束下实现系统性能最大化的可靠调度的目标不一致；二是因为 TATS 算法对系统使用的总能耗进行了约束，而 DELI 算法没有，所以 DELI 算法可以无限制的执行更多的可选周期，获得更多的系统 QOS。在实际应用中，系统的资源是有限的，任务需要在有限资源下更好的展现性能，因此，对系统调度时的总能耗进行约束是合理的。

综合来看，本章提出的基于模型预测控制的温度感知调度方案在系统 QOS 上和峰值温度优化上表现良好，且不会占用太多的求解时间。相比于同类算法，本章提出的方案能大大缩减求解时间，平均可减少 98.23%，并且考虑的问题更全面复杂，更贴近实际应用。

4.5　本章小结

本章研究了多核异构平台上的 DAG 任务的温度感知调度问题，结合模型预测控制架构，提出了一个在满足任务实时性、相关性、可靠性和能量约束的以最大化系统 QOS 为目标的温度感知调度方案。根据常用的处理器温度模型，设计模型预测控制的温度预测模块，提出了快速简便预测温度的方法，结合滚动优化策略可以提前对可能违反温度阈值的情况进行调整，并验证了温度优化方案的高效性。此外，本章对模型预测控制控制器进行合理设计，使得整体方案求解快速便捷，并结合模型预测控制的特性设计了温度优化控制方案。通过结合使用热负载均衡和 DVFS 策略，对处理器温度进行合理的优化控制。相比于没有进行温度预测的启发式算法，本章提出的算法在系统 QOS 和 makespan 上性能更优。在系统 QOS 上，平均可增加 14.54%，最多可增加 26.14%；在 makespan 上，平均可减少 18.58%，最多减少 40.1%。相比于同样使用模型预测控制进行调度的一般调度算法，本章提出的算法能大大缩减求解时间，平均可减少 98.23%，并且适用于具有相关性的实时任务，考虑的问题模型更复杂全面，贴合实际应用。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第五章 异构多核平台上实时任务的可靠温度感知调度设计

5.1 引言

随着半导体技术的提升，芯片的特征尺寸持续缩短，功耗密度和集成度的增加使得芯片工作时的平均温度持续上升，带来了任务执行中断、电子元件加速老化、电路永久故障发生概率增加、处理器寿命减短等问题。基于多核异构平台，本章研究的问题是提出以最大化系统平均无故障时间为目标，满足任务实时性、相关性、可靠性和系统峰值温度约束的可靠温度感知调度方案。针对过高的温度会对处理器造成负面影响，本章同时考虑了瞬时故障和永久故障，并根据对处理器硬件影响较大的电迁移和热循环机制综合考虑建立系统平均无故障时间模型。此外增加任务的执行频率可以减小任务瞬时故障发生的概率，但同时也会增加处理器的温度，增加系统永久故障发生的概率。因此，本章考虑的问题是在满足任务可靠性阈值的基础上最大化系统的平均无故障时间。本章的主要工作有：1）为了尽可能的移除热循环机制对处理器硬件的热影响，研究了如何最小化处理器上温度变化范围的问题；2）快速确定满足目标和约束的任务最优频率的方法；3）为了优化处理器的峰值温度，提出了空闲时间大小可计算的空闲分布策略；4）提出了基于系统平均无故障时间最大化的可靠温度感知调度算法，并通过仿真实验对比验证了该算法的性能。

5.2 系统模型和问题定义

由于本章的研究内容是解决基于异构多核平台的 DAG 任务的可靠温度感知调度问题，处理器模型和温度模型直接引用第四章中的内容，其他的任务模型、功耗模型和可靠性模型直接引用第三章中的内容。

5.2.1 芯片平均无故障时间模型

芯片的寿命模型通常用系统平均无故障时间（Mean Time To Failure, MTTF）来衡量。这里的故障是指处理器上发生的永久故障。与可恢复的瞬时故障不同，永久故障是指电路或电子元件由于制作工艺误差、老化、电迁移、热循环等原因，发生的永久性的硬件损伤。永久故障的发生与系统的温度密切相关。导致处理器永久故障发生的几种机制已在 2.2.1小节中进行了详细的介绍，同时对考虑所有机制对处理器硬件的影响比较复杂。文献 [84] 已通过实验证明影响处理器的寿命的主要因素是电迁移（Electro-Migration, EM）和热循环（Thermal Cycling, TC）机制，因此本文主要考虑这两种机制对处理器平均无故障时间的影响。

中国知网 https://www.cnki.net

电迁移[71] 是指在电路互连时，金属离子在电场作用下出现了迁移运动，导致电路出现开路或者短路的故障，从而使电路失效。电迁移机制对应的系统平均无故障时间可表示为：

$$MTTF_{EM} = \frac{A_{EM}}{J^n} e^{\frac{E_a}{KT}}, \tag{5.1}$$

其中 A_{EM} 表示电路中基于互连结构和材料的常数，J 表示金属互联线的电流密度，n 是经验常数，取决于材料，E_a 是激活能，K 是玻尔兹曼常数，T 表示处理器的开尔文温度。从公式 (5.1)中可以明显看出，系统的平均无故障时间与处理器的温度呈负相关性关系，即处理器温度越低，系统受电迁移机制的负面影响越小，处理器的平均无故障时间就越长。

热循环[74] 是指电路中电子元件在温度不断变化的环境下，由于电介质材料和金属间膨胀系数存在差异，使材料层出现疲劳变形、裂纹等现象，最终导致电路产生永久故障。热循环对应的系统平均无故障时间可表示为：

$$MTTF_{TC} = b \left(\frac{1}{\Delta T} \right)^q, \tag{5.2}$$

其中 b 是与材料相关的正常数，ΔT 表示处理器运行时的温度变化，q 是科芬-曼森指数。从公式 (5.1)中可以看出，系统平均无故障时间与处理器的温度变化呈负相关性关系，即处理器上的温度变化范围越小，系统受热循环机制的负面影响越小，处理器的平均无故障时间就会越长。

为了建立由电迁移机制和热循环机制共同作用下的处理器平均无故障时间模型，采用文献 [84] 中描述的独立故障机制的平均无故障时间模型。相比于传统的考虑恒定故障率的 SOFR[85] 模型，该模型对处理器的平均无故障时间的描述具有更高的准确度。因此本章考虑基于独立故障机制的平均无故障时间模型，单个处理器 n 的平均无故障时间可表示为处理器上两种故障机制对应的系统平均无故障时间的最小值，如下：

$$MTTF_n = \min\{MTTF_{EM,n}, MTTF_{TC,n}\} = \min\left\{ \frac{A_{EM}}{J^n} e^{\frac{E_a}{KT_n(t)}}, b\left(\frac{1}{\Delta T_n(t)} \right)^q \right\}, \tag{5.3}$$

其中 $MTTF_{EM,n}$ 和 $MTTF_{TC,n}$ 分别表示处理器 n 上电迁移机制和热循环机制作用下的处理器平均无故障时间。$T_n(t)$ 表示 t 时刻处理器 n 上的温度，$\Delta T_n(t) = \max\{T_n(t)\} - T_e$ 表示处理器 n 在运行期间的温度变化范围，最小值一般可视为处理器初始温度，这里用环境温度代替。扩展到多核处理器平台，整个系统的平均无故障时间是所有处理器的平均无故障时间的最小值，得到系统的平均无故障时间可表示为：

$$MTTF_{sys} = \min\{MTTF_n\},\ \forall n \in \mathcal{M}. \tag{5.4}$$

5.2.2 问题描述

基于以上模型，本章的研究内容是对于给定的具有相关性的实时任务，同时考虑系统瞬时故障和永久故障的发生，目标是在任务实时性、相关性、可靠性和温度约束下，

中国知网 https://www.cnki.net

最大化系统的平均无故障时间。针对瞬时故障对任务执行的影响，建立任务可靠性模型，保障每个任务的可靠执行；针对永久故障对处理器硬件的影响，综合考虑导致系统永久故障发生的电迁移和热循环两种主要机制，建立描述系统平均无故障时间的模型，尽可能减少系统永久故障发生概率，增加系统寿命。本章的研究问题是设计以最大化系统 MTTF 为目标，同时满足任务可靠性、实时性、相关性和系统温度的可靠温度感知调度方案。因此，本章需要确定的变量有：1）任务到处理器的映射，2）任务的频率选择，3）任务执行的开始时间，4）任务执行的先后顺序。优化问题可表示为：

$$\max_{\boldsymbol{X_{i,n}},\boldsymbol{Y_{i,l}},\boldsymbol{ts_i},\boldsymbol{Q_{i,j}}} MTTF_{sys} = \max\left\{\min\left\{\frac{A_{EM}}{J^n}e^{\frac{E_a}{KT_n(t)}}, b\left(\frac{1}{\Delta T_n(t)}\right)^q\right\}\right\},\ \forall n \in \mathcal{M} \tag{5.5}$$
$$\text{s.t. } (3.11) \sim (3.12),\ (3.14) \sim (3.17),\ (3.24)$$

处理器上的温度与任务分配情况和任务执行的频率相关。一般来说，处理器在执行任务时使用的频率越高，处理器上的温度增涨的越快，越容易产生较高的芯片温度。而高温会增加系统的永久故障发生概率，影响系统的寿命和可靠性，违背了本章的最大化系统平均无故障时间的目标。为了进一步描述处理器温度对约束和目标函数的影响，引入 0-1 变量 $\beta_{i,j,n}$ 来表示任务在多核处理器系统中执行时的重叠情况。当 $\beta_{i,j,n}=1$ 时，表示在任务 τ_i 开始执行时，在处理器 n 上存在任务 τ_j 处于执行中状态，即 $X_{j,n}=1$ & $ts_j \le ts_i \le te_j - \epsilon$，$\epsilon$ 是极小的正常数。为了获得正确的 $\beta_{i,j,n}$ 值，在第 三章由公式 (3.15) 和 (3.16)描述的任务执行顺序约束的基础上，对任务的开始时间和结束时间加入以下约束：

$$\begin{gathered}
\forall i \neq j \in \mathcal{T},\ \forall n \in \mathcal{M}, \\
te_j \ge ts_i + (\beta_{i,j,n} - 1)\,H, \\
ts_j \le ts_i + (1 - \beta_{i,j,n})\,H, \\
\beta_{i,j,n} + X_{i,n} \le 1, \\
te_j - \epsilon - (1 - O_{j,i})H - (1 - X_{j,n})H \le ts_i + \beta_{i,j,n}H,
\end{gathered} \tag{5.6}$$

考虑相邻处理器的影响，处理器的温度可表示如下：

$$T_{i,n} = T_e + \frac{1}{G_{n,e}}\sum\nolimits_{j=1}^{N}\left(\beta_{i,j,n}P_{i,l}\right) + \frac{1}{G_{n,e}}\sum\nolimits_{m=1}^{M} G_{n,m}\left(T_n(t) - T_m(t)\right), \tag{5.7}$$

其中 $T_{i,n}$ 表示在执行任务 τ_i 时，第 n 个处理器的温度，且功耗 $P_{i,l} = C_{eff}v_l^2 f_l + K_1(v_l)T + K_2(v_l)$，代入可得：

$$\begin{aligned}
T_{i,n} = T_e + \frac{1}{G_{n,e}}\sum\nolimits_{j=1}^{N}\left\{\beta_{i,j,n}\left[K_1(v_l)T_{i,n} + \sum\nolimits_{l=1}^{l_{sum}} Y_{i,l}\left(C_{eff}v_l^2 f_l + K_2(v_l)\right)\right]\right\} \\
+ \frac{1}{G_{n,e}}\sum\nolimits_{m=1}^{M} G_{n,m}\left(T_n(t) - T_m(t)\right),
\end{aligned} \tag{5.8}$$

上述处理器温度计算公式考虑了相邻处理器间的热影响。上述公式也可以表示处理器的温度值与处理器执行的频率 $Y_{i,l}$ 和任务在处理器的映射情况相关。因此，处理器上的温

中国知网　https://www.cnki.net

度约束可改写为：

$$T_{i,n} \leq T_{thr},\ \forall i \in \mathcal{N},\ \forall n \in \mathcal{M}, \tag{5.9}$$

同时，公式 (5.5)的优化问题可改写为：

$$\max_{\boldsymbol{X_{i,n}},\boldsymbol{Y_{i,l}},\boldsymbol{ts_i},\boldsymbol{Q_{i,j}},\boldsymbol{T_{i,n}}} MTTF_{sys} = \max\left\{\min\left\{\frac{A_{EM}}{J^n}e^{\frac{E_a}{KT_{i,n}}}, b\left(\frac{1}{\Delta T_{i,n}}\right)^q\right\}\right\},\ \forall i \in \mathcal{N},\ \forall n \in \mathcal{M}, \tag{5.10}$$

$$\text{s.t. } (3.11) \sim (3.12),\ (3.14) \sim (3.17),\ (5.9)$$

其中 $\Delta T_{i,n} = \max T_{i,n} - T_e$。

5.3 算法设计

本章同时考虑系统瞬时故障和永久故障对异构多核处理器系统可靠性的影响。针对瞬时故障，根据公式 (3.2)展示的任务可靠性模型，通过动态电压/频率调整技术技术提高任务的执行频率可以减小任务瞬时故障的发生概率，保障任务的可靠执行，但是频率的增加意味着执行相同的任务处理器会增加更多的功耗和热，从而获得更高的系统峰值温度；而根据处理器的平均无故障时间公式 (5.1)和公式 (5.2)，处理器温度的增加会影响处理器的寿命和可靠性，减少处理器的平均无故障时间，并且处理器上温度的变化范围的越大，处理器的平均无故障时间降低的越多。此外，现有的工作大多只考虑单一机制对硬件永久故障的影响，忽略了热循环机制对处理器的负面影响。因此，本章联合考虑电迁移和热循环机制对永久故障的影响，建立系统平均无故障模型，考虑到系统中瞬时故障和永久故障间的冲突，本章考虑在保障任务的可靠执行的前提下，满足任务实时性、相关性和温度约束的同时，实现最大化系统的平均无故障时间的优化目标。

5.3.1 任务温度变化范围对处理器温度变化范围的影响

根据处理器系统的平均无故障时间模型，可以得到处理器温度越低且处理器温度变化越低，处理器的平均无故障时间越长的结论。因此如何最大化系统的平均无故障时间的问题可以转换为如何最小化处理器上的温度变化范围的问题。为了求解该问题，本文给出并证明了推论 5.1。

定义 5.1：当处理器处于开机状态但是不执行任务时，处理器是在执行动态功耗为 0 的空闲任务。

推论 5.1：每一个任务运行期间的温度变化范围最小化（$\min \Delta T_i,\ \forall i \in \mathcal{T}$）是整个处理器上温度变化范围最小化的充分条件（$\min \Delta T$）。

证明：假设任务集中有 N 个任务，处理器的初始温度为 T_0。并且考虑稳态温度计算模型，即任务在其执行前后的温度变化是单调一致的，即只能升高或者降低或者持平不变，不会出现波动的情况。令 ΔT_{i-1}^i 表示第 i 个任务的温度变化范围，则 ΔT_0^i 表示处理器上开始到执行完第 i 个任务的总温度变化范围。

中国知网 https://www.cnki.net

（1）当只有一个任务，即 $N=1$ 时，有温度值 $\{T_0,T_1\}$，T_1 表示执行完任务 τ_1 的温度。温度变化情况有三种：温度升高、温度降低和温度保持不变。无论哪种变化情况，温度变化范围都可表示为 $\Delta T_0^1=T_{max}-T_{min}=|T_1-T_0|=|\Delta T_0^1|$。通过最小化任务 τ_1 的温度变化范围 $|\Delta T_0^1|$ 可以最小化整个处理器的温度变化范围。因此，当处理器上只有一个任务时，推论成立。

（2）当 $N=2$ 时，当处理器只执行完任务 τ_1 还未执行任务 τ_2 时，此时处理器上的温度有一个最大值 T_{max} 和最小值 T_{min}，且两个值都可在 $\{T_0,T_1\}$ 在中产生。令执行完任务 τ_2 后的温度值为 T_2，讨论以下三种情况：

1. T_2 没有改变原来处理器的最大值或者最小值，即 $T_{min}\le T_2\le T_{max}$。此时，处理器温度变化范围 $\Delta T_0^2=|\Delta T_0^1|$，最小化任务 τ_1 和任务 τ_2 的温度变化范围是最小化整个设备的温度变化范围的充分条件，推论成立。

2. T_2 改变了原来处理器的最大值，即 $T_{max}=T_2$。此时，处理器的温度变化范围为 $\Delta T_0^2=T_2-T_{min}$。若温度最小值是 T_0，即 $T_{min}=T_0$，显然 $T_0\le T_1\le T_2$，所以处理器的温度变化范围 $\Delta T_0^2=T_2-T_0=T_2-T_1+T_1-T_0=|T_2-T_1|+|T_1-T_0|=\Delta T_1^2+\Delta T_0^1$。因此，最小化处理器的温度变化范围 ΔT_0^2 等价于最小化任务 τ_1 和任务 τ_2 的温度变化范围 $\{\Delta T_0^1,\Delta T_1^2\}$，推论成立。若温度最小值是 T_1，即 $T_{min}=T_1$，处理器的温度变化范围 $\Delta T_0^2=T_2-T_1=\Delta T_1^2$。因此，最小化任务 τ_2 的温度变化范围可以达到最小化整个设备的温度变化范围的效果，推论成立。所以第二种情况下，推论成立。

3. T_2 改变了原来处理器的最小值，即 $T_{min}=T_2$。此时，处理器的温度变化范围为 $\Delta T_0^2=T_{max}-T_2$。若温度最大值是 T_0，即 $T_{max}=T_0$，显然 $T_2\le T_1\le T_0$，所以处理器的温度变化范围 $\Delta T_0^2=T_0-T_2=T_0-T_1+T_1-T_2=|T_0-T_1|+|T_1-T_2|=\Delta T_0^1+\Delta T_1^2$。此时，推论成立。若温度最大值是 T_1，即 $T_{max}=T_1$，处理器的温度变化范围 $\Delta T_0^2=T_1-T_2=\Delta T_1^2$。推论成立。因此第三种情况下，推论成立。
   总结，当 $N=2$ 时，推论成立。

（3）假设当 $N\le n$ 时，推论成立。处理器上温度有的最大值 T_{max} 和最小值 T_{min} 在 $\{T_0,T_1...,T_n\}$ 在中产生，且 $\Delta T_0^n=T_{max}-T_{min}$。根据第二数学归纳法，假设当 $N\le n$ 时，推论成立，如果可以证明当 $N=n+1$ 时，推论仍然成立，就可以退出推论是正确的。

（4）当 $N=n+1$ 时，对于新执行的任务 τ_{n+1} 所获得的温度值 T_{n+1}，讨论以下三种情况：

1. T_{n+1} 没有改变原来处理器的最大值或者最小值，即 $T_{min}\le T_{n+1}\le T_{max}$。此时的问题与 $N=n$ 时等价。因此，推论成立。

中国知网　https://www.cnki.net

2. T_{n+1} 改变了原来处理器的最大值，即 $T_{max} = T_{n+1}$。此时，处理器的温度变化范围为 $\Delta T_0^{n+1} = T_{n+1} - T_{min}$。若温度最小值是 T_n，即 $T_{min} = T_n$，显然 $T_n \leq T_{n+1}$，所以处理器的温度变化范围 $\Delta T_0^{n+1} = T_{n+1} - T_n = \Delta T_n^{n+1}$。因此，最小化任务 τ_{n+1} 的温度变化范围可以达到最小化整个设备的温度变化范围的效果，推论成立。若温度最小值是 $\{T_0, T_1, ..., T_{n-1}\}$ 中的某个时，处理器的温度变化范围 $\Delta T_0^{n+1} = T_{n+1} - T_n + T_n - T_{min} = \Delta T_n^{n+1} + \Delta T_{min}^n$。$\Delta T_{min}^n$ 同样满足推论。因此，最小化每一个任务的温度变化范围以达到最小化整个设备的温度变化范围 ΔT_0^{n+1} 的效果。第二种情况下，推论成立。

3. T_{n+1} 改变了原来处理器的最小值，即 $T_{min} = T_{n+1}$。此时，使用在第二种情况时同样的推导方法可得，在第三种情况下，推论成立。
总结，当 $N = n + 1$ 时，推论成立。

（5）综上所述，推论成立，即使得每个任务的温度变化范围最小化可以使得整个处理器的温度变化范围最小化。

5.3.2 温度变化范围最小化的任务最优执行频率

上一小节讨论并证明了任务执行时的温度变化范围与处理器整体温度变化范围的关系，得到了将处理器上每个任务的温度变化范围最小化可以实现整个处理器上温度变化范围最小化的推论。将如何使得处理器上温度变化范围最小化的问题转换为如何使得每个任务的温度变化范围最小化的问题。

根据公式 (4.12)推导出的处理器 n 温度计算的表达式，可以得到在处理器上执行任务 τ_i 前后的温度表达式：

$$T_i = \frac{C_{eff} v_l^2 f_l + K_2(v_l) + G_{n,e} T_e}{G_{n,e} - K_1(v_l)} \left(1 - e^{-\frac{G_{n,e} - K_1(v_l)}{C_n} \Delta t}\right) + T_{i-1} e^{-\frac{G_{n,e} - K_1(v_l)}{C_n} \Delta t}, \tag{5.11}$$

其中 T_i 和 T_{i-1} 分别表示任务 τ_i 执行前和后的温度，Δt 表示任务 τ_i 的执行时间。令 $\theta_1 = \frac{G_{n,e} - K_1(v_l)}{C_n}$，$\theta_2 = \frac{C_{eff} v_l^2 f_l + K_2(v_l) + G_{n,e} T_e}{C_n}$，则公式可表示为：

$$T_i = T_{i-1} e^{-\theta_1 \Delta t} + \frac{\theta_2}{\theta_1} \left(1 - e^{-\theta_1 \Delta t}\right), \tag{5.12}$$

因此，执行任务 τ_i 的温度变化范围 ΔT_{i-1}^i 可表示如下：

$$\Delta T_{i-1}^i = T_i - T_{i-1} = \left(T_{i-1} - \frac{\theta_2}{\theta_1}\right) \left(e^{-\theta_1 \Delta t} - 1\right), \tag{5.13}$$

$$\left|\Delta T_{i-1}^i\right| = \left|\left(T_{i-1} - \frac{\theta_2}{\theta_1}\right)\right| \left|\left(e^{-\theta_1 \Delta t} - 1\right)\right|, \tag{5.14}$$

因此，最小化任务执行时的温度变化范围 $\left|\Delta T_{i-1}^i\right|$ 等价于最小化 $\left|\left(T_{i-1} - \frac{\theta_2}{\theta_1}\right)\right|$ 和 $\left|\left(e^{-\theta_1 \Delta t} - 1\right)\right|$ 这两项式子。当任务执行时的温度变化范围越小，热循环机制对处理器的负面影响越

中国知网 https://www.cnki.net

小。因此，为了尽可能减少热循环机制对处理器平均无故障时间的影响，令 $\left|\Delta T_{i-1}^{i}\right|=0$，得：

$$e^{-\frac{G_{n,e}-K_1(v_l)}{C_n}\Delta t}=1, \tag{5.15}$$

$$\frac{C_{eff}v_l^2 f_l+K_2(v_l)+G_{n,e}T_e}{G_{n,e}-K_1(v_l)}-T_{i-1}=0, \tag{5.16}$$

等式 (5.15)中，$G_{n,e}$ 和 C_n 是与处理器硬件相关的正常数，$K_1(v_l)$ 是与处理器工作电压相关的正常数，且一般情况下，$\frac{G_{n,e}-K_1(v_l)}{C_n}$ 恒小于 0。因此，当且仅当 $\Delta t=0$ 时，等式 (5.15)成立。又因为在上一节中，已经将处理器处于开机但不执行任务时的状态定义为是在执行空闲任务。因此，当任务执行时间为 0 时，无实际意义，排除不作考虑。对于等式 (5.16)，将 T_{i-1} 看作已知量，可得任务执行频率的表达式，如下：

$$f_l=\frac{T_{i-1}(G_{n,e}-K_1(v_l))-K_2(v_l)-G_{n,e}T_e}{C_{eff}v_l^2}, \tag{5.17}$$

因此，根据公式 (5.17)，可以得到最小化任务执行时温度变化范围的最优频率，从未最小化系统中处理器上的温度变化范围，消除热循环对处理器的热影响。然而，虽然可以求解得出移除热循环效应的最优频率，但是求解得到的频率却不一定有实际意义，即无法保证求解得到的最优频率一定在处理器可提供的离散的工作电压/频率 $\mathcal{VF}_{\mathcal{B}}\triangleq\{(v_1,f_1),\ldots,(v_{l_B},f_{l_B})\},\ldots,(v_{l_B+l_L},f_{l_B+l_L})\}$ 中。另外，基于实时任务，任务频率的选择还需保障任务实时性和可靠性约束。因此，当处理器无法提供任务的最优执行频率时，选取处理器可提供的、满足实际需求的频率中最接近理论求解的最优频率作为任务的实际执行频率。求解出的任务最优频率将作为求解任务实际频率的部分依据，具体内容将在小节详细叙述。

5.3.3　基于 MTTF 的可靠温度感知调度算法设计

在上面两节中，针对热循环对处理器硬件寿命的负面影响，讨论并证明了最小化每个任务执行的温度变化可以最小化处理器上的温度变化，从而尽可能的延长处理器的平均无故障时间。且通过对处理器温度计算公式进行推导，可以得到理论上的任务执行的最优频率，消除热循环机制对处理器的负面影响。针对以最大化系统平均无故障时间为目标，同时满足任务实时性、相关性、可靠性和温度约束的调度问题，本节设计了可靠温度感知调度方案，对于任务到处理器的映射、任务的频率选择以及任务执行的开始时间进行了合理设计。算法伪代码可见 5.1。

首先，对任务和处理器进行初始化工作（第 1∼ 2 行）。对等待调度的 DAG 任务按照深度大小顺序排序，可见图 4-2，并获得任务的相关数据，如相关性 $\mathcal{R}$、截止时间 $\mathcal{D}$。初始化处理器的状态，包括最早空闲时间 $ptime=\{0\}$ 和初始温度 $pT=\{T_0\}$，T_0 表示初始温度。根据任务的相关性可获得 DAG 任务的先后顺序，确定每个任务到处理器上的分配、任务频率选择和任务的开始时间（第 3∼24 行）。对每个任务，优先选择空闲时间最早的处理器执行任务。并且在开始时间相同情况下，选择温度最低的处理器

中国知网　https://www.cnki.net

(第 4 行)。在 4.4.2小节中，通过实验验证过，在同等条件下，使用热负载均衡策略可获得更低处理器温度。因此，此处通过最早开始时间策略和热负载均衡策略综合考虑，选择任务执行的处理器是合理有效的。

任务频率的选择不仅要综合考虑对永久故障和瞬时故障的影响，还需保证任务能在截止时间前完成（第 4∼12 行)。针对瞬时故障，任务可靠性模型，求解得到满足任务可靠性的最低频率 $f_{min,1}$（第 5 行)；针对任务的实时性约束，由于任务间存在依赖关系，除了要考虑当前任务的截止时间，还需保障当前任务的执行不会影响其后继任务的实时性约束。因此，求解当前任务的最晚结束时间，可获得使任务的结束时间满足最晚结束时间约束的最低频率 $f_{min,2}$。其中 ls^A_{start} 表示任务 τ_i 的所有后续任务的最晚开始时间集合（第 6∼10 行)。从而得到任务可执行频率范围的最小值（第 11 行)。针对永久故障，电迁移和热循环对处理器的影响，如果当前处理器上受电迁移影响较大，为了尽可能延长处理器的平均无故障时间，需要选择使处理器温度最低的频率，即可选频率集中满足大于频率范围的最小值的最小频率（第 14 行)。反之，如果当前处理器上受热循环影响较大，则根据公式 (5.17)求解得出使任务温度变化范围最小的最优频率 f_o（第 16 行)。最终任务的频率确定为在频率集中，满足最小值要求即 $f^{'} \geq \max\{f_{min,1}, f_{min,2}\}$ 且与温度变化范围最优频率 f_o 差值最小的频率（第 17 行)。

根据任务当前确定的处理器和频率，根据公式 (4.13)可以快速估计任务的结束时的处理器温度 T_i。如果当前处理器温度超过温度阈值，使用空闲分布策略来降低处理器的温度。在 3.4.2小节中，通过实验对比的方法已经展示和证明在任务执行前插入一断空闲时间，可以有效降低处理器的峰值温度。而空闲时间可以定义成无动态功耗的空闲任务。当任务 τ_i 温度超过温度阈值时，假设在任务之前插入一个执行时间为 Δt 的空闲任务，可以使得最终任务 τ_i 执行结束后的温度满足温度阈值。令空闲任务结束时的温度为 $T_{\Delta t}$，可得:

$$T_{\Delta t} = \frac{K_2(v_l) + G_{n,e}T_e}{G_{n,e} - K_1(v_l)} \left(1 - e^{-\frac{G_{n,e}-K_1(v_l)}{C_n}\Delta t}\right) + T_{i-1}e^{-\frac{G_{n,e}-K_1(v_l)}{C_n}\Delta t}, \tag{5.18}$$

$$T_{thr} = T_i = \frac{C_{eff}v_l^2 f_l + K_2(v_l) + G_{n,e}T_e}{G_{n,e} - K_1(v_l)} \left(1 - e^{-\frac{G_{n,e}-K_1(v_l)}{C_n}c_i}\right) + T_{\Delta t}e^{-\frac{G_{n,e}-K_1(v_l)}{C_n}c_i}, \tag{5.19}$$

其中 $G_{n,e}$、C_n 和 C_{eff} 是关于处理器硬件相关的正常数，$K_1(v_l), K_2(v_l)$ 是与处理器工作电压相关的正常数。令 $\theta_1 = \frac{G_{n,e}-K_1(v_l)}{C_n}$，将等式 (5.18)和 (5.19)联列，可得空闲任务执行时长 Δt 的表达式，如下:

$$\Delta t = \frac{1}{\theta_1} \ln \frac{T_{i-1}(G_{n,e} - K_1(v_l)) - K_2(v_l) - G_{n,e}T_e}{T_{thr}(G_{n,e} - K_1(v_l)) - C_{eff}v_l^2 f_l(1 - e^{-\theta_1 c_i})}, \tag{5.20}$$

因此，当任务温度超过阈值温度时，通过在任务执行前插入一段长度为 Δt 的空闲任务，可将任务的结束温度控制在安全阈值以下（第 21 行)。如果任务结束温度满足温度阈值约束，则无需插入空闲时间。

中国知网 https://www.cnki.net

算法 5.1 基于 MTTF 的可靠温度感知调度算法（MRTS）

输入: 任务集 $\mathcal{T}$, 处理器集 $\mathcal{M}$, 频率集 $\mathcal{VF}_{\mathcal{B}}, \mathcal{VF}_{\mathcal{L}}$，任务顺序 $order$，相关性 $\mathcal{R}$，截止时间 $\mathcal{D}$

输出: 可靠温度感知调度方案

1: $p^{'}$ 和 $f^{'}$ 用于存放任务执行的处理器和频率，ts 和 te 表示任务的开始和结束时间
2: 处理器最早空闲时间 $ptime = zeros(1, M)$，处理器温度 $pT = \{T_0\}$，T_0 表示初始温度
3: **for** $\forall i \in \mathcal{T}$ **do**
4: 　根据最早开始时间策略和热负载均衡策略选择任务 τ_i 的处理器 $p_i^{'}$
5: 　根据任务可靠性公式 (3.2)和约束 (3.17)计算任务的最低频率 $f_{min,1} = \frac{-\eta c_i \times 10^s}{\ln R_{min}}$
6: 　**for** $\forall i \in \mathcal{T}$（逆序） **do**
7: 　　$ls_{end,i} = min(d_i, ls^A_{start})$ // 最晚结束时间
8: 　　$ls_{start,i} = ls_{end,i} - ex_i$ // 最晚开始时间
9: 　**end for**
10: 　根据最晚结束时间获得任务的最低频率 $f_{min,2} = \frac{c_i}{ls_{end,i} - ts_i}$
11: 　确定任务频率 $f_i^{'}$ 的最小值，即 $f_i^{'} \geq \max\{f_{min,1}, f_{min,2}\}$
12: 　根据公式 (5.1)和(5.2)计算 $MTTF_{EM}$ 和 $MTTF_{TC}$
13: 　**if** $MTTF_{EM} < MTTF_{TC}$ **then**
14: 　　确定任务频率 $f_i^{'} \geq \max\{f_{min,1}, f_{min,2}\}$ 且是频率集中与其差值最小的频率
15: 　**else**
16: 　　根据公式 (5.17)求解任务的最优频率 f_o
17: 　　确定任务频率 $f_i^{'}$，需满足 $f_i^{'} \geq \max\{f_{min,1}, f_{min,2}\}$ 且是频率集中与 f_o 差值最小的频率
18: 　**end if**
19: 　根据公式 (4.13)预估任务的结束时的温度 T_i
20: 　**if** $T_i > T_{thr}$ **then**
21: 　　根据公式 (5.20)，在任务 τ_i 插入时间长度为 Δt 的空闲任务
22: 　**end if**
23: 　获得任务的 ts_i、te_i、$p_i^{'}$ 和 $f_i^{'}$，更新处理器的最早空闲时间 $ptime$ 和温度 pT
24: **end for**
25: 所有任务调度完毕，获得最终的调度方案

最后得到任务 τ_i 的开始 ts_i 和结束时间 te_i，以及任务分配的处理器和频率, 并更新系统中处理器的最早空闲时间 $ptime$ 和温度 pT（第 23 行）。当所有任务执行遍历完毕，可得到最终的可靠温度感知调度方案（第 25 行）。

中国知网　https://www.cnki.net

5.4 仿真结果

5.4.1 实验设置

本文中的所有仿真均在Matlab上仿真实现。由于本章考虑类似大小核结构的异构多核处理器系统，小核处理器的相关参数可见表3.2，大核处理器的相关参数可见表4.1。关于系统平均无故障时间相关的参数采用文献[86]中的设置，其中 $A_{EM} = 6.4 \times 10^{-8} cm^2$，$J = 15 \times 10^6 A/cm^2$，$E_a = 0.48eV$。实验主要通过生成不同周期数和任务数的任务集进行实验模拟。由于本章考虑的是没有可选周期的一般任务，每个任务只有强制周期而没有可选周期且每个任务的周期数设置在 $[4 \times 10^7, 6 \times 10^8]$ 的范围内。每个任务集的最大期限被设置为 $H = \sum_{i=1}^{N} M_i / f_{\max}$ 并且每个任务的截止时间可被设置为 $d_i = de_i H / \max_{\forall i}\{de_i\}$，其中 de_i 表示任务 τ_i 的深度。

5.4.2 与 MPCT 算法的性能对比

图5-1和图5-2展示了使用4.3小节中提出的基于模型预测控制的温度感知调度算法（MPCT）与本章提出的基于平均无故障时间最大化的可靠温度感知调度算法（MRTS）的性能对比。图5-1展示了不同实验案例下用两种算法调度后的系统峰值温度和所需的求解计算时间的情况，左坐标轴标注代表系统峰值温度的柱状图，右坐标轴标注代表求解时间的折线图，而图5-2展示了不同实验案例下系统平均无故障时间的情况。从图中可以明显看出，本章提出的基于系统平均无故障时间最大化的可靠温度感知调度算法在系统峰值温度、求解时间和系统平均无故障时间上都有较好的表现。

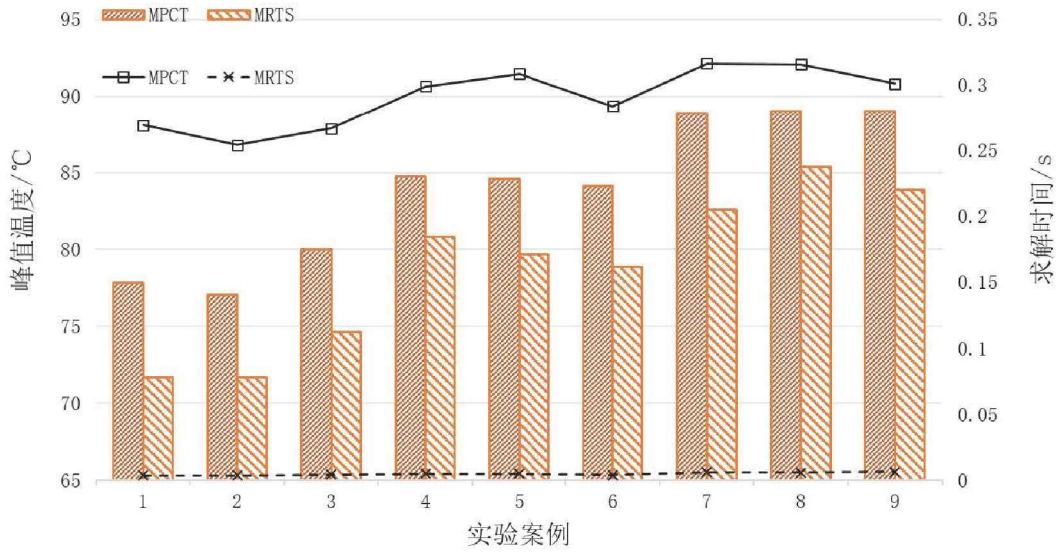


图5-1 MRTS与MPCT算法在不同实验案例下峰值温度和求解时间对比

相比于MPCT算法，本章提出的MRTS算法在系统峰值温度上平均可减少6.10%。虽然MPCT算法是基于模型预测控制的滚动优化策略，利用温度预测模型可以提前对

中国知网 https://www.cnki.net

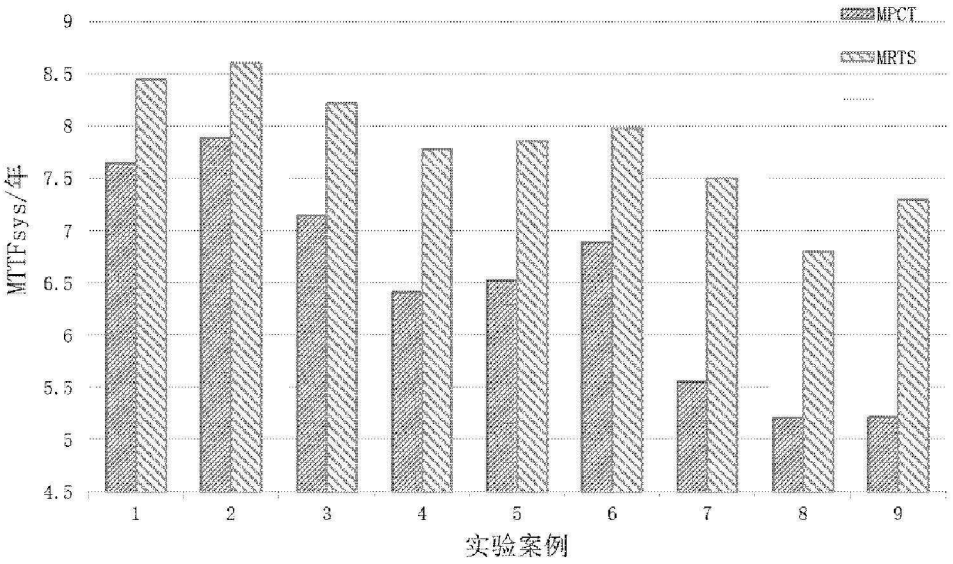


图 5-2 MRTS 与 MPCT 算法在不同实验案例下系统平均无故障时间对比

处理器温度进行优化，有效降低了处理器的峰值温度。但是 MPCT 算法考虑的是带有可选周期的近似任务并且目标是最大化系统的可选周期数而本章提出的的 MRTS 算法考虑的是只有强制周期的一般任务，从任务负载量上看，本章提出的 MRTS 算法考虑的任务负载更少，因此更容易获得更低的峰值温度。

在求解时间方面，本章提出的 MRTS 算法具有较大的提升。相比于 MPCT 算法，MRTS 算法平均可减少 98.39% 的计算时间，对每个 DAG 任务集生成基于系统平均无故障时间最大化的可靠温度感知调度方案的所需的求解时间小于 0.01s。由于 MPCT 算法使用滚动优化策略，对任务的调度实现先预测后执行的策略，并且执行区间是小于预测区间的，因此在求解过程中会消耗大量的时间来获得冗余的结果。本章提出的 MRTS 算法在确定任务的执行频率时，为了尽可能的消除热循环效应对处理器硬件的影响，讨论并证明了使任务执行时温度变化范围最小的最优频率的求解方法。并且根据任务可靠性模型和任务实时性、相关性要求，可以得到任务可执行频率的理论上的最小值，根据这个理论值和处理器上可使用的频率等级可以快速确定任务的执行频率，有效减少了确定任务执行频率的时间和步骤。此外，对于处理器上的温度优化，通过相关公式推导，对于处理器温度超过温度阈值的情况，提出了可计算空闲时间大小的空闲分布策略，可以有效快速的优化处理器的温度，使处理器的峰值温度满足阈值约束。因此，本章提出的 MRTS 算法在求解时间方面具有非常好的表现。

相比于 MPCT 算法，本章提出的 MRTS 算法在系统平均无故障时间方面可提升 20.49%。相比于仅考虑瞬时故障发生的任务可靠性问题而没有考虑永久故障发生的可靠性问题的 MPCT 算法，本章提出的 MRTS 算法在系统平均无故障时间上具有更好表现的原因，大致可分为两方面。一是因为 MRTS 算法获得的调度方案具有更低的峰值温度，而温度会影响处理器硬件的可靠性，且处理器温度越高，处理器硬件发生永久故障

中国知网　https://www.cnki.net

的概率就越高，影响处理器硬件的寿命和可靠性，系统的平均无故障时间也就越短。二是因为 MPCT 算法忽略热循环机制对处理器硬件的影响。MPCT 算法没有考虑处理器永久故障的可靠性，所以无法排除热循环机制对处理器平均无故障时间的负面影响。因此，因此，相比于 MPCT 算法，本章提出的 MRTS 算法在系统平均无故障时间方面具有更好的表现。

本章提出的 MRTS 算法考虑的是以最大化系统平均无故障时间为目标同时满足任务实时性、相关性、可靠性和温度约束的调度问题。MPCT 算法考虑的是以最大化系统 QOS 为目标同时满足任务实时性、相关性、可靠性、能量约束和温度约束的调度问题。虽然相比于 MPCT 算法，MRTS 算法在系统峰值温度、求解时间和系统平均无故障时间上都具有更好的表现，但是 MRTS 算法并不能替代 MPCT 算法，主要原因有是两个算法的研究目标不同。MPCT 算法考虑的是近似任务模型，目标是在尽可能最大化系统性能的同时保证处理器的温度约束。而 MRTS 算法的目标是尽可能的延长系统的平均无故障时间，没有考虑近似任务模型，在获得系统 QOS 方面的性能是弱于 MPCT 算法的。此外，在 MRTS 算法中，为了在任务实时性和相关性的约束下实现尽可能最大化系统 QOS 的目标，处理器上的任务需要一般以较高的频率执行，而处理器以较高频率工作时，容易增加处理器温度的增涨速度，使处理器处于温度较高的状态。而过高的温度会加速电子元件老化，增加电路永久故障发生概率增加，影响系统是寿命和可靠性，从而减少系统的平均无故障时间。因此，MRTS 算法为了实现尽可能增加系统平均无故障时间的目标，需要使处理器的温度不能过高。因此，MRTS 算法与 MPCT 算法的目标是相违背的，MRTS 算法不能替代 MPCT 算法。同理 MRTS 算法也不能替代 TATS 算法。

5.4.3 和其他算法的对比

为了更好的体现本章提出的 MRTS 算法的性能，本节实验选用了两种不同的算法作为对比。以下是这两种算法的简要介绍：

1. 基于遗传算法的最大化无平均故障时间调度算法（GA）[87]。该算法基于异构多核平台，仅考虑电迁移对系统永久故障的影响，建立系统平均无故障模型，不考虑瞬时故障。使用遗传算法获得关于 DAG 任务的以最大化系统平均无故障时间为目标的温度感知调度方案。

2. 基于模糊优势的可靠性感知调度算法（DRHEFT）[88]。该算法基于异构多核平台，同时考虑系统永久故障和瞬时故障。建立以系统瞬时故障率最小的和系统平均无故障时间最大的联合优化问题。使用模糊优势对目标问题的可行解进行排序，获得最终的调度方案。

图 5-3和图 5-4展示了本章提出的 MRTS 算法与 GA 算法和 DRHEFT 算法在不同实验案例下的性能对比。

中国知网 https://www.cnki.net

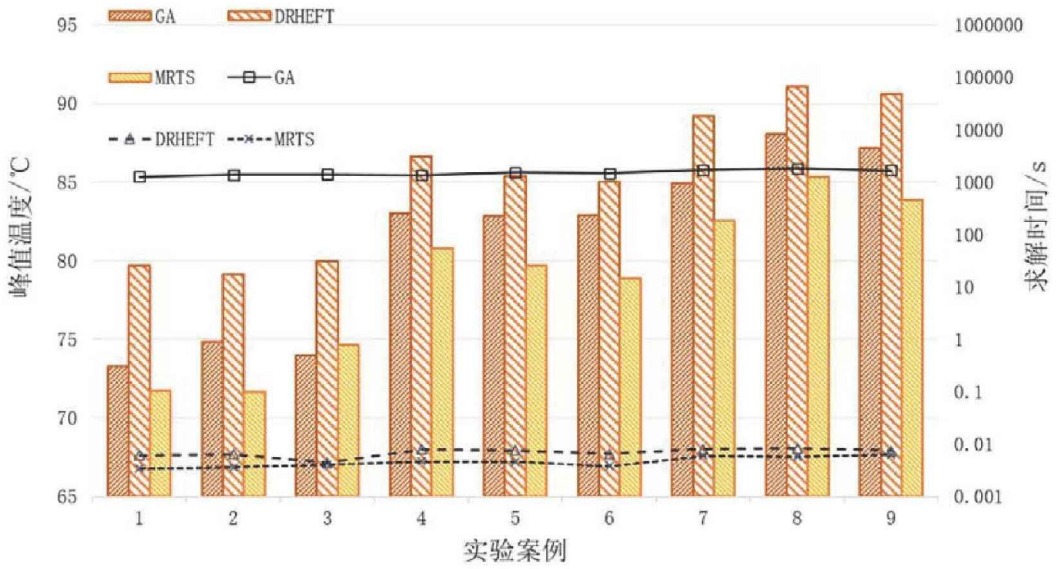


图 5-3 MRTS 与其他算法在不同实验案例下峰值温度和求解时间对比

图 5-3展示了三种算法得到的调度方案的峰值温度和求解时间的情况，左坐标轴标注代表系统峰值温度的柱状图，右坐标轴标注代表求解时间的折线图，而图 5-4展示了系统平均无故障时间的情况。从图中可以看出，本章提出的 MRTS 算法在峰值温度、求解时间和系统平均无故障时间上都有较好的表现。

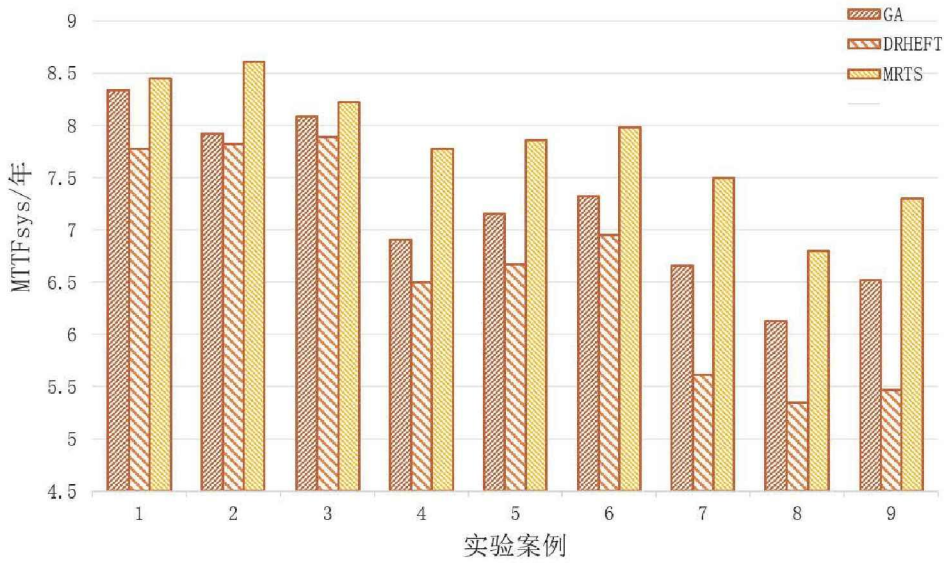


图 5-4 MRTS 与其他算法在不同实验案例下系统平均无故障时间对比

相比于 GA 算法，本章提出的 MRTS 算法在峰值温度上有略微减少，差距不大，可见图 5-3中的柱状图；在求解时间方面，由于遗传算法需要进行大量的迭代过程来找到更优的调度方案，因此 GA 算法在求解时间上消耗过多，不具备优势，可见图 5-3中的

中国知网　https://www.cnki.net

折线图。在系统平均无故障时间方面，在系统峰值温度较低时，本章提出的 MRTS 算法只略微高于 GA 算法，平均增加了 3.82% 的系统平均无故障时间，可见图 5-3和图 5-4中的案例 1、2 和 3；但是在系统峰值温度较高时，MRTS 算法在系统平均无故障时间上有更高的增益，平均可增加 11.86% 的系统平均无故障时间，可见图 5-3和图 5-4中的案例 7、8 和 9。分析造成这种差异的主要原因可能是 GA 算法没有考虑热循环效应对处理器平均无故障时间的负面影响。这种负面影响在处理器峰值温度较低时还不明显，当处理器平均温度较高时，处理器的温度差值也越大，热循环机制对处理器造成的损伤也越大。因此，在考虑处理器永久故障发生时，不能忽略热循环机制对处理器的热影响。相比于 GA 算法，本章提出的 MRTS 算法平均可减少 2.98% 的系统峰值温度、减少 100% 求解时间和增加 8.38% 的系统平均无故障时间。

相比于 DRHEFT 算法，本章提出的 MRTS 算法在降低处理器峰值温度上有较好表现，平均可减少 7.51% 的系统峰值温度。DRHEFT 算法综合考虑了系统中的瞬时故障和永久故障，将最小化瞬时故障率放入目标函数中。要想降低瞬时故障的故障率，需要提高任务的执行频率，而当处理器上的任务以更高的频率执行时容易增加处理器温度的增长速度。因此，相比于 MRTS 算法和 GA 算法，DRHEFT 算法具有更高的峰值温度。相应的，DRHEFT 算法考虑系统瞬时故障率最小化和系统平均无故障时间最大化联合优化的目标，为了保障瞬时故障率尽可能小，需要牺牲掉部分永久故障的可靠性。这种牺牲在处理器上的平均温度较高的时候，对系统平均无故障时间的负面影响更为明显，同时也是 DRHEFT 算法的得到的调度方案的系统平均无故障时间低于 MRTS 算法和 GA 算法的部分原因，可见图 5-3和 5-4中的案例 7、8 和 9。另外，DRHEFT 算法对于系统平均无故障时间的建模也只考虑了电迁移机制对处理器硬件的负面影响，忽略了处理器上的热循环机制。这也可能导致处理器的系统平均无故障时间的减少。并且在不同的处理器温度下，DRHEFT 算法出现与 GA 算法同样的现象，即随着处理器峰值温度的升高，DRHEFT 算法得到的调度方案的系统平均无故障时间与 MRTS 算法的差值变大。相比于 DRHEFT 算法，本章提出的 MRTS 算法平均可减少 7.51% 的峰值温度、减少 33.06% 求解时间和增加 17.42% 的系统平均无故障时间。

经过综合比较，本章提出的 MRTS 算法在处理器峰值温度、求解时间和系统平均无故障时间都具有良好表现。相比于使用遗传算法求解系统平均无故障时间最大化的调度方案，本章提出的算法平均可增加 8.38% 的系统平均无故障时间。相比于 DRHEFT 算法，本章提出的 MRTS 算法平均可减少 7.51% 的峰值温度、减少 33.06% 求解时间和增加 17.42% 的系统平均无故障时间。并且本章提出的方案考虑的问题更全面复杂，且更高效。

5.5 本章小结

本章研究了多核异构平台上的 DAG 任务的可靠温度感知的调度问题。同时考虑系统中瞬时故障和永久故障发生的情况。针对永久故障，综合考虑了造成永久故障发生的

中国知网 https://www.cnki.net

两种主要机制：电迁移和热循环，并使用当前准确度最高的系统平均无故障时间模型进行联合建模。针对影响处理器永久故障发生概率的温度和温度变化范围大小，讨论并证明了最小化每个任务执行的温度变化范围可以最小化整个处理器的温度变化范围，从而将问题转换为最小化任务温度变化量的问题。并且通过公式推导和计算可得到任务温度变化范围最小时的最优频率。该最优频率在选择任务执行频率时起参考作用。此外，针对处理器的峰值温度优化，通过公式推导和计算，得到了空闲时间大小可知的空闲分布策略。最终得到以系统平均无故障时间最大化为目标，满足任务实时性、相关性、可靠性和温度阈值约束的可靠温度感知调度方案。通过仿真实验对比得出，本章提出的方案在系统峰值温度、求解时间和系统平均无故障时间上有较好表现。相比于使用遗传算法求解系统平均无故障时间最大化的调度方案，本章提出的算法平均可增加 8.38% 的系统平均无故障时间。相比于 DRHEFT 算法，本章提出的 MRTS 算法平均可减少 7.51% 的峰值温度、减少 33.06% 求解时间和增加 17.42% 的系统平均无故障时间。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第六章 总结与展望

6.1 全文总结

随着半导体技术的提升，芯片的特征尺寸持续缩短，功耗密度和集成度的增加使得芯片工作时的平均温度持续上升。过高的温度不仅会影响任务的可靠执行，还会加速电子元件老化、增加电路永久故障发生的概率，影响处理器的使用寿命。另外，处理器温度的增加也会增加处理器的静态功耗，而功耗的增加又会增加处理器温度的增涨速度。处理器的温度和功耗之间存在循环相关性，通过增加芯片核心数目来增强芯片性能的模式已不可行，高温和高功耗密度已经成为限制处理器性能发展的重要因素。因此，本文基于多核处理器平台，研究实时任务的调度和温度优化策略。本文的主要贡献如下：

1） 针对同构多核平台，考虑满足任务实时性、相关性、可靠性、系统能量和温度约束的系统性能最大化的调度问题，提出了一个两阶段式的静态调度算法。在第一阶段设计了以最大化系统性能，满足任务实时性、相关性、可靠性和系统能量约束的调度方案，引入一个平衡分数来平衡任务在能量消耗大小和执行时间长短间的选择，从而选择任务最优的处理器和执行频率，得到初始的调度；在第二阶段，基于稳态温度模型，计算初始调度方案的峰值温度，对于违反温度阈值的调度，设计了结合 DVFS 和空闲分布策略的温度优化算法。该方案可以在处理器峰值温度满足阈值约束的同时不影响任务执行的实时性和可靠性。通过仿真实验，本章提出的算法相比于求解器求解的最优解，极大减少了求解时间，平均可减少 99.6% 的求解时间并且保证求得的解与最优解之间的差距在可接受范围内。相比于 DELI 算法，本章提出的算法在峰值温度上平均可降低 4.72%。

2） 研究异构多核平台上实时任务的温度感知调度问题，考虑将调度和温度优化联合设计，提出了基于模型预测控制的动态温度感知调度算法。由于传统的模型预测控制的调度方案存在模型复杂、步骤繁琐只适用于求解小规模的简单问题的不足，本章对模型预测控制中的温度预测模型做出了改进，提出了可快速估算处理器温度的模型。此外，基于传统的模型预测控制调度方案中，使用时间间隔作为步长，在滚动优化过程中，需要耗费一些不必要的时间。因此提出以任务深度作为步长的改进方案，简化了求解步骤和时间。同时，针对异构平台，在任务调度时加入热均衡策略，在同等参数选择下，选择温度更低的处理器执行任务。相比于同样使用模型预测控制进行调度的一般调度算法，本章提出的算法能大大缩减求解时间，平均可减少 98.23%，并且适用于具有相关性的实时任务，考虑的问题模型更全面。

中国知网 https://www.cnki.net

3） 研究异构多核平台上实时任务的可靠温度感知调度设计，同时考虑瞬时故障和永久故障，提出了以最大化系统平均无故障时间的温度感知调度方案。针对瞬时故障，建立任务可靠性模型；针对永久故障，综合考虑了造成永久故障发生的两种主要机制：电迁移和热循环，并使用当前准确度最高的系统平均无故障模型进行联合建模。通过分析温度对处理器寿命的影响，研究并证明了移除热循环效应的充分条件，即当处理器上的任务执行时温度变化范围最小时可以使得处理器上温度变化范围最小，从而尽可能减少热循环对处理器硬件的影响。同时，通过公式推导和计算可得到使任务温度变化范围最小时的最优频率。该最优频率在选择任务执行频率时起参考作用。此外，针对处理器的峰值温度优化问题，通过公式推导和计算，研究了空闲时间大小可知的空闲分布策略。最后通过实验对比证明了本章算法在最大化系统平均无故障时间和温度优化方面的有效性。仿真实验表明，相比于使用遗传算法求解系统平均无故障时间最大化的调度方案，本章提出的算法平均可增加 8.38% 的系统平均无故障时间。相比于其他同类算法，本章提出的算法平均可减少 7.51% 的峰值温度、减少 33.06% 求解时间和增加 17.42% 的系统平均无故障时间。

6.2 研究展望

在现有工作的基础上，本文的工作在以下几个方面还能做进一步的研究：

1） 本文忽略了处理器之间的通信，在实际调度问题中还需要考虑处理器之间的通信时间和通信能耗。当具有相关性的任务调度在不同处理器上执行时，处理器间的距离不同，传输数据的时间和能耗也不同。

2） 本文建立和推导的处理器温度计算模型，没有进行实物的实验，导致模型与真实的芯片系统可能会存在一些误差。在未来硬件条件容许的情况下，还需要通过硬件实验测试，对模型进行一定的修正。

3） 本文在研究任务的可靠性问题时，主要使用了频率调整的方法保证任务执行的可靠性，对于常用的任务复制方法没有进行研究，后续工作中，可加入任务复制策略并讨论对比两种策略的应用场景和性能。此外，系统中瞬时故障和永久故障分别对应了不同的可靠性模型，考虑是否可以提出一个统一的量化标准，可以同时结合瞬时故障和永久故障的可靠性模型。

中国知网 https://www.cnki.net

参考文献

[1] Martin C. Multicore processors: challenges, opportunities, emerging trends[C]. Proc. Embedded World Conference: volume 2014. 2014. 1.

[2] 徐海, 文张洋. 面向新一代众核处理器的高性能 SNC 的设计与验证[J]. 计算机与数字工程, 2021, 49(8):1707-1713.

[3] Zhang J, Sadiqbatcha S, Jin W, et al. Accurate power density map estimation for commercial multi-core microprocessors[C]. 2020 Design, Automation & Test in Europe Conference & Exhibition. IEEE, 2020. 1085-1090.

[4] Esmaeilzadeh H, Blem E, St. Amant R, et al. Dark silicon and the end of multicore scaling [C]. Proceedings of the 38th annual international symposium on Computer architecture. 2011. 365-376.

[5] Singh A K, Dey S, McDonald-Maier K, et al. Dynamic energy and thermal management of multi-core mobile platforms: a survey[J]. IEEE Design & Test, 2020, 37(5):25-33.

[6] 杨世贵, 王媛媛, 刘韦辰, 等. 基于强化学习的温度感知多核任务调度[J]. 软件学报, 2021, 32(8):2408-2424.

[7] Kim Y G, Kim M, Kong J, et al. An adaptive thermal management framework for heterogeneous multi-core processors[J]. IEEE Transactions on Computers, 2020, 69(6):894-906.

[8] Sheikh H F, Ahmad I, Wang Z, et al. An overview and classification of thermal-aware scheduling techniques for multi-core processing systems[J]. Sustainable Computing: Informatics and Systems, 2012, 2(3):151-169.

[9] 彭晓明, 郭浩然, 庞建民. 多核处理器——技术, 趋势和挑战[J]. 计算机科学, 2012 (S3):320-326.

[10] Hanada T, Sasaki H, Inoue K, et al. Performance evaluation of 3d stacked multi-core processors with temperature consideration[C]. 2011 IEEE International 3D Systems Integration Conference. IEEE, 2012. 1-5.

[11] Xu S, Koren I, Krishna C M. Thermal aware task scheduling for enhanced cyber-physical systems sustainability[J]. IEEE Transactions on Sustainable Computing, 2019, 5(4):581-593.

中国知网 https://www.cnki.net

[12] Cortés L A, Eles P, Peng Z. Quasi-static assignment of voltages and optional cycles in imprecise-computation systems with energy considerations[J]. IEEE transactions on very large scale integration systems, 2006, 14(10):1117-1129.

[13] Yu K, Xiao H, Zhao Y, et al. A framework for deep q-learning based hybrid dvfs algorithms for real-time systems[C]. 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking. IEEE, 2021. 900-907.

[14] da Silva E C, Gabriel P H. A comprehensive review of evolutionary algorithms for multiprocessor dag scheduling[J]. Computation, 2020, 8(2):26.

[15] Nasini S, Nessah R. A multi-machine scheduling solution for homogeneous processing: Asymptotic approximation and applications[J]. International Journal of Production Economics, 2022, 251:108555.

[16] Dhanesh L, Deepa S, Elangovan P, et al. Enhanced and energy-efficient program scheduling for heterogeneous multi-core processors system[C]. Advances in Electrical Control and Signal Systems: Select Proceedings of AECSS 2019. Springer, 2020. 737-747.

[17] Jamal M H, Chaudhry M T, Tahir U, et al. Hotspot-aware workload scheduling and server placement for heterogeneous cloud data centers[J]. Energies, 2022, 15(7):2541.

[18] Leung L F, Tsui C Y, Ki W H. Simultaneous task allocation, scheduling and voltage assignment for multiple-processors-core systems using mixed integer nonlinear programming[C]. 2003 IEEE International Symposium on Circuits and Systems: volume 5. IEEE, 2003. V-V.

[19] CHANTEM T, SHARON HU X, DICK R P. Temperature-aware scheduling and assignment for hard real-time applications on mpsocs[J]. IEEE transactions on very large scale integration systems, 2011, 19(10):1884-1897.

[20] Yang L, Liu W, Guan N, et al. Optimal application mapping and scheduling for network-on-chips with computation in stt-ram based router[J]. IEEE Transactions on Computers, 2019, 68(8):1174-1189.

[21] Wang L, Liu J, Hu J, et al. Optimal assignment for tree-structure task graph on heterogeneous multicore systems considering time constraint[C]. 2012 IEEE 6th International Symposium on Embedded Multicore SoCs. 2012. 121-127.

[22] Boyd S, Mattingley J. Branch and bound methods[J]. Notes for EE364b, Stanford University, 2007, 2006:07.

中国知网 https://www.cnki.net

[23] Zhang G, Hu Y, Sun J, et al. An improved genetic algorithm for the flexible job shop scheduling problem with multiple time constraints[J]. Swarm and Evolutionary Computation, 2020, 54:100664.

[24] Tanha M, Hosseini Shirvani M, Rahmani A M. A hybrid meta-heuristic task scheduling algorithm based on genetic and thermodynamic simulated annealing algorithms in cloud computing environments[J]. Neural Computing and Applications, 2021, 33:16951-16984.

[25] Zhou T, Lin M. Cpu frequency scheduling of real-time applications on embedded devices with temporal encoding-based deep reinforcement learning[J]. Journal of Systems Architecture, 2023, 142:102955.

[26] Zhang Q, Lin M, Yang L T, et al. Energy-efficient scheduling for real-time systems based on deep q-learning model[J]. IEEE Transactions on Sustainable Computing, 2019, 4(1): 132-141.

[27] Chang S, Zhao X, Liu Z, et al. Real-time scheduling and analysis of parallel tasks on heterogeneous multi-cores[J]. Journal of Systems Architecture, 2020, 105:101704.

[28] Mo L, Li X, Kritikakou A, et al. Contention and reliability-aware energy efficiency task mapping on noc-based mpsocs[J]. IEEE Transactions on Reliability, 2024:1-17.

[29] Dey S, Guajardo E Z, Basireddy K R, et al. Edgecoolingmode: An agent-based thermal management mechanism for DVFS enabled heterogeneous mpsocs[C]. International Conference on Embedded Systems. 2019. 19-24.

[30] Mittal S. A survey of techniques for approximate computing[J]. ACM Computing Surveys, 2016, 48(4):1-33.

[31] Mo L, Kritikakou A, Sentieys O. Energy-quality-time optimized task mapping on dvfs-enabled multicores[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(11):2428-2439.

[32] Chakraborty S, Saha S, Sjalander M, et al. RePAiR: A strategy for reducing peak temperature while maximising accuracy of approximate real-time computing: Work-in-progress [C]. International Conference on Hardware/Software Codesign and System Synthesis. 2020. 8-10.

[33] Chakraborty S, Saha S, Själander M, et al. Prepare: Power-aware approximate real-time task scheduling for energy-adaptive qos maximization[J]. ACM Transactions on Embedded Computing Systems, 2021, 20(5s):1-25.

中国知网 https://www.cnki.net

[34] Sharma Y, Chakraborty S, Moulik S. Eta-hp: an energy and temperature-aware real-time scheduler for heterogeneous platforms[J]. The Journal of Supercomputing, 2022, 78(8): 1-25.

[35] Moulik S. Reset: A real-time scheduler for energy and temperature aware heterogeneous multi-core systems[J]. Integration, 2021, 77:59-69.

[36] Sheikh H F, Ahmad I, Fan D. An evolutionary technique for performance-energy-temperature optimized scheduling of parallel tasks on multi-core processors[J]. IEEE Transactions on Parallel and Distributed Systems, 2015, 27(3):668-681.

[37] Singh A K, Dey S, McDonald-Maier K, et al. Dynamic energy and thermal management of multi-core mobile platforms: A survey[J]. IEEE Design & Test, 2020, 37(5):25-33.

[38] Ranjbar B, Nguyen T D, Ejlali A, et al. Online peak power and maximum temperature management in multi-core mixed-criticality embedded systems[C]. 2019 22nd Euromicro Conference on Digital System Design. IEEE, 2019. 546-553.

[39] Yu H, Ha Y, Wang J. Quality optimization of resilient applications under temperature constraints[C]. Proceedings of the Computing Frontiers Conference. 2017. 9-16.

[40] Yu H, Syed R, Ha Y. Thermal-aware frequency scaling for adaptive workloads on heterogeneous mpsocs[C]. 2014 Design, Automation & Test in Europe Conference & Exhibition. IEEE, 2014. 1-6.

[41] Tilli A, Garone E, Conficoni C, et al. A two-layer distributed mpc approach to thermal control of multiprocessor systems-on-chip[J]. Control Engineering Practice, 2022, 122: 105099.

[42] Zhang T, Seo M, Donyanavard B, et al. Predicting failures in embedded systems using long short-term inference[J]. IEEE Embedded Systems Letters, 2020, 13(3):85-89.

[43] Dey S, Guajardo E Z, Basireddy K R, et al. Edgecoolingmode: An agent based thermal management mechanism for dvfs enabled heterogeneous mpsocs[C]. 2019 32nd International Conference on VLSI Design and 2019 18th International Conference on Embedded Systems. IEEE, 2019. 19-24.

[44] Isuwa S, Dey S, Singh A K, et al. Teem: Online thermal-and energy-efficiency management on cpu-gpu mpsocs[C]. 2019 Design, Automation & Test in Europe Conference & Exhibition. IEEE, 2019. 438-443.

中国知网 https://www.cnki.net

[45] 张冬松, 吴彤, 陈芳园, 等. 多核系统中基于 Global EDF 的在线节能实时调度算法[J]. 软件学报, 2012, 23(4):996-1009.

[46] Zhou J, Cao K, Cong P, et al. Reliability and temperature constrained task scheduling for makespan minimization on heterogeneous multi-core platforms[J]. Journal of Systems and Software, 2017, 133:1-16.

[47] 黄柯衡, 张正鸿, 王海, 等. 基于混合优化的多核处理器动态热管理方法[J]. 电子工艺技术, 2018, 39(2):71-75.

[48] Cui M, Kritikakou A, Mo L, et al. Fault-tolerant mapping of real-time parallel applications under multiple dvfs schemes[C]. 2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium. IEEE, 2021. 387-399.

[49] Li T, Zhang T, Yu G, et al. Minimizing temperature and energy of real-time applications with precedence constraints on heterogeneous mpsoc systems[J]. Journal of Systems Architecture, 2019, 98:79-91.

[50] Tan L, Kothapalli S, Chen L, et al. A survey of power and energy efficient techniques for high performance numerical linear algebra operations[J]. Parallel Computing, 2014, 40 (10):559-573.

[51] Zhou J, Wei T, Chen M, et al. Thermal-aware task scheduling for energy minimization in heterogeneous real-time mpsoc systems[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2016, 35(8):1269-1282.

[52] Pourmohseni B, Wildermann S, Smirnov F, et al. Task migration policy for thermal-aware dynamic performance optimization in many-core systems[J]. IEEE Access, 2022, 10: 33787-33802.

[53] Sajid M, Raza Z. Energy-efficient quantum-inspired stochastic q-hype algorithm for batch-of-stochastic-tasks on heterogeneous dvfs-enabled processors[J]. Concurrency and Computation: Practice and Experience, 2019, 31(20):e5327.

[54] Ansari M, Safari S, Yari-Karin S, et al. Thermal-aware standby-sparing technique on heterogeneous real-time embedded systems[J]. IEEE Transactions on Emerging Topics in Computing, 2021, 10(4):1883-1897.

[55] 毛南, 黄岚, 王忠义, 等. 实时嵌入式容错系统的关键技术研究[J]. 计算机工程与设计, 2007, 28(14):3433-3435,3439.

中国知网 https://www.cnki.net

[56] Koser E, Berthold K, Kumar Pujari R, et al. A chip-level redundant threading (crt) scheme for shared-memory protection[C]. 2016 International Conference on High Performance Computing Simulation. 2016. 116-124.

[57] Mousavi Nik S S, Naghibzadeh M, Sedaghat Y. Task replication to improve the reliability of running workflows on the cloud[J]. Cluster Computing, 2021, 24(1):343-359.

[58] Swain C K, Saini N, Sahu A. Reliability aware scheduling of bag of real time tasks in cloud environment[J]. Computing, 2020, 102:451-475.

[59] Sun Y, Zhou S, Niu Z. Distributed task replication for vehicular edge computing: Performance analysis and learning-based algorithm[J]. IEEE Transactions on Wireless Communications, 2021, 20(2):1138-1151.

[60] Namazi A, Abdollahi M, Safari S, et al. Lrtm: Life-time and reliability-aware task mapping approach for heterogeneous multi-core systems[C]. 2018 11th International Workshop on Network on Chip Architectures. IEEE, 2018. 1-6.

[61] Miedema L, Rouxel B, Grelck C. Task-level redundancy vs instruction-level redundancy against single event upsets in real-time dag scheduling[C]. 2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip. 2021. 373-380.

[62] Hassan H A, Salem S A, Saad E M. A smart energy and reliability aware scheduling algorithm for workflow execution in dvfs-enabled cloud environment[J]. Future Generation Computer Systems, 2020, 112:431-448.

[63] Cao E, Musa S, Chen M, et al. Energy and reliability-aware task scheduling for cost optimization of dvfs-enabled cloud workflows[J]. IEEE Transactions on Cloud Computing, 2023, 11(2):2127-2143.

[64] Yeganeh-Khaksar A, Ansari M, Safari S, et al. Ring-dvfs: Reliability-aware reinforcement learning-based dvfs for real-time embedded systems[J]. IEEE Embedded Systems Letters, 2021, 13(3):146-149.

[65] Zhou J, Sun J, Zhang M, et al. Dependable scheduling for real-time workflows on cyber–physical cloud systems[J]. IEEE Transactions on Industrial Informatics, 2021, 17(11): 7820-7829.

[66] Bankar A S, Sha S, Chaturvedi V, et al. Thermal aware lifetime reliability optimization for automotive distributed computing applications[C]. 2020 IEEE 38th International Conference on Computer Design. 2020. 498-505.

中国知网 https://www.cnki.net

[67] Liu W, Yi J, Li M, et al. Energy-efficient application mapping and scheduling for lifetime guaranteed mpsocs[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 38(1):1-14.

[68] Huang J, Blech J O, Raabe A, et al. Analysis and optimization of fault-tolerant task scheduling on multiprocessor embedded systems[C]. Proceedings of the seventh IEEE/ACM/IFIP international conference on Hardware/software codesign and system synthesis. 2011. 247-256.

[69] Das A, Kumar A, Veeravalli B, et al. Combined dvfs and mapping exploration for lifetime and soft-error susceptibility improvement in mpsocs[C]. 2014 Design, Automation Test in Europe Conference Exhibition. 2014. 1-6.

[70] Zhou J, Hu X S, Ma Y, et al. Balancing lifetime and soft-error reliability to improve system availability[C]. 2016 21st Asia and South Pacific Design Automation Conference. 2016. 685-690.

[71] Mishra V, Sapatnekar S S. Probabilistic wire resistance degradation due to electromigration in power grids[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2016, 36(4):628-640.

[72] Chao S, Lin X, Yang D, et al. Study on stress migration of fcqfn package with unbalanced arrangement of bumps[C]. 2022 China Semiconductor Technology International Conference. 2022. 1-4.

[73] Kim D H, Milor L. Tddb-emerald: A methodology for estimating memory reliability degradation resulting from time-dependent dielectric breakdown[C]. 2016 Conference on Design of Circuits and Integrated Systems. 2016. 1-6.

[74] Coskun A K, Strong R, Tullsen D M, et al. Evaluating the impact of job scheduling and power management on processor lifetime for chip multiprocessors[C]. Proceedings of the eleventh international joint conference on Measurement and modeling of computer systems. 2009. 169-180.

[75] Tariq R, Aadil F, Malik M F, et al. Directed acyclic graph based task scheduling algorithm for heterogeneous systems[C]. Intelligent Systems and Applications: Proceedings of the 2018 Intelligent Systems Conference Volume 2. Springer, 2019. 936-947.

[76] El Sayed M A, Saad E S M, Aly R F, et al. Energy-efficient task partitioning for real-time scheduling on multi-core platforms[J]. Computers, 2021, 10(1):1-21.

中国知网 https://www.cnki.net

[77] Digitale J C, Martin J N, Glymour M M. Tutorial on directed acyclic graphs[J]. Journal of Clinical Epidemiology, 2022, 142:264-267.

[78] Ozceylan B, Haverkort B R, de Graaf M, et al. Minimizing the maximum processor temperature by temperature-aware scheduling of real-time tasks[J]. IEEE Transactions on Very Large Scale Integration Systems, 2022, 30(8):1084-1097.

[79] Saha S, Chakraborty S, Agarwal S, et al. Delicious: Deadline-aware approximate computing in cache-conscious multicore[J]. IEEE Transactions on Parallel and Distributed Systems, 2023, 34(2):718-733.

[80] Wang H, Wan J, Tan S X D, et al. A fast leakage-aware full-chip transient thermal estimation method[J]. IEEE Transactions on Computers, 2018, 67(5):617-630.

[81] Karamanakos P, Liegmann E, Geyer T, et al. Model predictive control of power electronic systems: Methods, results, and challenges[J]. IEEE Open Journal of Industry Applications, 2020, 1:95-114.

[82] Chen G, Huang K, Knoll A. Abstract: Energy optimization for real-time multiprocessor system-on-chip with optimal dvfs and dpm combination[C]. The 11th IEEE Symposium on Embedded Systems for Real-time Multimedia. 2013. 40-40.

[83] Tilli A, Garone E, Conficoni C, et al. A two-layer distributed mpc approach to thermal control of multiprocessor systems-on-chip[J]. Control Engineering Practice, 2022, 122: 105099.

[84] Wu K C, Lee M C, Marculescu D, et al. Mitigating lifetime underestimation: A system-level approach considering temperature variations and correlations between failure mechanisms[C]. 2012 Design, Automation & Test in Europe Conference & Exhibition. IEEE, 2012. 1269-1274.

[85] Srinivasan J, Adve S V, Bose P, et al. Lifetime reliability: Toward an architectural solution [J]. Ieee Micro, 2005, 25(3):70-80.

[86] Huang L, Yuan F, Xu Q. On task allocation and scheduling for lifetime extension of platform-based mpsoc designs[J]. IEEE Transactions on Parallel and Distributed Systems, 2011, 22(12):2088-2099.

[87] Bankar A S, Sha S, Bhimani J, et al. Thermal aware system-wide reliability optimization for automotive distributed computing applications[J]. IEEE Transactions on Vehicular Technology, 2022, 71(10):10442-10457.

中国知网 https://www.cnki.net

[88] Zhou J, Zhang M, Sun J, et al. Drheft: Deadline-constrained reliability-aware heft algorithm for real-time heterogeneous mpsoc systems[J]. IEEE Transactions on Reliability, 2022, 71(1):178-189.

中国知网 https://www.cnki.net

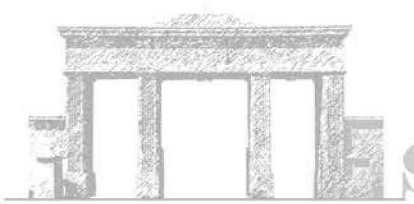
毕业/学位论文答辩委员会名单

毕业/学位论文题目		多核计算平台温度优化实时任务调度策略研究		
作　者		彭楚君		
专　业		电子信息		
研究方向		多核处理器调度和温度优化		
导　师		莫磊副教授		
答辩委员会组成	**姓　名**	**职　称**	**学科专业**	**工作单位**
	陈夕松（主席）	教授	控制科学与工程	东南大学
	余亮	教授	控制科学与工程	南京邮电大学
	聂晓凯	副教授	控制科学与工程	东南大学
	陈怡超	高级工程师	控制科学与工程	中国电子科技集团公司第二十八研究所
	吴晓娜（秘书）	助教	计算机科学与技术	东南大学

备注：
1、本表格适用于所有研究生。
2、本表格排版在终版毕业/学位论文中，附在毕业/学位论文的最后。

中国知网　https://www.cnki.net

心於至善



SOUTHEAST UNIVERSITY

中国知网 https://www.cnki.net